

Studentas projektas

1.2

Generated by Doxygen 1.9.8

Chapter 1

Hierarchical Index

1.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Skirstymorezultatas	??
TimerResults	??
Tyrimolrasas	??
Zmogus	??
Studentas	??

Chapter 2

Class Index

2.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Skirstymorezultatas	Saugo skirstymo į grupes rezultatus	??
Studentas	Konkreči klasė, paveldinti iš Zmogus	??
TimerResults	Saugo kiekvienos programos fazės laiko matavimus (sekundėmis)	??
Tirimolrasas		??
Zmogus	Abstrakti bazinė klasė, apibrėžianti žmogaus sąsają	??

Chapter 3

File Index

3.1 File List

Here is a list of all files with brief descriptions:

Generavimas.cpp	??
Generavimas.h	
Studentų failų generavimo ir skirstymo į grupes funkcijų deklaracijos	??
kontaineris.h	
Kompiliavimo laiku pasirenkamas studentų kontaineris	??
laikai.cpp	??
laikai.h	
Laiko matavimų struktūra ir pagalbinės funkcijos	??
main.cpp	??
menu.cpp	??
menu.h	
Pagrindinio programos menu funkcijos deklaracija	??
skaitymas.cpp	??
skaitymas.h	
Studentų duomenų skaitymo funkcijų deklaracijos	??
spausdinam.cpp	??
spausdinam.h	
Studentų duomenų rikiavimo ir išvedimo funkcijų deklaracijos	??
Studentas.cpp	
Studentas klasės metodų realizacija	??
Studentas.h	
Studento klasės apibrėžimas	??
testas.cpp	??
testas.h	??
tyrimas.cpp	??
tyrimas.h	
Konteinerių ir skirstymo strategijų greیتaveikos tyrimo funkcijos	??
unit_tests.cpp	
Google Test unit testai Studentas klasei	??
Zmogus.cpp	??
Zmogus.h	
Abstrakti bazinė klasė žmogui	??

Chapter 4

Class Documentation

4.1 Skirstymorezultatas Struct Reference

Saugo skirstymo į grupes rezultatus.

```
#include <Generavimas.h>
```

Public Attributes

- [StudContainer](#) vargsiukai
- [StudContainer](#) protai

4.1.1 Detailed Description

Saugo skirstymo į grupes rezultatus.

Definition at line [41](#) of file [Generavimas.h](#).

4.1.2 Member Data Documentation

4.1.2.1 protai

[StudContainer](#) Skirstymorezultatas::protai

Definition at line [44](#) of file [Generavimas.h](#).

Referenced by [skirstymasGrupes\(\)](#), [skirstymasStrategija1\(\)](#), [skirstymasStrategija2\(\)](#), and [skirstymasStrategija3\(\)](#).

4.1.2.2 vargsiukai

`StudContainer Skirstymorezultatas::vargsiukai`

Definition at line 43 of file [Generavimas.h](#).

Referenced by [skirstymasGrupes\(\)](#), [skirstymasStrategija1\(\)](#), [skirstymasStrategija2\(\)](#), and [skirstymasStrategija3\(\)](#).

The documentation for this struct was generated from the following file:

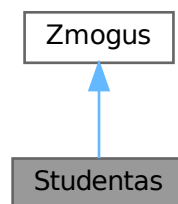
- [Generavimas.h](#)

4.2 Studentas Class Reference

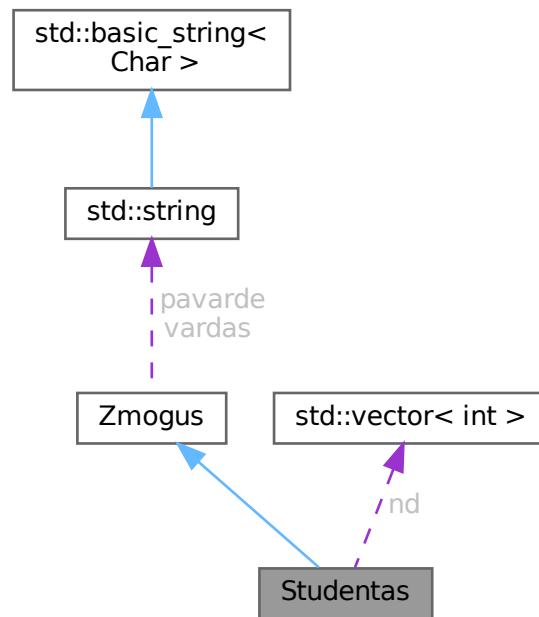
Konkreiti klasė, paveldinti iš [Zmogus](#).

```
#include <Studentas.h>
```

Inheritance diagram for Studentas:



Collaboration diagram for Studentas:



Public Member Functions

Rule of Five

- `Studentas ()`
Numatytasis konstruktorius.
- `Studentas (std::istream &is)`
Srautinis konstruktorius.
- `Studentas (const Studentas &other)`
Kopijavimo konstruktorius.
- `Studentas & operator= (const Studentas &other)`
Kopijavimo priskyrimo operatorius.
- `Studentas (Studentas &&other)`
Perkėlimo konstruktorius.
- `Studentas & operator= (Studentas &&other)`
Perkėlimo priskyrimo operatorius.
- `~Studentas ()` override=default
Destruktorius.
- `const std::vector< int > & getND () const`
Grąžina namų darbų pažymių vektorių (tik skaitymui).
- `int getEgzaminas () const`
Grąžina egzamino pažymį.
- `float getVidurkis () const`
Grąžina namų darbų pažymių vidurkį.
- `float getgalrezVid () const`
Grąžina galutinį balą apskaičiuotą pagal vidurkį.
- `float getgalrezMed () const`
Grąžina galutinį balą apskaičiuotą pagal medianą.

Keitimo metodai (setters)

- void [setVardas](#) (const std::string &v)
Nustato studento vardą.
- void [setPavarde](#) (const std::string &p)
Nustato studento pavardę.
- void [setEgzaminas](#) (int e)
Nustato egzamino pažymį.

Funkcijos

- std::istream & [readStudent](#) (std::istream &is)
Nuskaito studento duomenis iš srauto.
- void [readNdInteractive](#) ()
Interaktyviai įveda namų darbų pažymius iš konsolės.
- void [parinktiAtsitiktinius](#) ()
Sugeneruoja atsitiktinius namų darbų ir egzamino pažymius.
- void [suskaiciuotiGalutini](#) () override
Apskaičiuoja galutinį įvertinimą (vidurkio ir medianos variantus).
- void [spausdinti](#) (std::ostream &os) const override
Išveda studento duomenis į nurodytą srautą.

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()=default
Numatytasis konstruktorius.
- [Zmogus](#) (const std::string &v, const std::string &p)
Konstruktorius su parametrais.
- [Zmogus](#) (const [Zmogus](#) &)=default
Kopijavimo konstruktorius.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &)=default
Kopijavimo priskyrimo operatorius.
- [Zmogus](#) ([Zmogus](#) &&)=default
Perkėlimo konstruktorius.
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&)=default
Perkėlimo priskyrimo operatorius.
- virtual [~Zmogus](#) ()=default
Virtualus destruktorius.
- virtual std::string [getVardas](#) () const
- virtual std::string [getPavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)

Private Attributes

- int [egzaminas](#)
- std::vector< int > [nd](#)
- float [vidurkis](#)
- float [galrezVid](#)
- float [galrezMed](#)

Related Symbols

(Note that these are not member symbols.)

Laisvosios funkcijos

- `std::ostream & operator<<` (`std::ostream &os`, `const Studentas &s`)
Išvesties operatorius srautui.
- `std::istream & operator>>` (`std::istream &is`, `Studentas &s`)
Išvesties operatorius srautui.
- `bool comparePagalVarda` (`const Studentas &a`, `const Studentas &b`)
Lyginimo funkcija rikiavimui pagal vardą.
- `bool comparePagalPavarde` (`const Studentas &a`, `const Studentas &b`)
Lyginimo funkcija rikiavimui pagal pavardę.
- `bool comparePagalGalVid` (`const Studentas &a`, `const Studentas &b`)
Lyginimo funkcija rikiavimui pagal galutinį vidurkio balą.
- `bool comparePagalGalMed` (`const Studentas &a`, `const Studentas &b`)
Lyginimo funkcija rikiavimui pagal galutinį medianos balą.

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- `std::string vardas`
- `std::string pavarde`

4.2.1 Detailed Description

Konkreči klasė, paveldinti iš [Zmogus](#).

Saugo ir apdoroja studento namų darbų pažymius, egzamino rezultata, skaičiuoja galutinius įvertinimus (pagal vidurkį ir medianą).

4.2.1.0.1 Rule of Five

Klasė realizuoja visus penkis specialiuosius metodus:

- Numatytąjį konstruktorių
- Kopijavimo konstruktorių
- Kopijavimo priskyrimo operatorių
- Perkėlimo konstruktorių
- Perkėlimo priskyrimo operatorių
- Destruktorių (paveldi iš [Zmogus](#))

4.2.1.0.2 Galutinio balo formulė

$$galrezVid = 0.4 \cdot vidurkis + 0.6 \cdot egzaminas$$

$$galrezMed = 0.4 \cdot mediana + 0.6 \cdot egzaminas$$

Definition at line 46 of file [Studentas.h](#).

4.2.2 Constructor & Destructor Documentation

4.2.2.1 Studentas() [1/4]

```
Studentas::Studentas ( )
```

Numatytasis konstruktorius.

Inicializuoja visus skaičinius laukus į 0, o vektorių – į tuščią.

Inicializuoja skaičinius laukus į 0.

Definition at line 24 of file [Studentas.cpp](#).

4.2.2.2 Studentas() [2/4]

```
Studentas::Studentas (
    std::istream & is ) [explicit]
```

Srautinis konstruktorius.

Nuskaito studento duomenis iš nurodyto įvesties srauto (pvz. failo eilutės pateiktos kaip `std::istringstream`).

Parameters

<i>is</i>	Įvesties srautas.
-----------	-------------------

Nuskaito eilutę iš srauto per [readStudent\(\)](#).

Parameters

<i>is</i>	Įvesties srautas (pvz. <code>std::istringstream</code>).
-----------	---

Definition at line 34 of file [Studentas.cpp](#).

References [readStudent\(\)](#).

Here is the call graph for this function:



4.2.2.3 Studentas() [3/4]

```
Studentas::Studentas (
    const Studentas & other )
```

Kopijavimo konstruktorius.

Sukuria gilią kopiją kito `Studentas` objekto.

Parameters

<code>other</code>	Kopijuojamas objektas.
--------------------	------------------------

Sukuria gilią kopiją.

Parameters

<code>other</code>	Kopijuojamas objektas.
--------------------	------------------------

Definition at line 44 of file `Studentas.cpp`.

4.2.2.4 Studentas() [4/4]

```
Studentas::Studentas (
    Studentas && other )
```

Perkėlimo konstruktorius.

Perkelia resursus iš `other` į šį objektą. Po perkėlimo `other` lieka tuščios būsenos.

Parameters

<code>other</code>	Perkeliamas objektas (rvalue nuoroda).
--------------------	--

Po perkėlimo šaltinio skaičiniai laukai nulinami, `nd` vektorius perkeliamas.

Parameters

<i>other</i>	Perkeliamas objektas (rvalue).
--------------	--------------------------------

Definition at line 84 of file [Studentas.cpp](#).

4.2.2.5 ~Studentas()

```
Studentas::~~Studentas ( ) [override], [default]
```

Destruktorius.

Paveldi iš [Zmogus](#). Automatiškai atlaisvina nd vektoriaus atmintį.

4.2.3 Member Function Documentation

4.2.3.1 getEgzaminas()

```
int Studentas::getEgzaminas ( ) const [inline]
```

Grąžina egzamino pažymį.

Returns

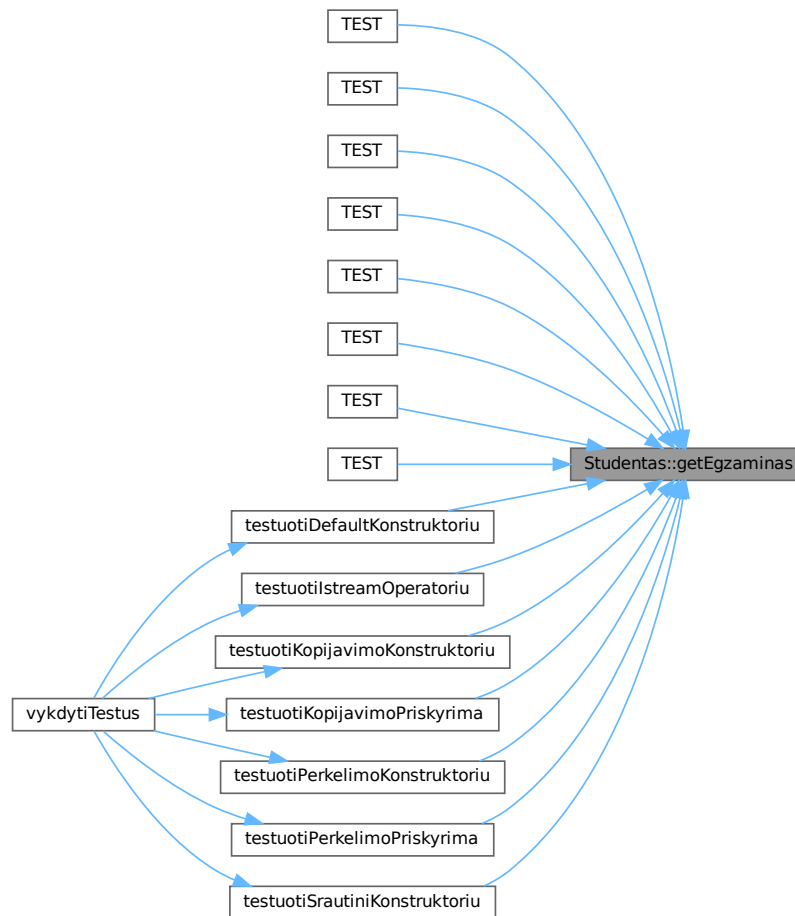
Egzamino pažymys (1–10).

Definition at line 135 of file [Studentas.h](#).

References [egzaminas](#).

Referenced by [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [testuotiDefaultKonstruktoriu\(\)](#), [testuotiIstreamOperatoriu\(\)](#), [testuotiKopijavimoKonstruktoriu\(\)](#), [testuotiKopijavimoPriskyrima\(\)](#), [testuotiPerkelimoKonstruktoriu\(\)](#), [testuotiPerkelimoPriskyrima\(\)](#), and [testuotiSrautiniKonstruktoriu\(\)](#).

Here is the caller graph for this function:



4.2.3.2 getgalrezMed()

```
float Studentas::getgalrezMed ( ) const [inline]
```

Grąžina galutinį balą apskaičiuotą pagal medianą.

Returns

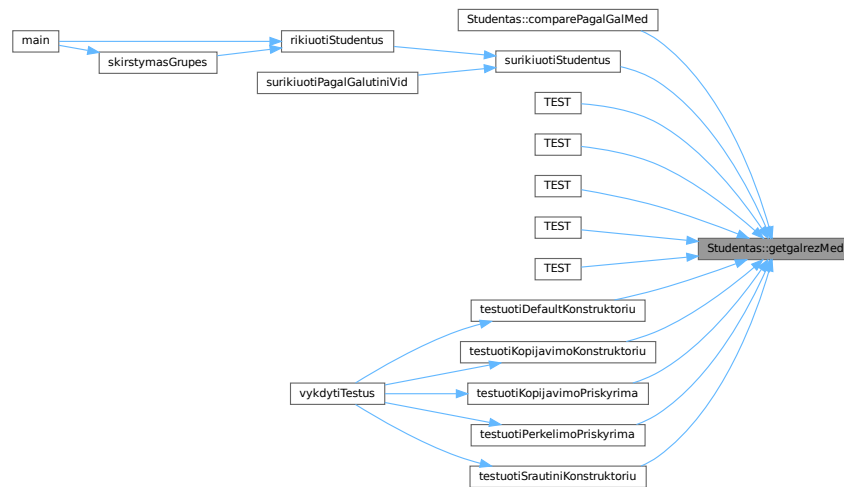
Galutinis balas (medianos variantas).

Definition at line 153 of file [Studentas.h](#).

References [galrezMed](#).

Referenced by [comparePagalGalMed\(\)](#), [surikiuotiStudentus\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [testuotiDefaultKonstruktoriu\(\)](#), [testuotiKopijavimoKonstruktoriu\(\)](#), [testuotiKopijavimoPriskyrima\(\)](#), [testuotiPerkelimoPriskyrima\(\)](#), and [testuotiSrautiniKonstruktoriu\(\)](#).

Here is the caller graph for this function:



4.2.3.3 getgalrezVid()

```
float Studentas::getgalrezVid ( ) const [inline]
```

Grąžina galutinį balą apskaičiuotą pagal vidurkį.

Returns

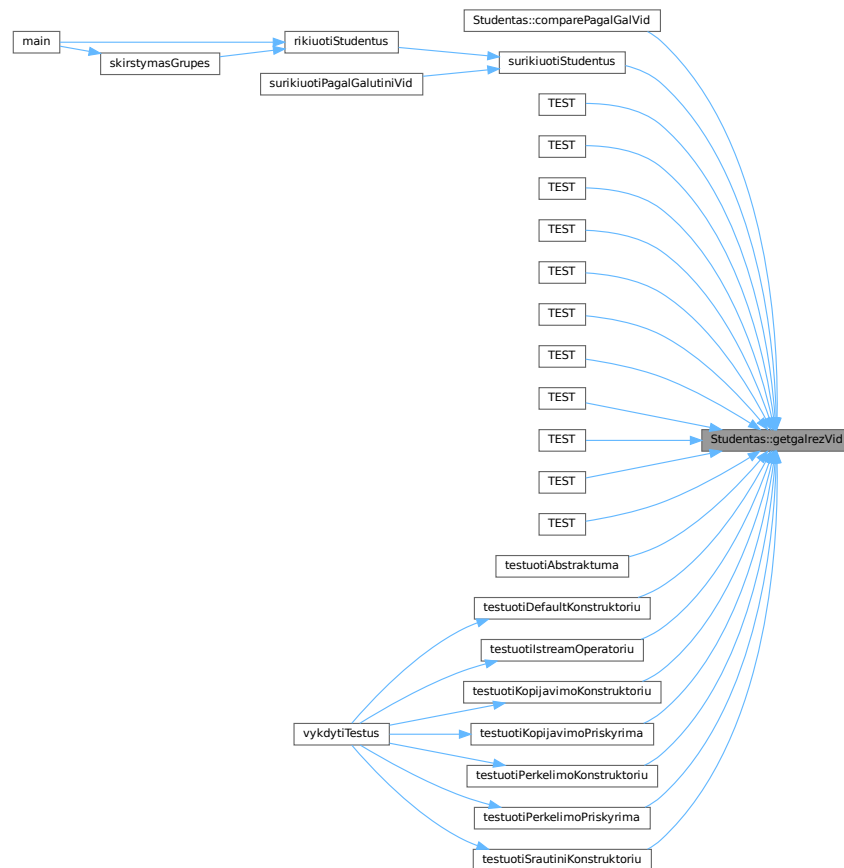
Galutinis balas (vidurkio variantas).

Definition at line 147 of file [Studentas.h](#).

References [galrezVid](#).

Referenced by [comparePagalGalVid\(\)](#), [surikiuotiStudentus\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [testuotiAbstraktuma\(\)](#), [testuotiDefaultKonstruktoriu\(\)](#), [testuotiIstreamOperatoriu\(\)](#), [testuotiKopijavimoKonstruktoriu\(\)](#), [testuotiKopijavimoPriskyrima\(\)](#), [testuotiPerkelimoKonstruktoriu\(\)](#), [testuotiPerkelimoPriskyrima\(\)](#), and [testuotiSrautiniKonstruktoriu\(\)](#).

Here is the caller graph for this function:



4.2.3.4 getND()

```
const std::vector< int > & Studentas::getND ( ) const [inline]
```

Grąžina namų darbų pažymių vektorį (tik skaitymui).

Returns

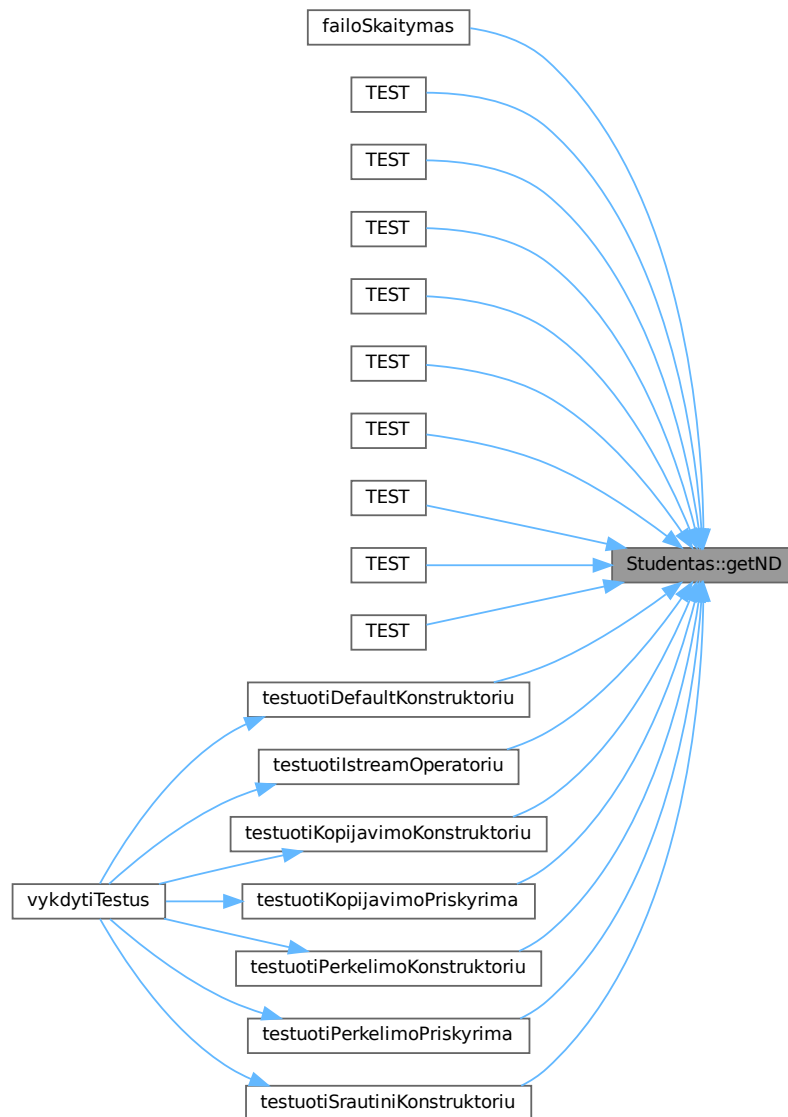
Konstantinė nuoroda į `nd` vektorių.

Definition at line 129 of file [Studentas.h](#).

References [nd](#).

Referenced by [failoSkaitymas\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [testuotiDefaultKonstruktoriu\(\)](#), [testuotiIstreamOperatoriu\(\)](#), [testuotiKopijavimoKonstruktoriu\(\)](#), [testuotiKopijavimoPriskyrima\(\)](#), [testuotiPerkelimoKonstruktoriu\(\)](#), [testuotiPerkelimoPriskyrima\(\)](#), and [testuotiSrautiniKonstruktoriu\(\)](#).

Here is the caller graph for this function:



4.2.3.5 getVidurkis()

```
float Studentas::getVidurkis ( ) const [inline]
```

Grąžina namų darbų pažymių vidurkį.

Returns

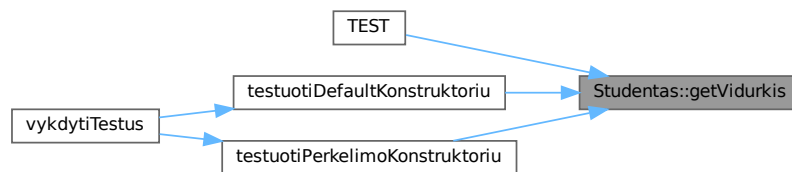
Vidurkis kaip `float`.

Definition at line 141 of file [Studentas.h](#).

References [vidurkis](#).

Referenced by [TEST\(\)](#), [testuotiDefaultKonstruktoriu\(\)](#), and [testuotiPerkelimoKonstruktoriu\(\)](#).

Here is the caller graph for this function:



4.2.3.6 operator=() [1/2]

```
Studentas & Studentas::operator= (
    const Studentas & other )
```

Kopijavimo priskyrimo operatorius.

Nukopijuoja visus duomenis iš `other` į šį objektą. Saugiai tvarko savipriskyrimo atvejį.

Parameters

<i>other</i>	Priskiriamas objektas.
--------------	------------------------

Returns

Nuoroda į šį objektą (`*this`).

Naudoja savipriskyrimo apsaugą (`if (this == &other)`).

Parameters

<i>other</i>	Priskiriamas objektas.
--------------	------------------------

Returns

Nuoroda į šį objektą.

Definition at line 60 of file [Studentas.cpp](#).

References [egzaminas](#), [galrezMed](#), [galrezVid](#), [nd](#), [Zmogus::operator=\(\)](#), and [vidurkis](#).

Here is the call graph for this function:



4.2.3.7 operator=() [2/2]

```
Studentas & Studentas::operator= (
    Studentas && other )
```

Perkėlimo priskyrimo operatorius.

Perkelia resursus iš `other` į šį objektą priskyrimo metu. Saugiai tvarko savipriskyrimo atvejį.

Parameters

<i>other</i>	Perkeliamas objektas (rvalue nuoroda).
--------------	--

Returns

Nuoroda į šį objektą (*`this`).

Savipriskyrimo apsauga + resursų perkėlimas.

Parameters

<i>other</i>	Perkeliamas objektas (rvalue).
--------------	--------------------------------

Returns

Nuoroda į šį objektą.

Definition at line 104 of file [Studentas.cpp](#).

References [egzaminas](#), [galrezMed](#), [galrezVid](#), [nd](#), [Zmogus::operator=\(\)](#), and [vidurkis](#).

Here is the call graph for this function:



4.2.3.8 parinktiAtsitiktinius()

```
void Studentas::parinktiAtsitiktinius ( )
```

Sugeneruoja atsitiktinius namų darbų ir egzamino pažymius.

Naudoja `std::mt19937` generatorių su `std::random_device` sėkla. Pažymiai generuojami intervale [1, 10].

Naudoja `std::mt19937` su `std::random_device` sėkla. Pažymiai – intervale [1, 10].

Definition at line 312 of file [Studentas.cpp](#).

References [egzaminas](#), [nd](#), and [vidurkis](#).

Referenced by [skaitomRanka\(\)](#), and [vykdytiMeniu\(\)](#).

Here is the caller graph for this function:



4.2.3.9 readNdInteractive()

```
void Studentas::readNdInteractive ( )
```

Interaktyviai įveda namų darbų pažymius iš konsolės.

Interaktyviai įveda namų darbų pažymius.

Prašo vartotojo įvesti pažymius vieną po kito. Įvedimas baigiamas naudojant 0 arba neigiamą skaičių.

Exceptions

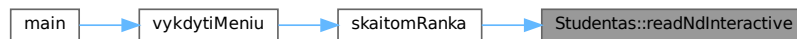
<code>std::runtime_error</code>	Jei nepavyksta nuskaityti nė vieno pažymio.
<code>std::runtime_error</code>	Jei neįvestas nė vienas pažymys.

Definition at line 266 of file [Studentas.cpp](#).

References [nd](#), and [vidurkis](#).

Referenced by [skaitomRanka\(\)](#).

Here is the caller graph for this function:



4.2.3.10 readStudent()

```
std::istream & Studentas::readStudent (
    std::istream & is )
```

Nuskaityto studento duomenis iš srauto.

Nuskaityto studento duomenis iš srauto (vidinė funkcija).

Formatas: Vardas Pavardė ND1 ND2 ... NDn Egzaminas Paskutinis skaičius laikomas egzamino pažymiu.

Parameters

<i>is</i>	Įvesties srautas.
-----------	-------------------

Returns

Nuoroda į srautą (grandininiam kvietimui).

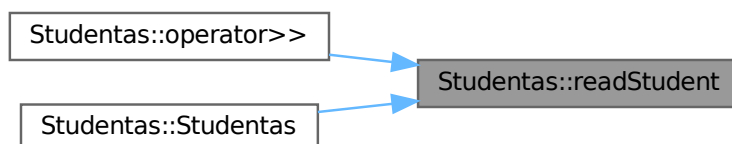
Formatas: Vardas Pavardė ND1 ND2 ... NDn Egzaminas Paskutinis skaičius laikomas egzamino pažymiu.

Definition at line 220 of file [Studentas.cpp](#).

References [egzaminas](#), [nd](#), [Zmogus::pavarde](#), [Zmogus::vardas](#), and [vidurkis](#).

Referenced by [operator>>\(\)](#), and [Studentas\(\)](#).

Here is the caller graph for this function:



4.2.3.11 setEgzaminas()

```
void Studentas::setEgzaminas (
    int e ) [inline]
```

Nustato egzamino pažymį.

Parameters

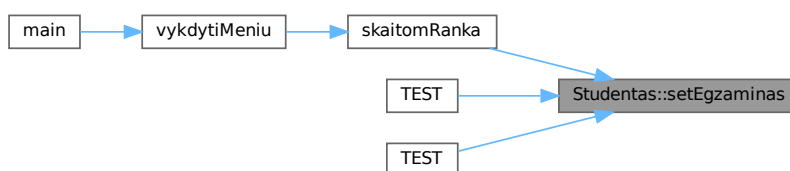
<i>e</i>	Egzamino pažymys (rekomenduojama 1–10).
----------	---

Definition at line 177 of file [Studentas.h](#).

References [egzaminas](#).

Referenced by [skaitomRanka\(\)](#), [TEST\(\)](#), and [TEST\(\)](#).

Here is the caller graph for this function:



4.2.3.12 setPavarde()

```
void Studentas::setPavarde (
    const std::string & p ) [inline]
```

Nustato studento pavardę.

Parameters

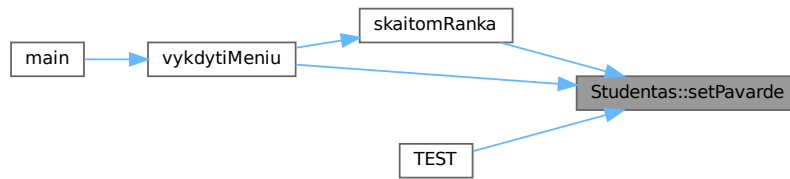
<i>p</i>	Nauja pavardė.
----------	----------------

Definition at line 171 of file [Studentas.h](#).

References [Zmogus::pavarde](#).

Referenced by [skaitomRanka\(\)](#), [TEST\(\)](#), and [vykdytiMeniu\(\)](#).

Here is the caller graph for this function:



4.2.3.13 setVardas()

```
void Studentas::setVardas (
    const std::string & v ) [inline]
```

Nustato studento vardą.

Parameters

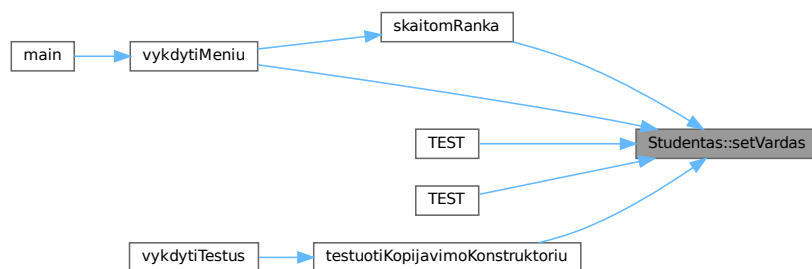
v	Naujas vardas.
---	----------------

Definition at line 165 of file [Studentas.h](#).

References [Zmogus::vardas](#).

Referenced by [skaitomRanka\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [testuotiKopijavimoKonstruktoriu\(\)](#), and [vykdytiMenu\(\)](#).

Here is the caller graph for this function:



4.2.3.14 spausdinti()

```
void Studentas::spausdinti (
    std::ostream & os ) const [override], [virtual]
```

Išveda studento duomenis į nurodytą srautą.

Išveda studento duomenis į srautą.

Realizuoja grynąją virtualią funkciją iš [Zmogus](#).

Parameters

<code>os</code>	Išvesties srautas.
-----------------	--------------------

Realizuoja `Zmogus::spausdinti()`.

Parameters

<code>os</code>	Išvesties srautas.
-----------------	--------------------

Implements `Zmogus`.

Definition at line 133 of file `Studentas.cpp`.

References `egzaminas`, `galrezMed`, `galrezVid`, `nd`, `Zmogus::pavarde`, and `Zmogus::vardas`.

Referenced by `operator<<()`.

Here is the caller graph for this function:



4.2.3.15 suskaiciuotiGalutini()

```
void Studentas::suskaiciuotiGalutini ( ) [override], [virtual]
```

Apskaičiuoja galutinį įvertinimą (vidurkio ir medianos variantus).

Apskaičiuoja galutinį įvertinimą.

Realizuoja grynąją virtualią funkciją iš `Zmogus`.

Formulė:

- $\text{galrezVid} = 0.4 * \text{vidurkis} + 0.6 * \text{egzaminas}$
- $\text{galrezMed} = 0.4 * \text{mediana} + 0.6 * \text{egzaminas}$

Realizuoja `Zmogus::suskaiciuotiGalutini()`.

Formulė:

- $\text{galrezVid} = 0.4 * \text{vidurkis} + 0.6 * \text{egzaminas}$
- $\text{galrezMed} = 0.4 * \text{mediana} + 0.6 * \text{egzaminas}$

Mediana skaičiuojama iš surūšiuoto `nd` vektoriaus kopijos.

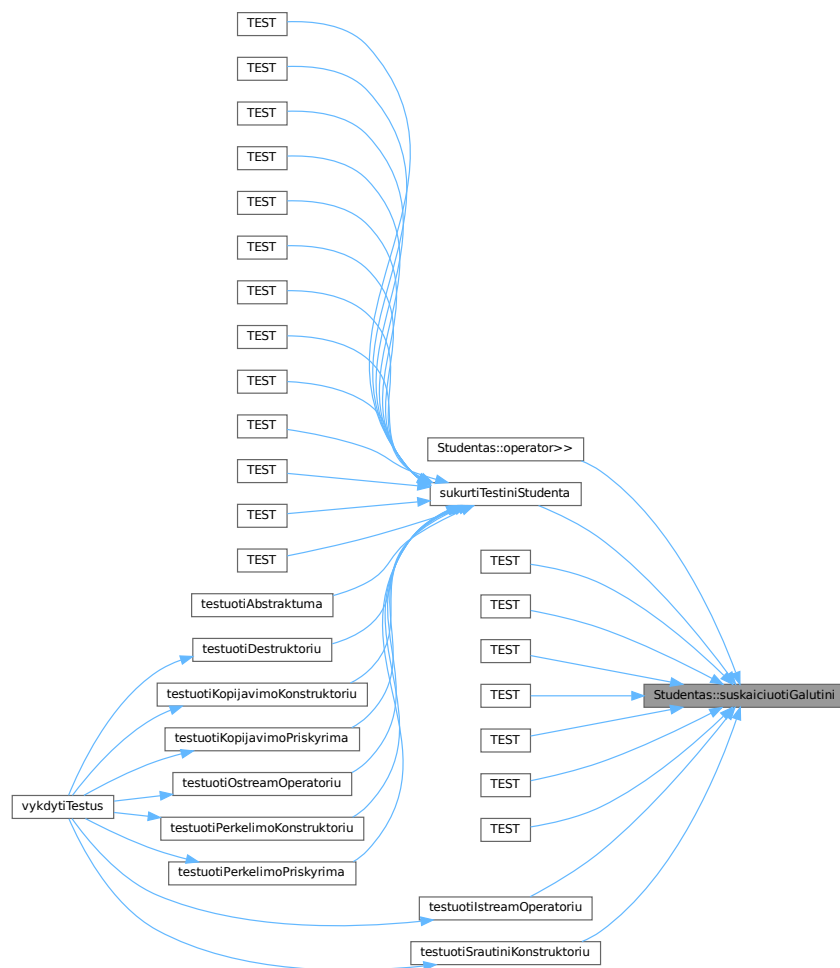
Implements [Zmogus](#).

Definition at line 162 of file [Studentas.cpp](#).

References [egzaminas](#), [galrezMed](#), [galrezVid](#), [nd](#), and [vidurkis](#).

Referenced by [operator>>\(\)](#), [sukurtiTestiniStudenta\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [testuotiIstreamOperatoriu\(\)](#), and [testuotiSrautiniKonstruktoriu\(\)](#).

Here is the caller graph for this function:



4.2.4 Friends And Related Symbol Documentation

4.2.4.1 comparePagalGalMed()

```

bool comparePagalGalMed (
    const Studentas & a,
    const Studentas & b ) [related]

```

Lyginimo funkcija rikiavimui pagal galutinį medianos balą.

Parameters

<i>a</i>	Pirmas studentas.
<i>b</i>	Antras studentas.

Returns

```
true jei a.galrezMed < b.galrezMed.
```

Definition at line 352 of file [Studentas.cpp](#).

References [getgalrezMed\(\)](#).

Here is the call graph for this function:



4.2.4.2 comparePagalGalVid()

```
bool comparePagalGalVid (  
    const Studentas & a,  
    const Studentas & b ) [related]
```

Lyginimo funkcija rikiavimui pagal galutinį vidurkio balą.

Parameters

<i>a</i>	Pirmas studentas.
<i>b</i>	Antras studentas.

Returns

```
true jei a.galrezVid < b.galrezVid.
```

Definition at line 346 of file [Studentas.cpp](#).

References [getgalrezVid\(\)](#).

Here is the call graph for this function:



4.2.4.3 comparePagalPavarde()

```
bool comparePagalPavarde (
    const Studentas & a,
    const Studentas & b ) [related]
```

Lyginimo funkcija rikiavimui pagal pavardę.

Parameters

<i>a</i>	Pirmas studentas.
<i>b</i>	Antras studentas.

Returns

true jei `a.pavarde < b.pavarde`.

Definition at line [340](#) of file [Studentas.cpp](#).

References [Zmogus::getPavarde\(\)](#).

Here is the call graph for this function:



4.2.4.4 comparePagalVarda()

```
bool comparePagalVarda (
    const Studentas & a,
    const Studentas & b ) [related]
```

Lyginimo funkcija rikiavimui pagal vardą.

Parameters

<i>a</i>	Pirmas studentas.
<i>b</i>	Antras studentas.

Returns

true jei `a.vardas < b.vardas`.

Definition at line [334](#) of file [Studentas.cpp](#).

References [Zmogus::getVardas\(\)](#).

Here is the call graph for this function:



4.2.4.5 operator<<()

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & s ) [related]
```

Išvesties operatorius srautui.

Delegacija į [Studentas::spausdinti\(\)](#).

Parameters

<i>os</i>	Išvesties srautas.
<i>s</i>	Studento objektas.

Returns

Nuoroda į srautą.

Definition at line [196](#) of file [Studentas.cpp](#).

References [spausdinti\(\)](#).

Here is the call graph for this function:



4.2.4.6 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s ) [related]
```

Ivesties operatorius srautui.

Nuskaityto studento duomenis ir iškart apskaičiuoja galutinį įvertinimą.

Parameters

<i>is</i>	Ivesties srautas.
<i>s</i>	Studento objektas, į kurį rašoma.

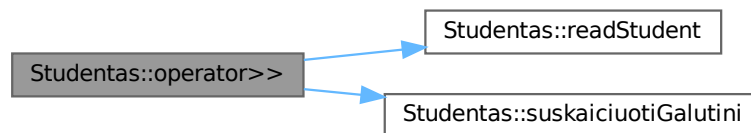
Returns

Nuoroda į srautą.

Definition at line 207 of file [Studentas.cpp](#).

References [readStudent\(\)](#), and [suskaiciuotiGalutini\(\)](#).

Here is the call graph for this function:



4.2.5 Member Data Documentation

4.2.5.1 egzaminas

```
int Studentas::egzaminas [private]
```

Definition at line 49 of file [Studentas.h](#).

Referenced by [getEgzaminas\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [parinktiAtsitiktinius\(\)](#), [readStudent\(\)](#), [setEgzaminas\(\)](#), [spausdinti\(\)](#), and [suskaiciuotiGalutini\(\)](#).

4.2.5.2 galrezMed

```
float Studentas::galrezMed [private]
```

Definition at line 53 of file [Studentas.h](#).

Referenced by [getgalrezMed\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [spausdinti\(\)](#), and [suskaiciuotiGalutini\(\)](#).

4.2.5.3 galrezVid

```
float Studentas::galrezVid [private]
```

Definition at line 52 of file [Studentas.h](#).

Referenced by [getgalrezVid\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [spausdinti\(\)](#), and [suskaiciuotiGalutini\(\)](#).

4.2.5.4 nd

```
std::vector<int> Studentas::nd [private]
```

Definition at line 50 of file [Studentas.h](#).

Referenced by [getND\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [parinktiAtsitiktinius\(\)](#), [readNdInteractive\(\)](#), [readStudent\(\)](#), [spausdinti\(\)](#), and [suskaiciuotiGalutini\(\)](#).

4.2.5.5 vidurkis

```
float Studentas::vidurkis [private]
```

Definition at line 51 of file [Studentas.h](#).

Referenced by [getVidurkis\(\)](#), [operator=\(\)](#), [operator=\(\)](#), [parinktiAtsitiktinius\(\)](#), [readNdInteractive\(\)](#), [readStudent\(\)](#), and [suskaiciuotiGalutini\(\)](#).

The documentation for this class was generated from the following files:

- [Studentas.h](#)
- [Studentas.cpp](#)

4.3 TimerResults Struct Reference

Saugo kiekvienos programos fazės laiko matavimus (sekundėmis).

```
#include <laikai.h>
```

Public Attributes

- double [generavimas](#) = 0.0
- double [skaitymas](#) = 0.0
- double [rusiavimas](#) = 0.0
- double [skirstymas](#) = 0.0
- double [isvedimas](#) = 0.0

4.3.1 Detailed Description

Saugo kiekvienos programos fazės laiko matavimus (sekundėmis).

Definition at line 20 of file [laikai.h](#).

4.3.2 Member Data Documentation

4.3.2.1 generavimas

```
double TimerResults::generavimas = 0.0
```

Definition at line 22 of file [laikai.h](#).

Referenced by [generuotiFaila\(\)](#).

4.3.2.2 isvedimas

```
double TimerResults::isvedimas = 0.0
```

Definition at line 26 of file [laikai.h](#).

Referenced by [skirstymasGrupes\(\)](#).

4.3.2.3 rusiavimas

```
double TimerResults::rusiavimas = 0.0
```

Definition at line 24 of file [laikai.h](#).

Referenced by [surikiuotiStudentus\(\)](#).

4.3.2.4 skaitymas

```
double TimerResults::skaitymas = 0.0
```

Definition at line 23 of file [laikai.h](#).

Referenced by [failoSkaitymas\(\)](#).

4.3.2.5 skirstymas

```
double TimerResults::skirstymas = 0.0
```

Definition at line 25 of file [laikai.h](#).

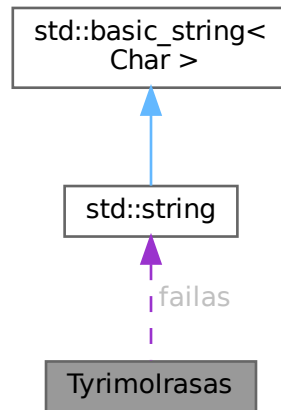
Referenced by [skirstymasGrupes\(\)](#).

The documentation for this struct was generated from the following file:

- [laikai.h](#)

4.4 Tyrimolrasas Struct Reference

Collaboration diagram for Tyrimolrasas:



Public Attributes

- `std::string failas`
- `std::size_t irasuKiekis = 0`
- `double skaitymas = 0.0`
- `double rusiavimas = 0.0`
- `double skirstymas = 0.0`

4.4.1 Detailed Description

Definition at line 16 of file [tyrimas.cpp](#).

4.4.2 Member Data Documentation

4.4.2.1 failas

```
std::string TyrimoIrasas::failas
```

Definition at line 18 of file [tyrimas.cpp](#).

4.4.2.2 irasuKiekis

```
std::size_t TyrimoIrasas::irasuKiekis = 0
```

Definition at line 19 of file [tyrimas.cpp](#).

4.4.2.3 rusiavimas

```
double TyrimoIrasas::rusiavimas = 0.0
```

Definition at line 21 of file [tyrimas.cpp](#).

4.4.2.4 skaitymas

```
double TyrimoIrasas::skaitymas = 0.0
```

Definition at line 20 of file [tyrimas.cpp](#).

4.4.2.5 skirstymas

```
double TyrimoIrasas::skirstymas = 0.0
```

Definition at line 22 of file [tyrimas.cpp](#).

The documentation for this struct was generated from the following file:

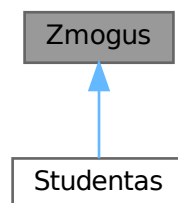
- [tyrimas.cpp](#)

4.5 Zmogus Class Reference

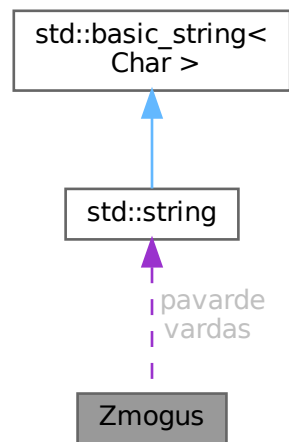
Abstrakti bazinė klasė, apibrėžianti žmogaus sąsają.

```
#include <Zmogus.h>
```

Inheritance diagram for Zmogus:



Collaboration diagram for Zmogus:



Public Member Functions

Rule of Five

- [Zmogus](#) ()=default
Numatytasis konstruktorius.
- [Zmogus](#) (const std::string &v, const std::string &p)
Konstruktorius su parametrais.
- [Zmogus](#) (const [Zmogus](#) &)=default
Kopijavimo konstruktorius.
- [Zmogus](#) & [operator=](#) (const [Zmogus](#) &)=default
Kopijavimo priskyrimo operatorius.
- [Zmogus](#) ([Zmogus](#) &&)=default
Perkėlimo konstruktorius.
- [Zmogus](#) & [operator=](#) ([Zmogus](#) &&)=default
Perkėlimo priskyrimo operatorius.
- virtual [~Zmogus](#) ()=default
Virtualus destruktorius.
- virtual std::string [getVardas](#) () const
- virtual std::string [getPavarde](#) () const
- void [setVardas](#) (const std::string &v)
- void [setPavarde](#) (const std::string &p)

Grynosios virtualios funkcijos

Išvestinės klasės privalo šias funkcijas realizuoti.

- virtual void [suskaiciuotiGalutini](#) ()=0
Apskaiciuoja galutinį įvertinimą.
- virtual void [spausdinti](#) (std::ostream &os) const =0
Išspausdina objekto duomenis į srautą.

Protected Attributes

- `std::string` [vardas](#)
- `std::string` [pavarde](#)

4.5.1 Detailed Description

Abstrakti bazinė klasė, apibrėžianti žmogaus sąsają.

Saugo vardą ir pavardę, bei deklaruoja grynąsias virtualias funkcijas, kurias privalo realizuoti išvestinės klasės. Realizuoja Rule of Five naudojant `= default` direktyvą.

Definition at line [27](#) of file [Zmogus.h](#).

4.5.2 Constructor & Destructor Documentation

4.5.2.1 Zmogus() [1/4]

```
Zmogus::Zmogus ( ) [default]
```

Numatytasis konstruktorius.

Inicializuoja tuščius laukus.

4.5.2.2 Zmogus() [2/4]

```
Zmogus::Zmogus (
    const std::string & v,
    const std::string & p )
```

Konstruktorius su parametrais.

Parameters

<i>v</i>	Vardas.
<i>p</i>	Pavardė.

Definition at line [3](#) of file [Zmogus.cpp](#).

4.5.2.3 Zmogus() [3/4]

```
Zmogus::Zmogus (
    const Zmogus & ) [default]
```

Kopijavimo konstruktorius.

4.5.2.4 Zmogus() [4/4]

```
Zmogus::Zmogus (
    Zmogus && ) [default]
```

Perkėlimo konstruktorius.

4.5.2.5 ~Zmogus()

```
virtual Zmogus::~~Zmogus ( ) [virtual], [default]
```

Virtualus destruktorius.

Užtikrina teisingą išvestinių klasių sunaikinimą per bazinės klasės rodyklę.

4.5.3 Member Function Documentation

4.5.3.1 getPavarde()

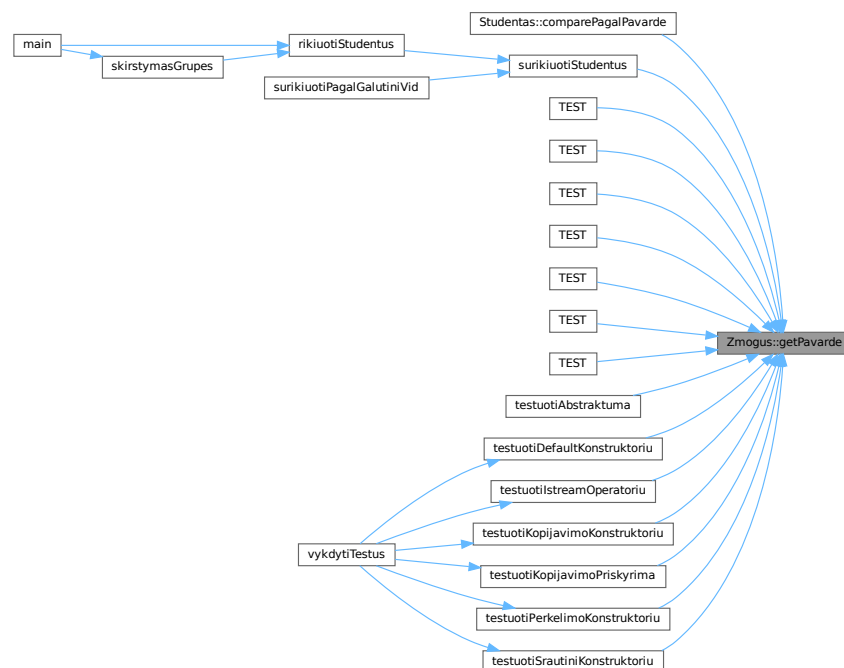
```
virtual std::string Zmogus::getPavarde ( ) const [inline], [virtual]
```

Definition at line 70 of file [Zmogus.h](#).

References [pavarde](#).

Referenced by [Studentas::comparePagalPavarde\(\)](#), [surikiuotiStudentus\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [testuotiAbstraktuma\(\)](#), [testuotiDefaultKonstruktoriu\(\)](#), [testuotiIstreamOperatoriu\(\)](#), [testuotiKopijavimoKonstruktoriu\(\)](#), [testuotiKopijavimoPriskyrima\(\)](#), [testuotiPerkelimoKonstruktoriu\(\)](#), and [testuotiSrautiniKonstruktoriu\(\)](#).

Here is the caller graph for this function:



4.5.3.2 getVardas()

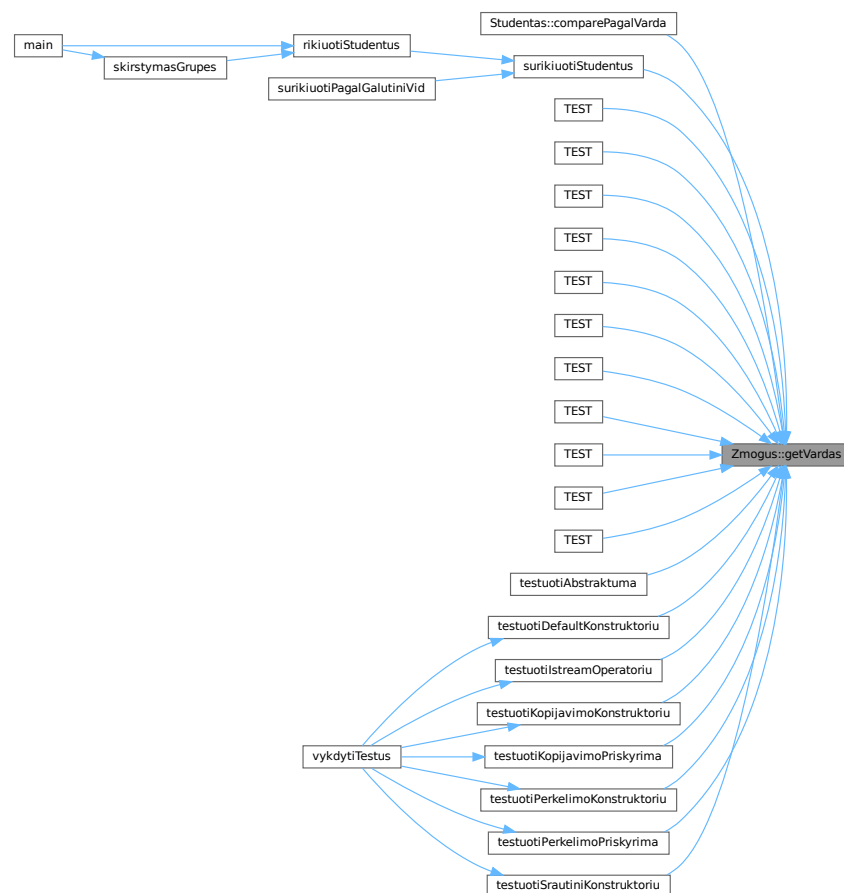
```
virtual std::string Zmogus::getVardas ( ) const [inline], [virtual]
```

Definition at line 69 of file [Zmogus.h](#).

References [vardas](#).

Referenced by [Studentas::comparePagalVarda\(\)](#), [surikiuotiStudentus\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [testuotiAbstraktuma\(\)](#), [testuotiDefaultKonstruktoriu\(\)](#), [testuotiIstreamOperatoriu\(\)](#), [testuotiKopijavimoKonstruktoriu\(\)](#), [testuotiKopijavimoPriskyrima\(\)](#), [testuotiPerkelimoKonstruktoriu\(\)](#), [testuotiPerkelimoPriskyrima\(\)](#), and [testuotiSrautiniKonstruktoriu\(\)](#).

Here is the caller graph for this function:



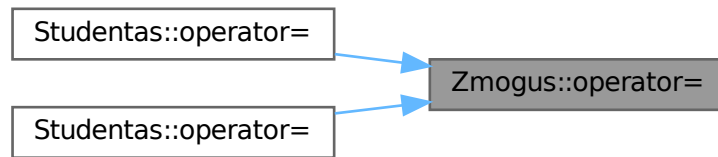
4.5.3.3 operator=() [1/2]

```
Zmogus & Zmogus::operator= (
    const Zmogus & ) [default]
```

Kopijavimo priskyrimo operatorius.

Referenced by [Studentas::operator=\(\)](#), and [Studentas::operator=\(\)](#).

Here is the caller graph for this function:



4.5.3.4 operator=() [2/2]

```
Zmogus & Zmogus::operator= (
    Zmogus && ) [default]
```

Perkėlimo priskyrimo operatorius.

4.5.3.5 setPavarde()

```
void Zmogus::setPavarde (
    const std::string & p ) [inline]
```

Definition at line 73 of file [Zmogus.h](#).

References [pavarde](#).

4.5.3.6 setVardas()

```
void Zmogus::setVardas (
    const std::string & v ) [inline]
```

Definition at line 72 of file [Zmogus.h](#).

References [vardas](#).

4.5.3.7 spausdinti()

```
virtual void Zmogus::spausdinti (
    std::ostream & os ) const [pure virtual]
```

Išspausdina objekto duomenis į srautą.

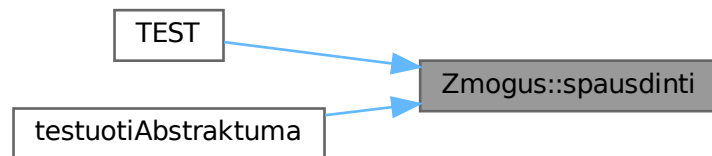
Parameters

<code>os</code>	Išvesties srautas.
-----------------	--------------------

Implemented in [Studentas](#).

Referenced by [TEST\(\)](#), and [testuotiAbstraktuma\(\)](#).

Here is the caller graph for this function:



4.5.3.8 suskaiciuotiGalutini()

```
virtual void Zmogus::suskaiciuotiGalutini ( ) [pure virtual]
```

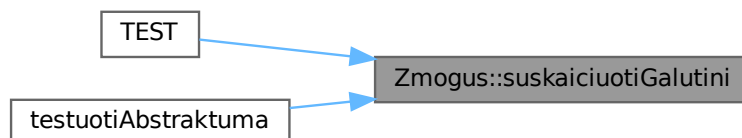
Apskaičiuoja galutinį įvertinimą.

Konkretus skaičiavimo algoritmas priklauso nuo išvestinės klasės.

Implemented in [Studentas](#).

Referenced by [TEST\(\)](#), and [testuotiAbstraktuma\(\)](#).

Here is the caller graph for this function:



4.5.4 Member Data Documentation

4.5.4.1 pavarde

`std::string Zmogus::pavarde` [protected]

Definition at line 31 of file [Zmogus.h](#).

Referenced by [getPavarde\(\)](#), [Studentas::readStudent\(\)](#), [Studentas::setPavarde\(\)](#), [setPavarde\(\)](#), and [Studentas::spausdinti\(\)](#).

4.5.4.2 vardas

`std::string Zmogus::vardas` [protected]

Definition at line 30 of file [Zmogus.h](#).

Referenced by [getVardas\(\)](#), [Studentas::readStudent\(\)](#), [Studentas::setVardas\(\)](#), [setVardas\(\)](#), and [Studentas::spausdinti\(\)](#).

The documentation for this class was generated from the following files:

- [Zmogus.h](#)
- [Zmogus.cpp](#)

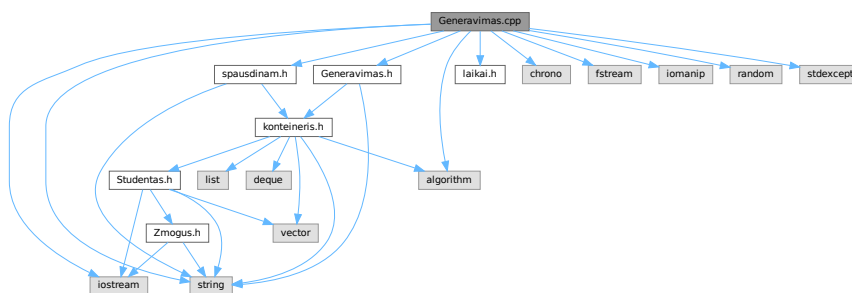
Chapter 5

File Documentation

5.1 Generavimas.cpp File Reference

```
#include "Generavimas.h"
#include "laikai.h"
#include "spausdinam.h"
#include <algorithm>
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
#include <random>
#include <stdexcept>
#include <string>
```

Include dependency graph for Generavimas.cpp:



Functions

- static bool [galimasPavadinimas](#) (const std::string &pav)
- std::string [failoPavadinimas](#) (const std::string &bazinisPavadinimas)
Sugeneruoja unikaly failo pavadinimą.
- [SkirstymoStrategija pasirinktiStrategija](#) ()
Interaktyviai prašo vartotojo pasirinkti skirstymo strategiją.
- [Skirstymorezultatas skirstymasStrategija1](#) (const [StudContainer](#) &studis)
Skirsto studentus pagal 1-ą strategiją.

- [Skirstymorezultatas skirstymasStrategija2](#) ([StudContainer](#) &[studis](#))
Skirsto studentus pagal 2-ą strategiją.
- [Skirstymorezultatas skirstymasStrategija3](#) ([StudContainer](#) &[studis](#))
Skirsto studentus pagal 3-ą strategiją (efektyviausia).
- [Skirstymorezultatas skirstymasGrupes](#) ([StudContainer](#) &[studis](#), [SkirstymoStrategija](#) strategija, [bool](#) irasyti↔
[IFailus](#), [bool](#) paprasytiRikiavimo)
Bendro naudojimo skirstymo funkcija.
- void [generuotiFaila](#) ()
Generuoja studentų duomenų failą.

5.1.1 Function Documentation

5.1.1.1 failoPavadinimas()

```
std::string failoPavadinimas (
    const std::string & bazinisPavadinimas )
```

Sugeneruoja unikalų failo pavadinimą.

Jei failas su nurodytu pavadinimu jau egzistuoja, automatiškai pridedamas skaitinis sufiksas (pvz. _2, _3).

Parameters

<i>bazinisPavadinimas</i>	Bazinis failo pavadinimas be plėtinio.
---------------------------	--

Returns

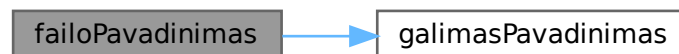
Unikalus failo pavadinimas su `.txt` plėtiniu.

Definition at line 22 of file [Generavimas.cpp](#).

References [galimasPavadinimas\(\)](#).

Referenced by [skirstymasGrupes\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



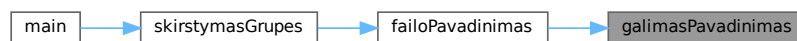
5.1.1.2 galimasPavadinimas()

```
static bool galimasPavadinimas (  
    const std::string & pav ) [static]
```

Definition at line 16 of file [Generavimas.cpp](#).

Referenced by [failoPavadinimas\(\)](#).

Here is the caller graph for this function:



5.1.1.3 generuotiFaila()

```
void generuotiFaila ( )
```

Generuoja studentų duomenų failą.

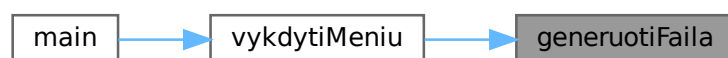
Interaktyviai prašo vartotojo įvesti failo pavadinimą ir studentų kiekį. Studentų vardai, pavardės ir pažymiai generuojami automatiškai.

Definition at line 174 of file [Generavimas.cpp](#).

References [TimerResults::generavimas](#), and [timers](#).

Referenced by [vykdytiMenu\(\)](#).

Here is the caller graph for this function:



5.1.1.4 pasirinktiStrategija()

```
SkirstymoStrategija pasirinktiStrategija ( )
```

Interaktyviai prašo vartotojo pasirinkti skirstymo strategiją.

Išveda meniu į konsolę ir nuskaito vartotojo pasirinkimą.

Returns

Pasirinkta SkirstymoStrategija.

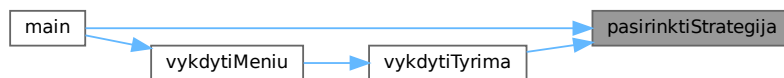
Exceptions

<code>std::runtime_error</code>	Jei vartotojo įvestis neteisinga.
---------------------------------	-----------------------------------

Definition at line 36 of file [Generavimas.cpp](#).

Referenced by [main\(\)](#), and [vykdytiTyrima\(\)](#).

Here is the caller graph for this function:



5.1.1.5 skirstymasGrupes()

```
Skirstymorezultatas skirstymasGrupes (
    StudContainer & studis,
    SkirstymoStrategija strategija,
    bool irasytiIFailus = true,
    bool paprasytiRikiavimo = true )
```

Bendro naudojimo skirstymo funkcija.

Išsaugo laiko matavimus į globalų `timers` objektą. Pagal parametrus gali rašyti į failus ir prašyti rikiavimo pasirinkimo.

Parameters

<i>studis</i>	Studentų konteineris.
<i>strategija</i>	Naudojama skirstymo strategija.
<i>irasytiIFailus</i>	Jei <code>true</code> , rezultatai išsaugomi į failus.
<i>paprasytiRikiavimo</i>	Jei <code>true</code> , vartotojas gali pasirinkti rikiavimą.

Returns

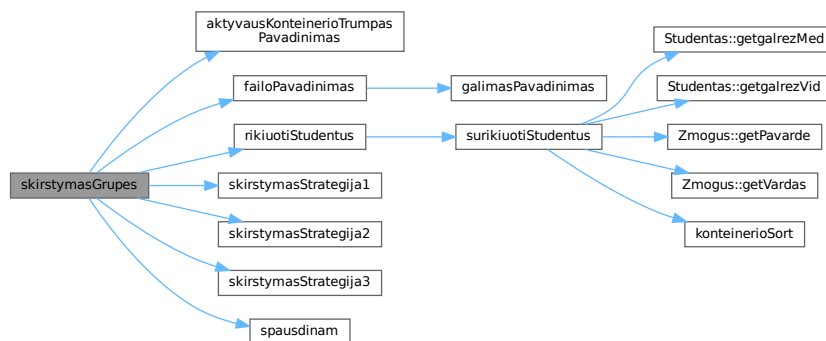
`Skirstymorezultatas` su vargiukais ir protais.

Definition at line 112 of file `Generavimas.cpp`.

References `aktyvausKonteinerioTrumpasPavadinimas()`, `Antra`, `failoPavadinimas()`, `TimerResults::isvedimas`, `Pirma`, `Skirstymorezultatas::protai`, `rikiuotiStudentus()`, `TimerResults::skirstymas`, `skirstymasStrategija1()`, `skirstymasStrategija2()`, `skirstymasStrategija3()`, `spausdinam()`, `timers`, `Trecia`, and `Skirstymorezultatas::vargsiukai`.

Referenced by `main()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.1.1.6 skirstymasStrategija1()

```
Skirstymorezultatas skirstymasStrategija1 (
    const StudContainer & studis )
```

Skirsto studentus pagal 1-ą strategiją.

Eina per visus studentus ir juos kopijuoja į atskirus konteinerius. Originalus konteineris nekeičiamas.

Parameters

<code>studis</code>	Studentų konteineris (const – nekeičiamas).
---------------------	---

Returns

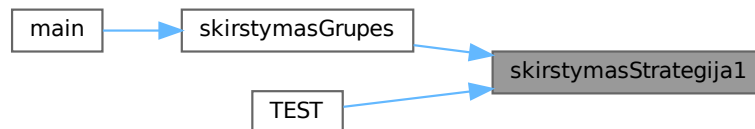
`Skirstymorezultatas` su dviem užpildytais konteineriais.

Definition at line 54 of file [Generavimas.cpp](#).

References [Skirstymorezultatas::protai](#), and [Skirstymorezultatas::vargsiukai](#).

Referenced by [skirstymasGrupes\(\)](#), and [TEST\(\)](#).

Here is the caller graph for this function:

**5.1.1.7 skirstymasStrategija2()**

```
Skirstymorezultatas skirstymasStrategija2 (
    StudContainer & studis )
```

Skirsto studentus pagal 2-ą strategiją.

Vargsiukai ištraukiami iš originalaus konteinerio (erase). Likę studentai (protai) perkeliama į rezultatą.

Parameters

<code>studis</code>	Studentų konteineris (keičiamas – iš jo šalinami vargsiukai).
---------------------	---

Returns

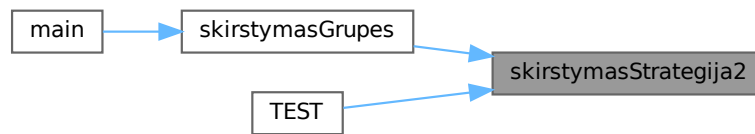
`Skirstymorezultatas`.

Definition at line 73 of file [Generavimas.cpp](#).

References [Skirstymorezultatas::protai](#), and [Skirstymorezultatas::vargsiukai](#).

Referenced by [skirstymasGrupes\(\)](#), and [TEST\(\)](#).

Here is the caller graph for this function:



5.1.1.8 skirstymasStrategija3()

```
Skirstymorezultatas skirstymasStrategija3 (  
    StudContainer & studis )
```

Skirsto studentus pagal 3-ą strategiją (efektyviausia).

Naudoja `std::partition` algoritmą, kuris perskirsto elementus vietoje, tuomet siunčia į du atskirus konteinerius.

Parameters

<code>studis</code>	Studentų konteineris (keičiamas).
---------------------	-----------------------------------

Returns

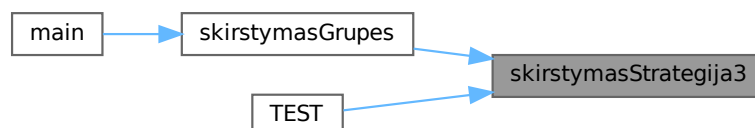
`Skirstymorezultatas`.

Definition at line 94 of file [Generavimas.cpp](#).

References [Skirstymorezultatas::protai](#), and [Skirstymorezultatas::vargsiukai](#).

Referenced by [skirstymasGrupes\(\)](#), and [TEST\(\)](#).

Here is the caller graph for this function:



5.2 Generavimas.cpp

[Go to the documentation of this file.](#)

```

00001 #include "Generavimas.h"
00002
00003 #include "laikai.h"
00004 #include "spausdinam.h"
00005
00006 #include <algorithm>
00007 #include <chrono>
00008 #include <fstream>
00009 #include <iomanip>
00010 #include <iostream>
00011 #include <random>
00012 #include <stdexcept>
00013 #include <string>
00014
00015
00016 static bool galimasPavadinimas(const std::string& pav)
00017 {
00018     std::ifstream f(pav);
00019     return f.is_open();
00020 }
00021
00022 std::string failoPavadinimas(const std::string& bazinisPavadinimas)
00023 {
00024     std::string pavadinimas = bazinisPavadinimas + ".txt";
00025     int count = 1;
00026
00027     while (galimasPavadinimas(pavadinimas))
00028     {
00029         count++;
00030         pavadinimas = bazinisPavadinimas + "_" + std::to_string(count) + ".txt";
00031     }
00032
00033     return pavadinimas;
00034 }
00035
00036 SkirstymoStrategija pasirinktiStrategija()
00037 {
00038     std::cout << "\nPasirinkite studentu skirstymo strategija:\n"
00039               << "1 - Du nauji konteineriai (vargsiukai ir protai)\n"
00040               << "2 - Tik vargsiuku konteineris; protus palikti bendrame\n"
00041               << "3 - Efektyvi strategija su std::partition\n";
00042
00043     int p = 0;
00044     std::cin >> p;
00045
00046     if (!std::cin || p < 1 || p > 3)
00047     {
00048         throw std::runtime_error("Neteisingas strategijos pasirinkimas.");
00049     }
00050
00051     return static_cast<SkirstymoStrategija>(p);
00052 }
00053
00054 Skirstymorezultatas skirstymasStrategija1(const StudContainer& studis)
00055 {
00056     Skirstymorezultatas rezultatas;
00057
00058     for (const auto& s : studis)
00059     {
00060         if (s.getgalrezVid() < 5.0f)
00061         {
00062             rezultatas.vargsiukai.push_back(s);
00063         }
00064         else
00065         {
00066             rezultatas.protai.push_back(s);
00067         }
00068     }
00069
00070     return rezultatas;
00071 }
00072
00073 Skirstymorezultatas skirstymasStrategija2(StudContainer& studis)
00074 {
00075     Skirstymorezultatas rezultatas;
00076
00077     for (auto i = studis.begin(); i != studis.end(); )
00078     {
00079         if (i->getgalrezVid() < 5.0f)
00080         {
00081             rezultatas.vargsiukai.push_back(*i);
00082             i = studis.erase(i);

```

```

00083     }
00084     else
00085     {
00086         i++;
00087     }
00088 }
00089
00090 rezultatas.protai = std::move(studis);
00091 return rezultatas;
00092 }
00093
00094 Skirstymorezultatas skirstymasStrategija3(StudContainer& studis)
00095 {
00096     Skirstymorezultatas rezultatas;
00097
00098     auto riba = std::partition(studis.begin(), studis.end(),
00099         [](const Studentas& s)
00100         {
00101             return s.getgalrezVid() >= 5.0f;
00102         });
00103
00104     rezultatas.vargsiukai.assign(riba, studis.end());
00105     rezultatas.protai.assign(studis.begin(), riba);
00106
00107     studis.clear();
00108
00109     return rezultatas;
00110 }
00111
00112 Skirstymorezultatas skirstymasGrupes(
00113     StudContainer& studis,
00114     SkirstymoStrategija strategija,
00115     bool irasytiIFailus,
00116     bool paprasytiRikiavimo)
00117 {
00118     auto start = std::chrono::high_resolution_clock::now();
00119
00120     Skirstymorezultatas rezultatas;
00121
00122     switch (strategija)
00123     {
00124     case SkirstymoStrategija::Pirma:
00125     {
00126         rezultatas = skirstymasStrategija1(studis);
00127         break;
00128     }
00129     case SkirstymoStrategija::Antra:
00130     {
00131         rezultatas = skirstymasStrategija2(studis);
00132         break;
00133     }
00134     case SkirstymoStrategija::Trecia:
00135     {
00136         rezultatas = skirstymasStrategija3(studis);
00137         break;
00138     }
00139     }
00140
00141     auto end = std::chrono::high_resolution_clock::now();
00142     timers.skirstymas = std::chrono::duration<double>(end - start).count();
00143
00144     if (!irasytiIFailus)
00145     {
00146         timers.isvedimas = 0.0;
00147         return rezultatas;
00148     }
00149
00150     if (paprasytiRikiavimo)
00151     {
00152         rikiuotiStudentus(rezultatas.vargsiukai, "vargsiukus");
00153         rikiuotiStudentus(rezultatas.protai, "protus");
00154     }
00155
00156     auto startPrint = std::chrono::high_resolution_clock::now();
00157
00158     const std::string sufiksas =
00159         "_" + aktyvausKonteinerioTrumpasPavadinimas() +
00160         "_S" + std::to_string(static_cast<int>(strategija));
00161
00162     spausdinam(rezultatas.vargsiukai, failoPavadinimas("Vargsiukai" + sufiksas));
00163     spausdinam(rezultatas.protai, failoPavadinimas("protai" + sufiksas));
00164
00165     auto endPrint = std::chrono::high_resolution_clock::now();
00166     timers.isvedimas = std::chrono::duration<double>(endPrint - startPrint).count();
00167
00168     std::cout << "Studentai suskirstyti. Vargsiuku: " << rezultatas.vargsiukai.size()
00169         << ", protu: " << rezultatas.protai.size() << std::endl;

```

```

00170
00171     return rezultatas;
00172 }
00173
00174 void generuotiFaila()
00175 {
00176     std::cout << "Iveskite failo pavadinima: ";
00177     std::string pav;
00178     std::cin >> pav;
00179
00180     std::ofstream write(pav);
00181     if (!write.is_open())
00182     {
00183         std::cerr << "Nepavyko sukurti failo." << std::endl;
00184         return;
00185     }
00186
00187     std::mt19937 gen(std::random_device{}());
00188     std::uniform_int_distribution<int> dist(1, 10);
00189
00190     std::cout << "Iveskite studentu kiekis: ";
00191     int kiekis = 0;
00192     std::cin >> kiekis;
00193
00194     auto start = std::chrono::high_resolution_clock::now();
00195
00196     const int ndKiek = dist(gen);
00197
00198     write << std::left << std::setw(15) << "Vardas" << std::setw(15) << "Pavarde";
00199     for (int j = 0; j < ndKiek; ++j)
00200     {
00201         write << std::setw(8) << ("ND" + std::to_string(j + 1));
00202     }
00203     write << std::setw(8) << "Egz." << '\n';
00204
00205     for (int i = 0; i < kiekis; ++i)
00206     {
00207         write << std::left
00208             << std::setw(15) << ("Vardas" + std::to_string(i + 1))
00209             << std::setw(15) << ("Pavarde" + std::to_string(i + 1));
00210
00211         for (int j = 0; j < ndKiek; ++j)
00212         {
00213             write << std::setw(8) << dist(gen);
00214         }
00215
00216         write << std::setw(8) << dist(gen) << '\n'; // egzaminas
00217     }
00218
00219     auto end = std::chrono::high_resolution_clock::now();
00220     timers.generavimas = std::chrono::duration<double>(end - start).count();
00221
00222     std::cout << "Failas \"" << pav << "\" sugeneruotas per "
00223         << timers.generavimas << " s." << std::endl;
00224 }

```

5.3 Generavimas.h File Reference

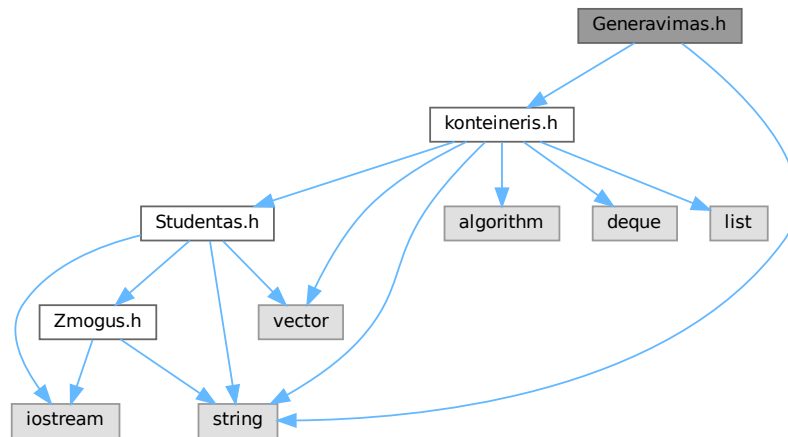
Studentų failų generavimo ir skirstymo į grupes funkcijų deklaracijos.

```

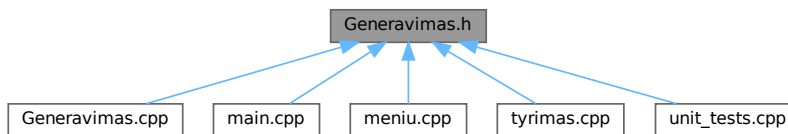
#include "konteineris.h"
#include <string>

```

Include dependency graph for Generavimas.h:



This graph shows which files directly or indirectly include this file:



Classes

- struct [Skirstymorezultatas](#)
Saugo skirstymo į grupes rezultatus.

Enumerations

- enum class [SkirstymoStrategija](#) { [Pirma](#) = 1 , [Antra](#) = 2 , [Trecia](#) = 3 }
Nurodo, kokių algoritmu skirstyti studentus į grupes.

Functions

- void [generuotiFaila](#) ()
Generuoja studentų duomenų failą.
- std::string [failoPavadinimas](#) (const std::string &bazinisPavadinimas)
Sugeneruoja unikalių failo pavadinimą.
- [SkirstymoStrategija pasirinktiStrategija](#) ()
Interaktyviai prašo vartotojo pasirinkti skirstymo strategiją.

- [Skirstymorezultatas skirstymasStrategija1](#) (const [StudContainer](#) &studis)
Skirsto studentus pagal 1-ą strategiją.
- [Skirstymorezultatas skirstymasStrategija2](#) ([StudContainer](#) &studis)
Skirsto studentus pagal 2-ą strategiją.
- [Skirstymorezultatas skirstymasStrategija3](#) ([StudContainer](#) &studis)
Skirsto studentus pagal 3-ą strategiją (efektyviausia).
- [Skirstymorezultatas skirstymasGrupes](#) ([StudContainer](#) &studis, [SkirstymoStrategija](#) strategija, bool irasyti↔ IFailus=true, bool paprasytiRikiavimo=true)
Bendro naudojimo skirstymo funkcija.

5.3.1 Detailed Description

Studentų failų generavimo ir skirstymo į grupes funkcijų deklaracijos.

Apibrėžia [SkirstymoStrategija](#) enumeraciją, [Skirstymorezultatas](#) struktūrą ir visas su studentų generavimu bei grupavimu susijusias funkcijas.

Author

[Studentas](#)

Version

2.0

Definition in file [Generavimas.h](#).

5.3.2 Enumeration Type Documentation

5.3.2.1 SkirstymoStrategija

```
enum class SkirstymoStrategija [strong]
```

Nurodo, koku algoritmu skirstyti studentus į grupes.

Reikšmė	Aprašymas
Pirma	Du nauji konteineriai (vargsiukai ir protai)
Antra	Vargsiukai ištraukiami iš bendro konteinerio; protai lieka
Trecia	Efektyvus <code>std::partition</code> algoritmas

Enumerator

Pirma	
Antra	
Trecia	

Definition at line 29 of file [Generavimas.h](#).

5.3.3 Function Documentation

5.3.3.1 failoPavadinimas()

```
std::string failoPavadinimas (  
    const std::string & bazinisPavadinimas )
```

Sugeneruoja unikalų failo pavadinimą.

Jei failas su nurodytu pavadinimu jau egzistuoja, automatiškai pridedamas skaitinis sufiksas (pvz. _2, _3).

Parameters

<i>bazinisPavadinimas</i>	Bazinis failo pavadinimas be plėtinio.
---------------------------	--

Returns

Unikalus failo pavadinimas su `.txt` plėtiniu.

Definition at line 22 of file [Generavimas.cpp](#).

References [galimasPavadinimas\(\)](#).

Referenced by [skirstymasGrupes\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.3.3.2 generuotiFaila()

```
void generuotiFaila ( )
```

Generuoja studentų duomenų failą.

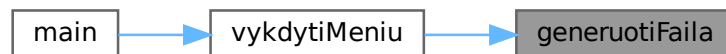
Interaktyviai prašo vartotojo įvesti failo pavadinimą ir studentų kiekį. Studentų vardai, pavardės ir pažymiai generuojami automatiškai.

Definition at line 174 of file [Generavimas.cpp](#).

References [TimerResults::generavimas](#), and [timers](#).

Referenced by [vykdytiMenu\(\)](#).

Here is the caller graph for this function:



5.3.3.3 pasirinktiStrategija()

```
SkirstymoStrategija pasirinktiStrategija ( )
```

Interaktyviai prašo vartotojo pasirinkti skirstymo strategiją.

Išveda meniu į konsolę ir nuskaito vartotojo pasirinkimą.

Returns

Pasirinkta `SkirstymoStrategija`.

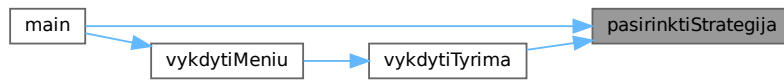
Exceptions

<code>std::runtime_error</code>	Jei vartotojo įvestis neteisinga.
---------------------------------	-----------------------------------

Definition at line 36 of file [Generavimas.cpp](#).

Referenced by [main\(\)](#), and [vykdytiTyrima\(\)](#).

Here is the caller graph for this function:



5.3.3.4 skirstymasGrupes()

```

Skirstymorezultatas skirstymasGrupes (
    StudContainer & studis,
    SkirstymoStrategija strategija,
    bool irasytiIFailus = true,
    bool paprasytiRikiavimo = true )
  
```

Bendro naudojimo skirstymo funkcija.

Išsaugo laiko matavimus į globalų `timers` objektą. Pagal parametrus gali rašyti į failus ir prašyti rikiavimo pasirinkimo.

Parameters

<i>studis</i>	Studentų konteineris.
<i>strategija</i>	Naudojama skirstymo strategija.
<i>irasytiIFailus</i>	Jei <code>true</code> , rezultatai išsaugomi į failus.
<i>paprasytiRikiavimo</i>	Jei <code>true</code> , vartotojas gali pasirinkti rikiavimą.

Returns

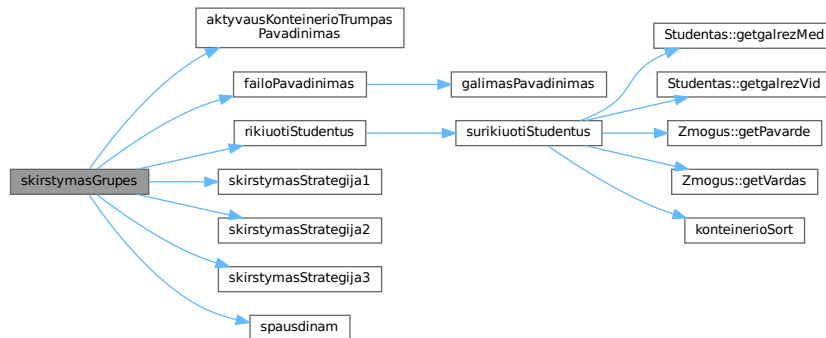
`Skirstymorezultatas` su vargsiukais ir protais.

Definition at line 112 of file [Generavimas.cpp](#).

References [aktyvausKonteinerioTrumpasPavadinimas\(\)](#), [Antra](#), [failoPavadinimas\(\)](#), [TimerResults::isvedimas](#), [Pirma](#), [Skirstymorezultatas::protai](#), [rikiuotiStudentus\(\)](#), [TimerResults::skirstymas](#), [skirstymasStrategija1\(\)](#), [skirstymasStrategija2\(\)](#), [skirstymasStrategija3\(\)](#), [spausdinam\(\)](#), [timers](#), [Trecia](#), and [Skirstymorezultatas::vargsiukai](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.3.3.5 skirstymasStrategija1()

```
Skirstymorezultatas skirstymasStrategija1 (
    const StudContainer & studis )
```

Skirsto studentus pagal 1-ą strategiją.

Eina per visus studentus ir juos kopijuoja į atskirus konteinerius. Originalus konteineris nekeičiamas.

Parameters

<i>studis</i>	Studentų konteineris (const – nekeičiamas).
---------------	---

Returns

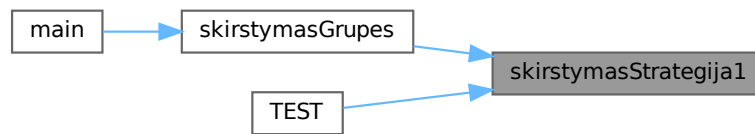
Skirstymorezultatas su dviem užpildytais konteineriais.

Definition at line 54 of file [Generavimas.cpp](#).

References [Skirstymorezultatas::protai](#), and [Skirstymorezultatas::vargsiukai](#).

Referenced by [skirstymasGrupes\(\)](#), and [TEST\(\)](#).

Here is the caller graph for this function:



5.3.3.6 skirstymasStrategija2()

```
Skirstymorezultatas skirstymasStrategija2 (  
    StudContainer & studis )
```

Skirsto studentus pagal 2-ą strategiją.

Vargsiukai ištraukiami iš originalaus konteinerio (erase). Likę studentai (protai) perkeliama į rezultatą.

Parameters

<code>studis</code>	Studentų konteineris (keičiamas – iš jo šalinami vargsiukai).
---------------------	---

Returns

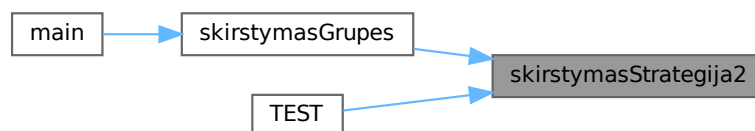
`Skirstymorezultatas`.

Definition at line 73 of file `Generavimas.cpp`.

References `Skirstymorezultatas::protai`, and `Skirstymorezultatas::vargsiukai`.

Referenced by `skirstymasGrupes()`, and `TEST()`.

Here is the caller graph for this function:



5.3.3.7 skirstymasStrategija3()

```
Skirstymorezultatas skirstymasStrategija3 (
    StudContainer & studis )
```

Skirsto studentus pagal 3-ą strategiją (efektyviausia).

Naudoja `std::partition` algoritmą, kuris perskirsto elementus vietoje, tuomet siunčia į du atskirus konteinerius.

Parameters

<code>studis</code>	Studentų konteineris (keičiamas).
---------------------	-----------------------------------

Returns

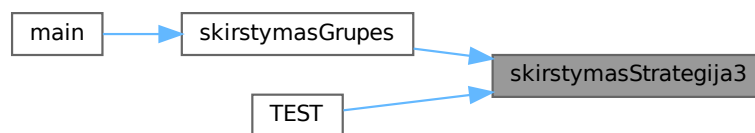
`Skirstymorezultatas`.

Definition at line 94 of file [Generavimas.cpp](#).

References [Skirstymorezultatas::protai](#), and [Skirstymorezultatas::vargsiukai](#).

Referenced by [skirstymasGrupes\(\)](#), and [TEST\(\)](#).

Here is the caller graph for this function:



5.4 Generavimas.h

[Go to the documentation of this file.](#)

```
00001
00012 #pragma once
00013
00014 #include "konteineris.h"
00015
00016 #include <string>
00017
00029 enum class SkirstymoStrategija
00030 {
00031     Pirma = 1,
00032     Antra = 2,
00033     Trecia = 3
00034 };
00035
00041 struct Skirstymorezultatas
00042 {
00043     StudContainer vargsiukai;
00044     StudContainer protai;
00045 };
```

```

00046
00053 void generuotiFaila();
00054
00065 std::string failoPavadinimas(const std::string& bazinisPavadinimas);
00066
00076 SkirstymoStrategija pasirinktiStrategija();
00077
00088 Skirstymorezultatas skirstymasStrategija1(const StudContainer& studis);
00089
00100 Skirstymorezultatas skirstymasStrategija2(StudContainer& studis);
00101
00112 Skirstymorezultatas skirstymasStrategija3(StudContainer& studis);
00113
00127 Skirstymorezultatas skirstymasGrupes(
00128     StudContainer& studis,
00129     SkirstymoStrategija strategija,
00130     bool irasytiIFailus = true,
00131     bool paprasytiRikiavimo = true);

```

5.5 konteineris.h File Reference

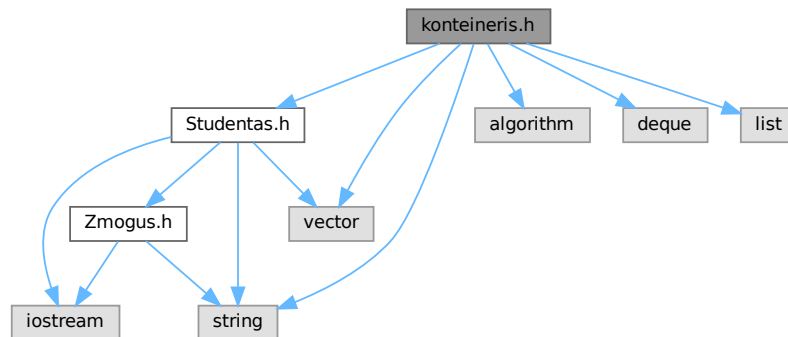
Kompiliavimo laiku pasirenkamas studentų konteineris.

```

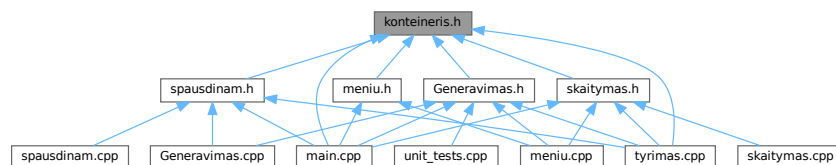
#include "Studentas.h"
#include <algorithm>
#include <deque>
#include <list>
#include <string>
#include <vector>

```

Include dependency graph for konteineris.h:



This graph shows which files directly or indirectly include this file:



Typedefs

- `template<typename T >`
`using StudentuKonteineris = std::vector< T >`
Studentų konteinerio tipo alias (std::vector variantas – numatytasis).
- `using StudContainer = StudentuKonteineris< Studentas >`
Pagrindinis studentų konteineris naudojamas per visą programą.

Functions

- `std::string aktyvausKonteinerioPavadinimas ()`
Grąžina aktyvaus konteinerio pilną pavadinimą.
- `std::string aktyvausKonteinerioTrumpasPavadinimas ()`
Grąžina aktyvaus konteinerio trumpąjį pavadinimą.
- `template<typename T , typename Alloc , typename Compare >`
`void konteinerioSort (std::list< T, Alloc > &konteineris, Compare comp)`
Rikiavimo specializacija std::list konteineriui.
- `template<typename Container , typename Compare >`
`void konteinerioSort (Container &konteineris, Compare comp)`
Universali rikiavimo funkcija bet kuriam atsitiktinės prieigos konteineriui.

5.5.1 Detailed Description

Kompiliavimo laiku pasirenkamas studentų konteineris.

Naudojant preprocesorių galima pasirinkti vieną iš trijų STL konteinerių tipų:

- `std::vector` (numatytasis)
- `std::list` (kompiliuoti su `-DUSE_LIST`)
- `std::deque` (kompiliuoti su `-DUSE_DEQUE`)

Author

[Studentas](#)

Version

2.0

Definition in file [konteineris.h](#).

5.5.2 Typedef Documentation

5.5.2.1 StudContainer

[StudContainer](#)

Pagrindinis studentų konteineris naudojamas per visą programą.

Tipas priklauso nuo kompiliacijos laiko makro (`USE_LIST`, `USE_DEQUE` arba numatytasis `std::vector`).

Definition at line 124 of file [konteineris.h](#).

5.5.2.2 StudentuKonteineris

```
template<typename T >
using StudentuKonteineris = std::vector<T>
```

Studentų konteinerio tipo alias (std::vector variantas – numatytasis).

Template Parameters

<i>T</i>	Elementų tipas.
----------	-----------------

Definition at line 92 of file [konteineris.h](#).

5.5.3 Function Documentation

5.5.3.1 aktyvausKonteinerioPavadinimas()

```
std::string aktyvausKonteinerioPavadinimas ( ) [inline]
```

Grąžina aktyvaus konteinerio pilną pavadinimą.

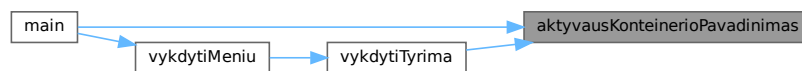
Returns

```
"std::vector"
```

Definition at line 99 of file [konteineris.h](#).

Referenced by [main\(\)](#), and [vykdytiTyrima\(\)](#).

Here is the caller graph for this function:



5.5.3.2 aktyvausKonteinerioTrumpasPavadinimas()

```
std::string aktyvausKonteinerioTrumpasPavadinimas ( ) [inline]
```

Grąžina aktyvaus konteinerio trumpąjį pavadinimą.

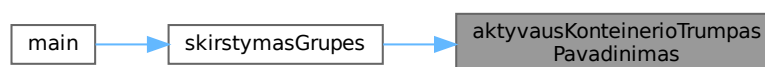
Returns

```
"vector"
```

Definition at line 109 of file [konteineris.h](#).

Referenced by [skirstymasGrupes\(\)](#).

Here is the caller graph for this function:



5.5.3.3 konteinerioSort() [1/2]

```
template<typename Container , typename Compare >
void konteinerioSort (
    Container & konteineris,
    Compare comp )
```

Universali rikiavimo funkcija bet kuriam atsitiktinės prieigos konteineriui.

Naudoja `std::sort` su pradžios ir pabaigos iteratoriais.

Template Parameters

<i>Container</i>	Konteinerio tipas (pvz. <code>std::vector</code> , <code>std::deque</code>).
<i>Compare</i>	Lyginimo funktorius.

Parameters

<i>konteineris</i>	Rikiuojamas konteineris.
<i>comp</i>	Lyginimo funktorius.

Definition at line 157 of file [konteineris.h](#).

5.5.3.4 konteinerioSort() [2/2]

```
template<typename T , typename Alloc , typename Compare >
void konteinerioSort (
    std::list< T, Alloc > & konteineris,
    Compare comp )
```

Rikiavimo specializacija `std::list` konteineriui.

`std::list` neturi iteratoriaus aritmetikos, todėl reikia naudoti `list::sort()` vietoj `std::sort()`.

Template Parameters

<i>T</i>	Elementų tipas.
<i>Alloc</i>	Alokatorius.
<i>Compare</i>	Lyginimo funktorius.

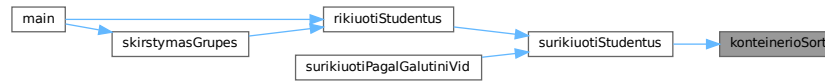
Parameters

<i>konteineris</i>	Rikiuojamas sąrašas.
<i>comp</i>	Lyginimo funktorius.

Definition at line 140 of file [konteineris.h](#).

Referenced by [surikiuotiStudentus\(\)](#).

Here is the caller graph for this function:



5.6 konteineris.h

[Go to the documentation of this file.](#)

```

00001
00015 #pragma once
00016
00017 #include "Studentas.h"
00018
00019 #include <algorithm>
00020 #include <deque>
00021 #include <list>
00022 #include <string>
00023 #include <vector>
00024
00025 #if defined(USE_LIST)
00031 template <typename T>
00032 using StudentuKonteineris = std::list<T>;
00033
00039 inline std::string aktyvausKonteinerioPavadinimas()
00040 {
00041     return "std::list";
00042 }
00043
00049 inline std::string aktyvausKonteinerioTrumpasPavadinimas()
00050 {
00051     return "list";
00052 }
00053
00054 #elif defined(USE_DEQUE)
00055
00061 template <typename T>
00062 using StudentuKonteineris = std::deque<T>;
00063
00069 inline std::string aktyvausKonteinerioPavadinimas()
00070 {
00071     return "std::deque";
00072 }
00073
00079 inline std::string aktyvausKonteinerioTrumpasPavadinimas()
00080 {
00081     return "deque";
00082 }
00083
00084 #else
00085
00091 template <typename T>
00092 using StudentuKonteineris = std::vector<T>;
00093
00099 inline std::string aktyvausKonteinerioPavadinimas()
00100 {
00101     return "std::vector";
00102 }
00103
00109 inline std::string aktyvausKonteinerioTrumpasPavadinimas()
00110 {
00111     return "vector";
00112 }
00113 #endif
00114
00123 // StudContainer dabar laiko objektus ( yay :) )
00124 using StudContainer = StudentuKonteineris<Studentas>;
00125
00139 template <typename T, typename Alloc, typename Compare>
00140 void konteinerioSort(std::list<T, Alloc>& konteineris, Compare comp)
00141 {
00142     konteineris.sort(comp);
00143 }
00144

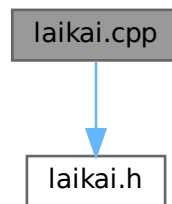
```

```
00156 template <typename Container, typename Compare>
00157 void konteinerioSort(Container& konteineris, Compare comp)
00158 {
00159     std::sort(konteineris.begin(), konteineris.end(), comp);
00160 }
```

5.7 laikai.cpp File Reference

```
#include "laikai.h"
```

Include dependency graph for laikai.cpp:



Functions

- void `nunulintiLaikus()`
Nulinina visus laiko matavimus.

Variables

- `TimerResults timers`
Globalus laiko matavimų objektas.

5.7.1 Function Documentation

5.7.1.1 `nunulintiLaikus()`

```
void nunulintiLaikus ( )
```

Nulinina visus laiko matavimus.

Iškviečiama kiekvienos naujos iteracijos pradžioje.

Definition at line 5 of file `laikai.cpp`.

References `timers`.

Referenced by `main()`.

Here is the caller graph for this function:



5.7.2 Variable Documentation

5.7.2.1 timers

```
TimerResults timers
```

Globalus laiko matavimų objektas.

Definition at line 3 of file [laikai.cpp](#).

Referenced by [failoSkaitymas\(\)](#), [generuotiFaila\(\)](#), [nunulintiLaikus\(\)](#), [skirstymasGrupes\(\)](#), and [surikiuotiStudentus\(\)](#).

5.8 laikai.cpp

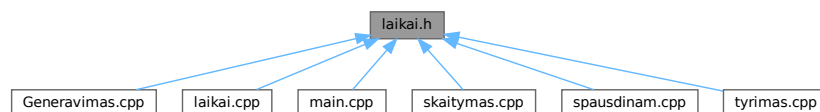
[Go to the documentation of this file.](#)

```
00001 #include "laikai.h"
00002
00003 TimerResults timers;
00004
00005 void nunulintiLaikus()
00006 {
00007     timers = TimerResults{};
00008 }
```

5.9 laikai.h File Reference

Laiko matavimų struktūra ir pagalbinės funkcijos.

This graph shows which files directly or indirectly include this file:



Classes

- struct [TimerResults](#)
Saugo kiekvienos programos fazės laiko matavimus (sekundėmis).

Functions

- void [nunulintiLaikus](#) ()
Nulinina visus laiko matavimus.

Variables

- [TimerResults timers](#)
Globalus laiko matavimų objektas.

5.9.1 Detailed Description

Laiko matavimų struktūra ir pagalbinės funkcijos.

Apibrėžia globalų `timers` objektą, kuriame kaupiami kiekvienos programos fazės trukmės matavimai.

Author

[Studentas](#)

Version

2.0

Definition in file [laikai.h](#).

5.9.2 Function Documentation

5.9.2.1 `nunulintiLaikus()`

```
void nunulintiLaikus ( )
```

Nulinina visus laiko matavimus.

Išskviečiama kiekvienos naujos iteracijos pradžioje.

Definition at line [5](#) of file [laikai.cpp](#).

References [timers](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



5.9.3 Variable Documentation

5.9.3.1 timers

`TimerResults` timers [extern]

Globalus laiko matavimų objektas.

Definition at line 3 of file `laikai.cpp`.

Referenced by `failoSkaitymas()`, `generuotiFaila()`, `nunulintiLaikus()`, `skirstymasGrupes()`, and `surikiuotiStudentus()`.

5.10 laikai.h

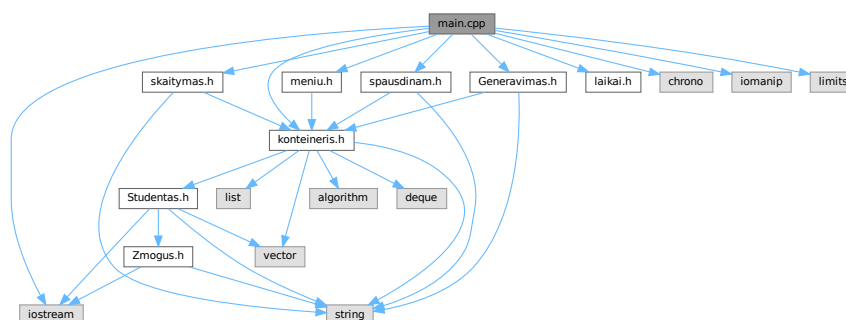
[Go to the documentation of this file.](#)

```
00001
00013 #pragma once
00014
00020 struct TimerResults
00021 {
00022     double generavimas = 0.0;
00023     double skaitymas = 0.0;
00024     double rusiavimas = 0.0;
00025     double skirstymas = 0.0;
00026     double isvedimas = 0.0;
00027 };
00028
00030 extern TimerResults timers;
00031
00038 void nunulintiLaikus();
```

5.11 main.cpp File Reference

```
#include "Generavimas.h"
#include "konteineris.h"
#include "laikai.h"
#include "menu.h"
#include "skaitymas.h"
#include "spausdinam.h"
#include <chrono>
#include <iomanip>
#include <iostream>
#include <limits>
```

Include dependency graph for `main.cpp`:



Functions

- int [main](#) ()

5.11.1 Function Documentation

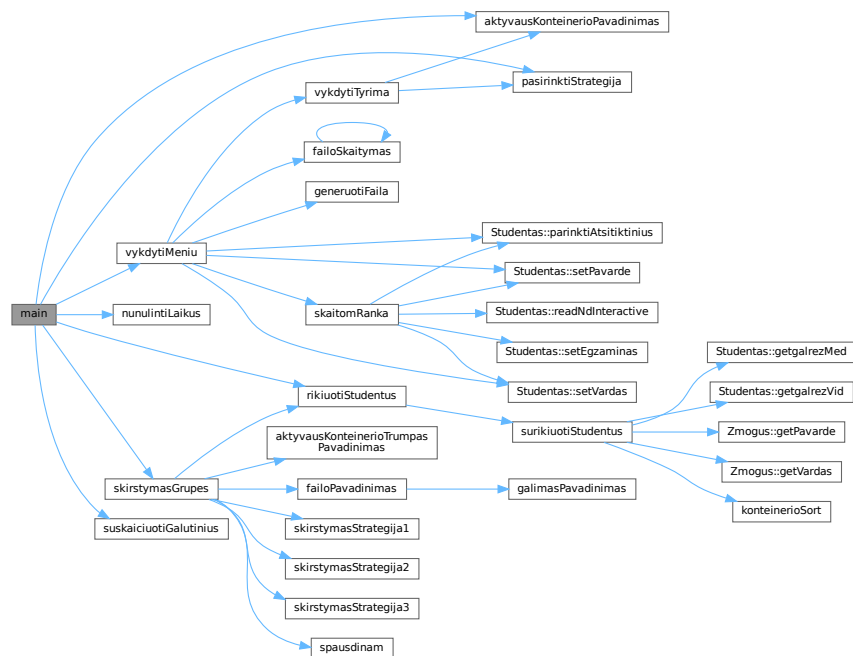
5.11.1.1 main()

```
int main ( )
```

Definition at line 122 of file [main.cpp](#).

References [aktyvausKonteinerioPavadinimas\(\)](#), [nunulintiLaikus\(\)](#), [pasirinktiStrategija\(\)](#), [rikiuotiStudentus\(\)](#), [skirstymasGrupes\(\)](#), [suskaiciuotiGalutinius\(\)](#), and [vykdytiMeniu\(\)](#).

Here is the call graph for this function:



5.12 main.cpp

[Go to the documentation of this file.](#)

```

00001 #include "Generavimas.h"
00002 #include "konteineris.h"
00003 #include "laikai.h"
00004 #include "menu.h"
00005 #include "skaitymas.h"
00006 #include "spausdinam.h"
00007
00008 #include <chrono>
00009 #include <iomanip>
00010 #include <iostream>
00011 #include <limits>
00012
00013

```

```

00014 namespace
00015 {
00016
00017 int gautiApdorojimoPasirinkima()
00018 {
00019     std::cout << "\nKa daryti su gautais studentais?\n"
00020     << "1 - Spausdinti visus studentus\n"
00021     << "2 - Suskirstyti i vargsiukus ir protus\n"
00022     << "3 - Abu veiksmi\n";
00023
00024     while (true)
00025     {
00026         int p = 0;
00027         std::cin >> p;
00028
00029         if (!std::cin || p < 1 || p > 3)
00030         {
00031             std::cerr << "Neteisinga ivestis. Bandykite is naujo.\n";
00032             std::cin.clear();
00033             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00034             continue;
00035         }
00036
00037         return p;
00038     }
00039 }
00040
00041 void pradetiSpausdinti(const StudContainer& studis)
00042 {
00043     std::cout << "I konsole ar i faila?\n1 - Konsole\n2 - Faila\n";
00044
00045     int failas = 0;
00046     while (true)
00047     {
00048         std::cin >> failas;
00049
00050         if (!std::cin)
00051         {
00052             std::cerr << "Neteisinga ivestis. Bandykite is naujo.\n";
00053             std::cin.clear();
00054             std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00055             continue;
00056         }
00057
00058         break;
00059     }
00060
00061     if (failas == 1)
00062     {
00063         // Naudojame get'eries - tiesioginis prieigos prie laukų nėra
00064         std::cout << std::left
00065         << std::setw(20) << "Vardas"
00066         << std::setw(20) << "Pavarde"
00067         << std::setw(20) << "Galutinis (Vid.)"
00068         << std::setw(15) << "Galutinis (Med.)" << '\n';
00069         std::cout << std::string(75, '-') << '\n';
00070
00071         for (const auto& s : studis)
00072         {
00073             std::cout << std::left
00074             << std::setw(20) << s.getVardas()
00075             << std::setw(20) << s.getPavarde()
00076             << std::setw(20) << std::fixed << std::setprecision(2) << s.getGalrezVid()
00077             << std::setw(15) << std::fixed << std::setprecision(2) << s.getGalrezMed()
00078             << '\n';
00079         }
00080     }
00081     else if (failas == 2)
00082     {
00083         std::cout << "Iveskite pavadinima: ";
00084         std::string pav;
00085
00086         while (true)
00087         {
00088             std::cin >> pav;
00089
00090             if (!std::cin)
00091             {
00092                 std::cerr << "Neteisinga ivestis. Bandykite is naujo.\n";
00093                 std::cin.clear();
00094                 std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00095                 continue;
00096             }
00097
00098             break;
00099         }
00100     }

```

```

00101         spausdinam(studis, pav);
00102     }
00103     else
00104     {
00105         std::cout << "Neteisingas pasirinkimas.\n";
00106     }
00107 }
00108
00109 void laikorezultatas(double progTrukme)
00110 {
00111     std::cout << "\n--- Laiko rezultatai ---\n"
00112         << "Failo skaitymas: " << timers.skaitymas << " s\n"
00113         << "Studentu rikiavimas: " << timers.rusiavimas << " s\n"
00114         << "Studentu skirstymas: " << timers.skirstymas << " s\n"
00115         << "Failu isvedimas: " << timers.isvedimas << " s\n"
00116         << "Visos programos trukme: " << progTrukme << " s\n";
00117 }
00118
00119 }
00120
00121
00122 int main()
00123 {
00124     auto progStart = std::chrono::high_resolution_clock::now();
00125
00126     try
00127     {
00128         while (true)
00129         {
00130             nunulintiLaikus();
00131
00132             std::cout << "\nAktyvus konteineris: " << aktyvusKonteinerioPavadinimas() << '\n';
00133
00134             StudContainer studis;
00135
00136             if (!vykdytiMenu(studis))
00137             {
00138                 break;
00139             }
00140
00141             if (studis.empty())
00142             {
00143                 std::cout << "Nera ivestu studentu.\n";
00144                 continue;
00145             }
00146
00147             //kiečia s.suskaiciuotiGalutinius() kiekvienam
00148             suskaiciuotiGalutinius(studis);
00149
00150             const int apdorojimas = gautiApdorojimoPasirinkima();
00151
00152             if (apdorojimas == 1 || apdorojimas == 3)
00153             {
00154                 rikiuotiStudentus(studis, "studentus");
00155                 pradetiSpausdinti(studis);
00156             }
00157
00158             if (apdorojimas == 2 || apdorojimas == 3)
00159             {
00160                 const SkirstymoStrategija strategija = pasirinktiStrategija();
00161                 skirstymasGrupes(studis, strategija, true, true);
00162             }
00163
00164             char gr = '\n';
00165             std::cout << "Grizti i pradzia? (y/n): ";
00166             std::cin >> gr;
00167
00168             if (!std::cin)
00169             {
00170                 std::cin.clear();
00171                 std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00172             }
00173
00174             if (gr != 'y' && gr != 'Y')
00175             {
00176                 break;
00177             }
00178         }
00179     }
00180     catch (const std::exception& e)
00181     {
00182         std::cerr << "Klaida: " << e.what() << std::endl;
00183         return 1;
00184     }
00185
00186     auto progEnd = std::chrono::high_resolution_clock::now();
00187     laikorezultatas(std::chrono::duration<double>(progEnd - progStart).count());

```

```

00188
00189     return 0;
00190 }

```

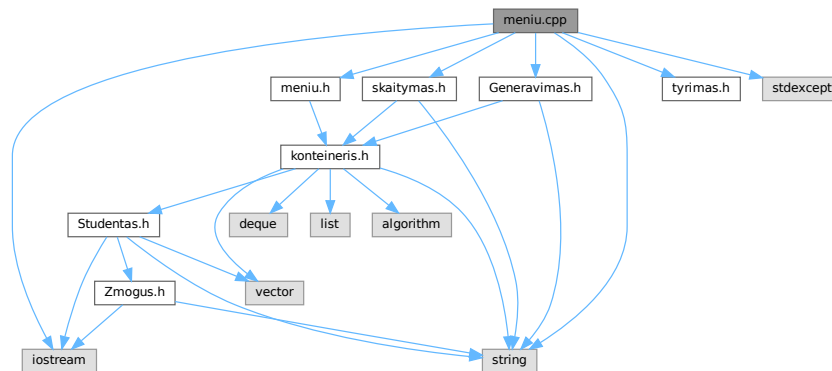
5.13 meniu.cpp File Reference

```

#include "menu.h"
#include "Generavimas.h"
#include "skaitymas.h"
#include "tyrimas.h"
#include <iostream>
#include <stdexcept>
#include <string>

```

Include dependency graph for menu.cpp:



Functions

- bool [vykdytiMenu](#) ([StudContainer](#) &studis)
Paleidžia pagrindinį interaktyvų programos meniu.

5.13.1 Function Documentation

5.13.1.1 vykdytiMenu()

```

bool vykdytiMenu (
    StudContainer & studis )

```

Paleidžia pagrindinį interaktyvų programos meniu.

Leidžia vartotojui pasirinkti vieną iš šių veiksmų:

- įvesti studentą rankiniu būdu;
- nuskaityti studentus iš failo;
- pridėti atsitiktinius studentus;

- generuoti duomenų failą;
- atlikti konteinerių greیتaveikos tyrimą;
- paleisti klasės unit testus;
- tikrinti [Zmogus](#) abstraktumą;
- baigti darbą.

Parameters

<i>studis</i>	Studentų konteineris, į kurį pridedami / keičiami duomenys.
---------------	---

Returns

`true` jei reikia tęsti programą (grįžti į pagrindinį ciklą), `false` jei vartotojas pasirinko baigti darbą.

Exceptions

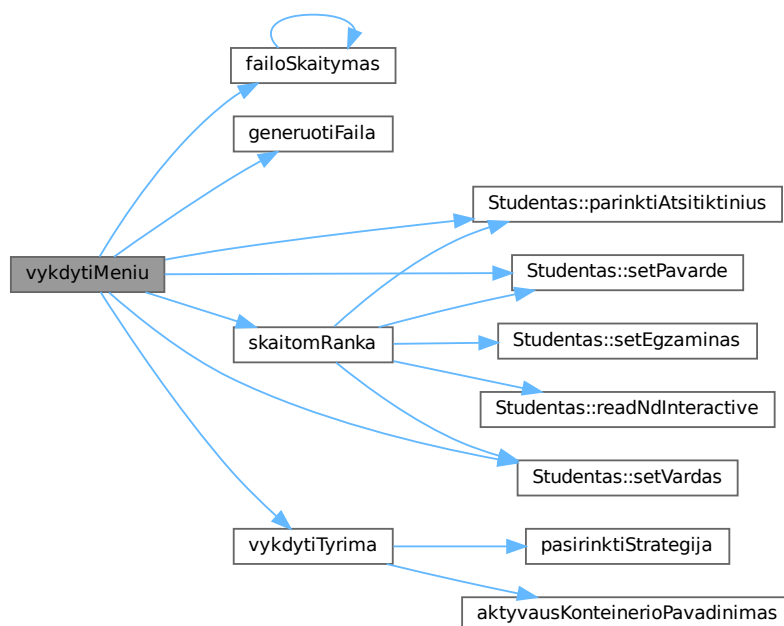
<i>std::runtime_error</i>	Klaidos įvesties atveju.
---------------------------	--------------------------

Definition at line 11 of file [menu.cpp](#).

References [failoSkaitymas\(\)](#), [generuotiFaila\(\)](#), [Studentas::parinktiAtsitiktinius\(\)](#), [Studentas::setPavarde\(\)](#), [Studentas::setVardas\(\)](#), [skaitomRanka\(\)](#), and [vykdytiTyrima\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.14 meniu.cpp

[Go to the documentation of this file.](#)

```

00001 #include "menu.h"
00002
00003 #include "Generavimas.h"
00004 #include "skaitymas.h"
00005 #include "tyrimas.h"
00006
00007 #include <iostream>
00008 #include <stdexcept>
00009 #include <string>
00010
00011 bool vykdytiMeniu(StudContainer& studis)
00012 {
00013     while (true)
00014     {
00015         std::cout << "\nPasirinkite programos eiga:\n"
00016             << "1 - Ivesti studenta ranka\n"
00017             << "2 - Skaityti is failo ir iskart apdoroti\n"
00018             << "3 - Prideti studenta su atsitiktiniais pazymiais (su vardu)\n"
00019             << "4 - Prideti sugeneruota studenta (auto vardas/pavarde)\n"
00020             << "5 - Sugeneruoti studentu faila\n"
00021             << "6 - Atlikti konteineriu ir strategiju tyrima\n"
00022             << "7 - Baigti darba\n";
00023
00024         int pasirinkimas = 0;
00025         std::cin >> pasirinkimas;
00026
00027         if (!std::cin)
00028         {
00029             throw std::runtime_error("Bloga ivestis.");
00030         }
00031
00032         switch (pasirinkimas)
00033         {
00034             case 1:
00035             {
00036                 Studentas s;
00037                 skaitomRanka(s);
00038                 studis.push_back(s);
00039                 break;
00040             }
00041
00042             case 2:
00043             {
00044                 if (failoSkaitymas(studis))
00045                 {
00046                     return true;
00047                 }
00048                 break;
00049             }
00050
00051             case 3:
00052             {
00053                 Studentas s;
00054                 std::string v, p;
00055                 std::cout << "Iveskite varda: ";
00056                 std::cin >> v;
00057                 s.setVardas(v);
00058                 std::cout << "Iveskite pavarde: ";
00059                 std::cin >> p;
00060                 s.setPavarde(p);
00061                 s.parinktiAtsitiktinius();

```

```

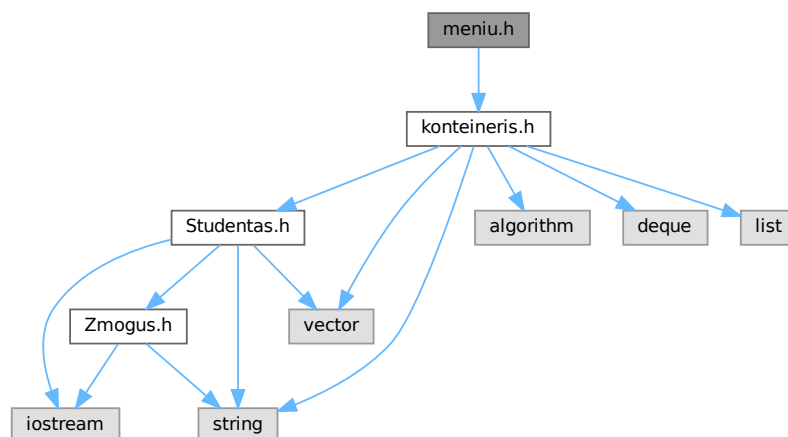
00062         studis.push_back(s);
00063         break;
00064     }
00065
00066     case 4:
00067     {
00068         static int nr = 1;
00069         Studentas s;
00070         s.setVardas("Vardas" + std::to_string(nr));
00071         s.setPavarde("Pavarde" + std::to_string(nr));
00072         nr++;
00073         s.parinktiAtsitiktinius();
00074         studis.push_back(s);
00075         break;
00076     }
00077
00078     case 5:
00079     {
00080         generuotiFaila();
00081         break;
00082     }
00083
00084     case 6:
00085     {
00086         vykdytiTyrima();
00087         break;
00088     }
00089     case 7:
00090     {
00091         return false;
00092     }
00093
00094     default:
00095         std::cout << "Neteisingas pasirinkimas.\n";
00096         break;
00097     }
00098 }
00099 }

```

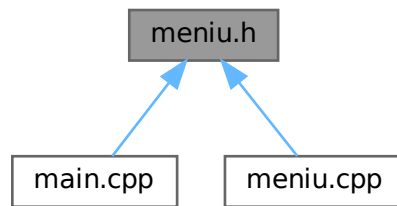
5.15 meniu.h File Reference

Pagrindinio programos meniu funkcijos deklaracija.

```
#include "konteineris.h"
Include dependency graph for meniu.h:
```



This graph shows which files directly or indirectly include this file:



Functions

- bool [vykdytiMenu](#) ([StudContainer](#) &studis)
Paleidžia pagrindinį interaktyvų programos meniu.

5.15.1 Detailed Description

Pagrindinio programos meniu funkcijos deklaracija.

Author

[Studentas](#)

Version

2.0

Definition in file [menu.h](#).

5.15.2 Function Documentation

5.15.2.1 vykdytiMenu()

```
bool vykdytiMenu (  
    StudContainer & studis )
```

Paleidžia pagrindinį interaktyvų programos meniu.

Leidžia vartotojui pasirinkti vieną iš šių veiksmų:

- įvesti studentą rankiniu būdu;
- nuskaityti studentus iš failo;
- pridėti atsitiktinius studentus;
- generuoti duomenų failą;
- atlikti konteinerių greیتaveikos tyrimą;
- paleisti klasės unit testus;
- tikrinti [Zmogus](#) abstraktumą;
- baigti darbą.

Parameters

<code>studis</code>	Studentų konteineris, į kurį pridedami / keičiami duomenys.
---------------------	---

Returns

`true` jei reikia tęsti programą (grįžti į pagrindinį ciklą), `false` jei vartotojas pasirinko baigti darbą.

Exceptions

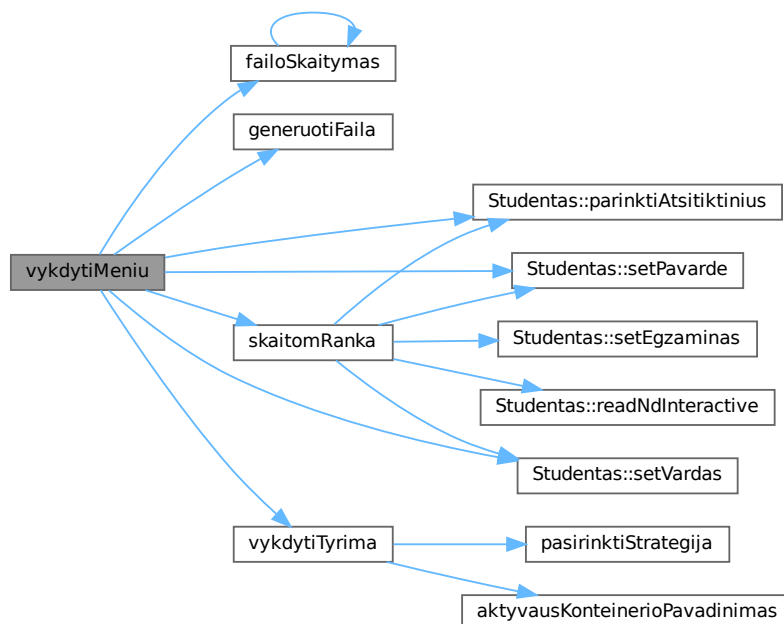
<code>std::runtime_error</code>	Klaidos įvesties atveju.
---------------------------------	--------------------------

Definition at line 11 of file [menui.cpp](#).

References [failoSkaitymas\(\)](#), [generuotiFaila\(\)](#), [Studentas::parinktiAtsitiktinius\(\)](#), [Studentas::setPavarde\(\)](#), [Studentas::setVardas\(\)](#), [skaitomRanka\(\)](#), and [vykdytiTyrima\(\)](#).

Referenced by [main\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.16 meniu.h

[Go to the documentation of this file.](#)

```

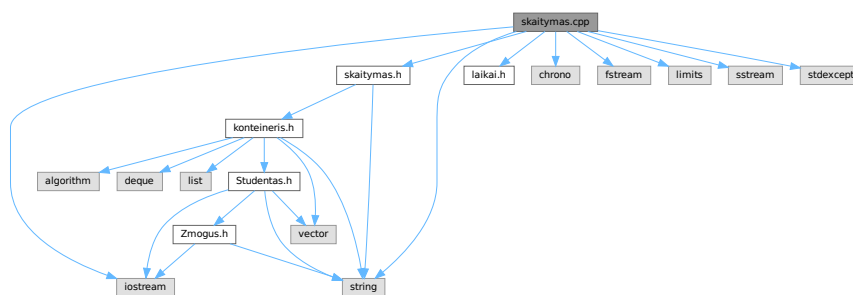
00001
00009 #pragma once
00010
00011 #include "konteineris.h"
00012
00032 bool vykdytiMeniu(StudContainer& studis);
  
```

5.17 skaitymas.cpp File Reference

```

#include "skaitymas.h"
#include "laikai.h"
#include <chrono>
#include <fstream>
#include <iostream>
#include <limits>
#include <sstream>
#include <stdexcept>
#include <string>
  
```

Include dependency graph for skaitymas.cpp:



Functions

- void [skaitomRanka](#) ([Studentas](#) &s)
Interaktyviai nuskaityti vieno studento duomenis iš konsolės.
- bool [failoSkaitymas](#) ([StudContainer](#) &studis)

Interaktyviai prašo failo pavadinimo ir nuskaito studentus.

- bool [failoSkaitymas](#) ([StudContainer](#) &studis, const std::string &pav)

Nuskaito studentus iš nurodyto failo.

- void [suskaiciuotiGalutinius](#) ([StudContainer](#) &studis)

Apskaičiuoja galutinius balus visiems studentams konteineryje.

5.17.1 Function Documentation

5.17.1.1 failoSkaitymas() [1/2]

```
bool failoSkaitymas (
    StudContainer & studis )
```

Interaktyviai prašo failo pavadinimo ir nuskaito studentus.

Pirmoji perkrovos versija – prašo vartotojo įvesti failo pavadinimą ir perduoda jį antrajai versijai.

Parameters

<code>studis</code>	Konteineris, į kurį pridedami nuskaityti studentai.
---------------------	---

Returns

`true` jei failas sėkmingai nuskaitytas, `false` klaidos atveju.

Definition at line 43 of file [skaitymas.cpp](#).

References [failoSkaitymas\(\)](#).

Referenced by [failoSkaitymas\(\)](#), and [vykdytiMeniu\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.17.1.2 failoSkaitymas() [2/2]

```
bool failoSkaitymas (
    StudContainer & studis,
    const std::string & pav )
```

Nuskaityti studentus iš nurodyto failo.

Antrarinė perkrovos versija su ekspllicitiniu failo pavadinimu. Failo formatas:

Vardas Pavardė ND1 ND2 ... NDn Egzaminas

Pirma eilutė (antraštė) ignoruojama. Matuoja nuskaitymo laiką ir išsaugo į globalų `timers` objektą.

Parameters

<i>studis</i>	Konteineris, į kurį pridedami nuskaityti studentai.
<i>pav</i>	Failo pavadinimas.

Returns

`true` jei failas sėkmingai nuskaitytas, `false` klaidos atveju.

Definition at line 52 of file [skaitymas.cpp](#).

References [Studentas::getND\(\)](#), [TimerResults::skaitymas](#), and [timers](#).

Here is the call graph for this function:



5.17.1.3 skaitomRanka()

```
void skaitomRanka (
    Studentas & s )
```

Interaktyviai nuskaityti vieno studento duomenis iš konsolės.

Prašo vartotojo įvesti vardą, pavardę ir (pagal pasirinkimą) namų darbų pažymius bei egzamino rezultatą rankiniu būdu arba sugeneruoja juos atsitiktinai.

Parameters

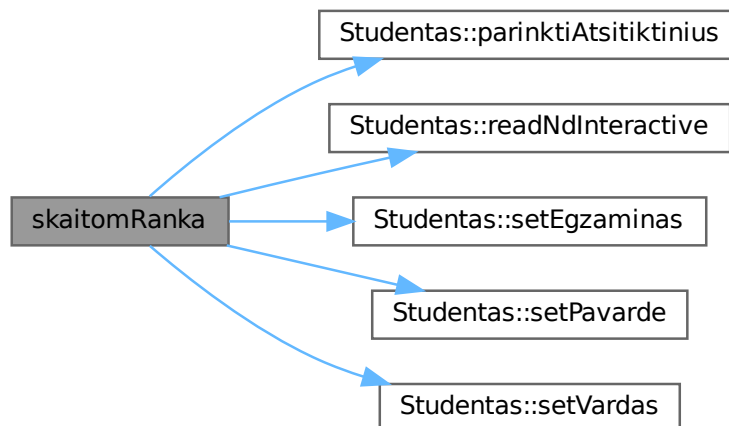
<i>s</i>	Studento objektas, į kurį įrašomi duomenys.
----------	---

Definition at line 12 of file [skaitymas.cpp](#).

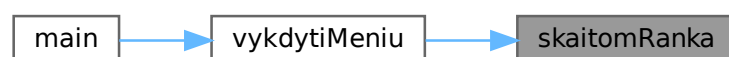
References [Studentas::parinktiAtsitiktinius\(\)](#), [Studentas::readNdInteractive\(\)](#), [Studentas::setEgzaminas\(\)](#), [Studentas::setPavarde\(\)](#), and [Studentas::setVardas\(\)](#).

Referenced by [vykdytiMenu\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.17.1.4 suskaiciuotiGalutinius()

```
void suskaiciuotiGalutinius (
    StudContainer & studis )
```

Apskaičiuoja galutinius balus visiems studentams konteineryje.

Iškviečia `Studentas::suskaiciuotiGalutini()` kiekvienam studentui.

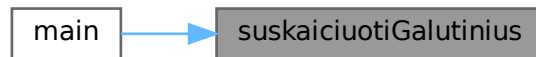
Parameters

<code>studis</code>	Studentų konteineris.
---------------------	-----------------------

Definition at line 99 of file [skaitymas.cpp](#).

Referenced by [main\(\)](#).

Here is the caller graph for this function:



5.18 skaitymas.cpp

[Go to the documentation of this file.](#)

```

00001 #include "skaitymas.h"
00002 #include "laikai.h"
00003
00004 #include <chrono>
00005 #include <fstream>
00006 #include <iostream>
00007 #include <limits>
00008 #include <sstream>
00009 #include <stdexcept>
00010 #include <string>
00011
00012 void skaitomRanka(Studentas& s)
00013 {
00014     std::string v, p;
00015
00016     std::cout << "Iveskite varda: ";
00017     std::cin >> v;
00018     s.setVardas(v);
00019
00020     std::cout << "Iveskite pavarde: ";
00021     std::cin >> p;
00022     s.setPavarde(p);
00023
00024     char c = 'y';
00025     std::cout << "Ar naudoti atsitiktinai parinktus n.d. rezultatus? (y/n): ";
00026     std::cin >> c;
00027
00028     if (c == 'n' || c == 'N')
00029     {
00030         s.readNdInteractive();
00031
00032         int egz = 0;
00033         std::cout << "Iveskite egzamino rezultatasultata (1-10): ";
00034         std::cin >> egz;
00035         s.setEgzaminas(egz);
00036     }
00037     else
00038     {
00039         s.parinktiAtsitiktinius();
00040     }
00041 }
00042
00043 bool failoSkaitymas(StudContainer& studis)
00044 {
00045     std::cout << "Iveskite failo pavadinima: ";
00046     std::string pav;
00047     std::cin >> pav;
00048     return failoSkaitymas(studis, pav);
00049 }
00050
00051
00052 bool failoSkaitymas(StudContainer& studis, const std::string& pav)
00053 {
00054     auto start = std::chrono::high_resolution_clock::now();
00055

```

```

00056     try
00057     {
00058         std::ifstream read(pav);
00059
00060         if (!read.is_open())
00061         {
00062             throw std::runtime_error("Neatidarem failo: " + pav);
00063         }
00064
00065         std::string eilute;
00066         std::getline(read, eilute);
00067
00068         while (std::getline(read, eilute))
00069         {
00070             if (eilute.empty())
00071             {
00072                 continue;
00073             }
00074
00075             // Studentas konstruktorius kviečia readStudent()
00076             std::stringstream ss(eilute);
00077             Studentas s(ss);
00078
00079             if (s.getND().empty())
00080             {
00081                 continue;
00082             }
00083
00084             studis.push_back(std::move(s));
00085         }
00086
00087         auto end = std::chrono::high_resolution_clock::now();
00088         timers.skaitymas = std::chrono::duration<double>(end - start).count();
00089
00090         return true;
00091     }
00092     catch (const std::exception& e)
00093     {
00094         std::cerr << "Klaida skaitant faila: " << e.what() << std::endl;
00095         return false;
00096     }
00097 }
00098
00099 void suskaiciuotiGalutinius(StudContainer& studis)
00100 {
00101     for (auto& s : studis)
00102     {
00103         s.suskaiciuotiGalutini();
00104     }
00105 }

```

5.19 skaitymas.h File Reference

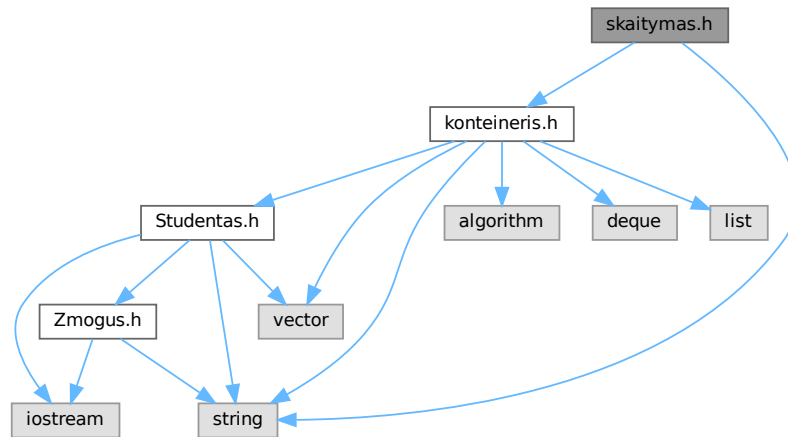
Studentų duomenų skaitymo funkcijų deklaracijos.

```

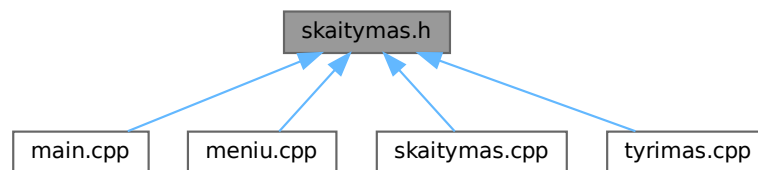
#include "konteineris.h"
#include <string>

```

Include dependency graph for skaitymas.h:



This graph shows which files directly or indirectly include this file:



Functions

- void [skaitomRanka](#) ([Studentas](#) &s)
Interaktyviai nuskaito vieno studento duomenis iš konsolės.
- bool [failoSkaitymas](#) ([StudContainer](#) &studis)
Interaktyviai prašo failo pavadinimo ir nuskaito studentus.
- bool [failoSkaitymas](#) ([StudContainer](#) &studis, const std::string &pav)
Nuskaito studentus iš nurodyto failo.
- void [suskaiciuotiGalutinius](#) ([StudContainer](#) &studis)
Apskaiciuoja galutinius balus visiems studentams konteineryje.

5.19.1 Detailed Description

Studentų duomenų skaitymo funkcijų deklaracijos.

Pateikia funkcijas duomenų nuskaitymui rankiniu būdu arba iš failo, bei galutinių balų apskaičiavimui visiems studentams konteineryje.

Author

[Studentas](#)

Version

2.0

Definition in file [skaitymas.h](#).

5.19.2 Function Documentation

5.19.2.1 failoSkaitymas() [1/2]

```
bool failoSkaitymas (
    StudContainer & studis )
```

Interaktyviai prašo failo pavadinimo ir nuskaityto studentus.

Pirmoji perkrovos versija – prašo vartotojo įvesti failo pavadinimą ir perduoda jį antrajai versijai.

Parameters

<code>studis</code>	Konteineris, į kurį pridedami nuskaityti studentai.
---------------------	---

Returns

`true` jei failas sėkmingai nuskaitytas, `false` klaidos atveju.

Definition at line 43 of file [skaitymas.cpp](#).

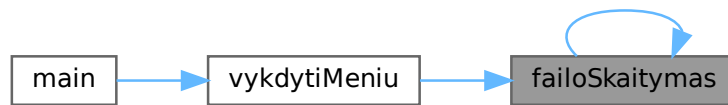
References [failoSkaitymas\(\)](#).

Referenced by [failoSkaitymas\(\)](#), and [vykdytiMenu\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.19.2.2 failoSkaitymas() [2/2]

```
bool failoSkaitymas (
    StudContainer & studis,
    const std::string & pav )
```

Nuskaityti studentus iš nurodyto failo.

Antrarinė perkrovos versija su ekspllicitiniu failo pavadinimu. Failo formatas:
Vardas Pavardė ND1 ND2 ... NDn Egzaminas

Pirma eilutė (antraštė) ignoruojama. Matuoja nuskaitymo laiką ir išsaugo į globalų `timers` objektą.

Parameters

<i>studis</i>	Konteineris, į kurį pridedami nuskaityti studentai.
<i>pav</i>	Failo pavadinimas.

Returns

`true` jei failas sėkmingai nuskaitytas, `false` klaidos atveju.

Definition at line 52 of file [skaitymas.cpp](#).

References [Studentas::getND\(\)](#), [TimerResults::skaitymas](#), and [timers](#).

Here is the call graph for this function:



5.19.2.3 skaitomRanka()

```
void skaitomRanka (  
    Studentas & s )
```

Interaktyviai nuskaito vieno studento duomenis iš konsolės.

Prašo vartotojo įvesti vardą, pavardę ir (pagal pasirinkimą) namų darbų pažymius bei egzamino rezultatą rankiniu būdu arba sugeneruoja juos atsitiktinai.

Parameters

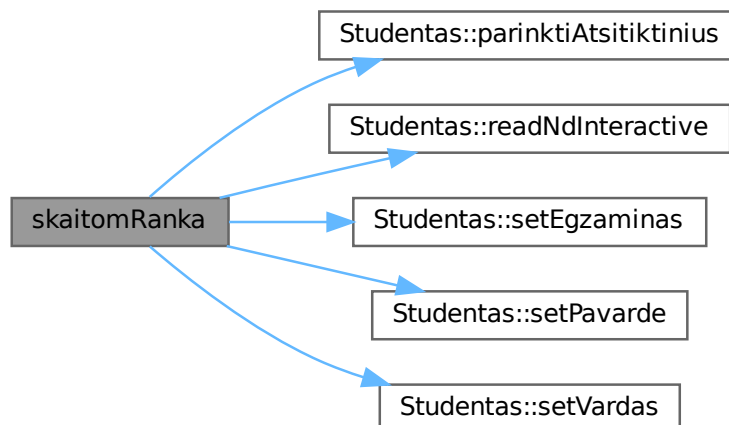
<code>s</code>	Studento objektas, į kurį įrašomi duomenys.
----------------	---

Definition at line 12 of file [skaitymas.cpp](#).

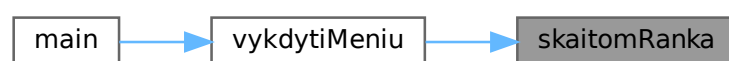
References [Studentas::parinktiAtsitiktinius\(\)](#), [Studentas::readNdInteractive\(\)](#), [Studentas::setEgzaminas\(\)](#), [Studentas::setPavarde\(\)](#), and [Studentas::setVardas\(\)](#).

Referenced by [vykdytiMenu\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.19.2.4 suskaiciuotiGalutinius()

```
void suskaiciuotiGalutinius (
    StudContainer & studis )
```

Apskaičiuoja galutinius balus visiems studentams konteineryje.

Iškviečia `Studentas::suskaiciuotiGalutini()` kiekvienam studentui.

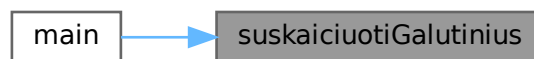
Parameters

<code>studis</code>	Studentų konteineris.
---------------------	-----------------------

Definition at line 99 of file `skaitymas.cpp`.

Referenced by `main()`.

Here is the caller graph for this function:



5.20 skaitymas.h

[Go to the documentation of this file.](#)

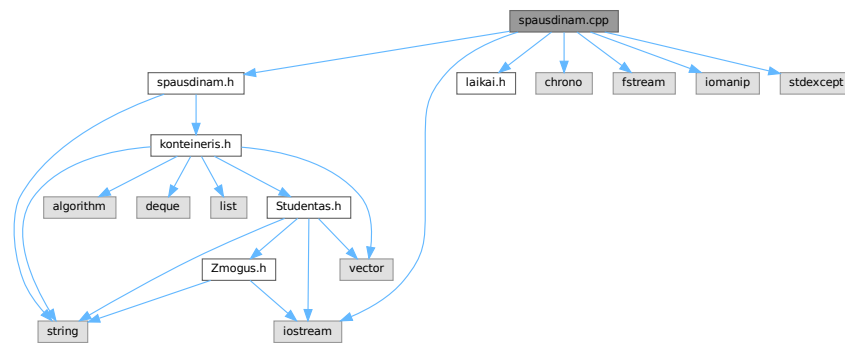
```
00001
00012 #pragma once
00013
00014 #include "konteineris.h"
00015
00016 #include <string>
00017
00028 void skaitomRanka(Studentas& s);
00029
00040 bool failoSkaitymas(StudContainer& studis);
00041
00058 bool failoSkaitymas(StudContainer& studis, const std::string& pav);
00059
00068 void suskaiciuotiGalutinius(StudContainer& studis);
```

5.21 spausdinam.cpp File Reference

```
#include "spausdinam.h"
#include "laikai.h"
#include <chrono>
#include <fstream>
#include <iomanip>
#include <iostream>
```

```
#include <stdexcept>
```

Include dependency graph for spausdinam.cpp:



Functions

- void [surikiuotiStudentus](#) ([StudContainer](#) &studis, int pasirinkimas)
Rikiuoja studentus pagal nurodytą kriterijų (int kodas).
- void [surikiuotiPagalGalutiniVid](#) ([StudContainer](#) &studis)
Rikiuoja studentus pagal galutinį vidurkio balą (didėjančia tvarka).
- void [rikiuotiStudentus](#) ([StudContainer](#) &studis, const std::string &target)
Interaktyviai prašo rikiavimo kriterijaus ir rikiuoja studentus.
- void [spausdinam](#) (const [StudContainer](#) &studis, const std::string &pav)
Išveda studentų sąrašą į nurodytą failą.

5.21.1 Function Documentation

5.21.1.1 rikiuotiStudentus()

```
void rikiuotiStudentus (
    StudContainer & studis,
    const std::string & target )
```

Interaktyviai prašo rikiavimo kriterijaus ir rikiuoja studentus.

Išveda meniu į konsolę, leidžia pasirinkti rikiavimą pagal: vardą, pavardę, galutinį vidurkio arba medianos balą.

Parameters

<i>studis</i>	Rikiuojamas konteineris.
<i>target</i>	Rodomas tekste vardas (pvz. "studentus", "vargsiukus").

Exceptions

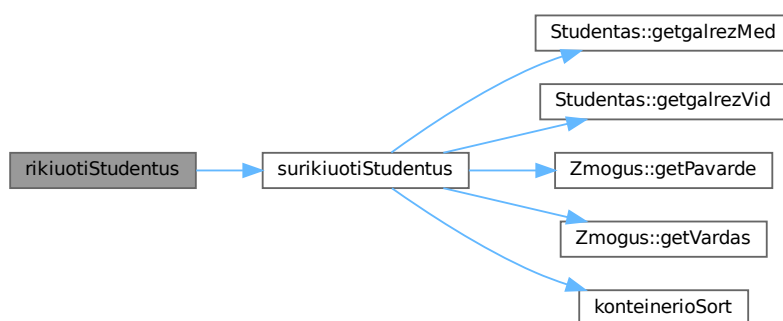
<i>std::runtime_error</i>	Jei vartotojo įvestis neteisinga.
---------------------------	-----------------------------------

Definition at line 55 of file [spausdinam.cpp](#).

References [surikiuotiStudentus\(\)](#).

Referenced by [main\(\)](#), and [skirstymasGrupes\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.21.1.2 spausdinam()

```

void spausdinam (
    const StudContainer & studis,
    const std::string & pav )
  
```

Išveda studentų sąrašą į nurodytą failą.

Rašo lentelę su kolonėlėmis: Vardas, Pavardė, Galutinis(Vid.), Galutinis(Med.).

Parameters

<i>studis</i>	Studentų konteineris.
<i>pav</i>	Išvesties failo pavadinimas.

Definition at line 76 of file [spausdinam.cpp](#).

Referenced by [skirstymasGrupes\(\)](#).

Here is the caller graph for this function:



5.21.1.3 surikiuotiPagalGalutiniVid()

```
void surikiuotiPagalGalutiniVid (
    StudContainer & studis )
```

Rikiuoja studentus pagal galutinį vidurkio balą (didėjančia tvarka).

Patogiausia funkcija tyrimui - nenaudoja interaktyvaus meniu.

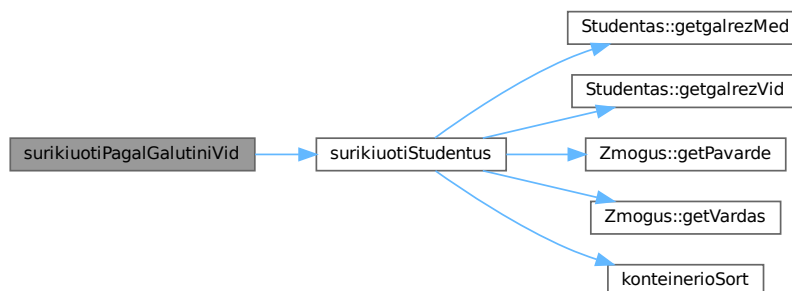
Parameters

<i>studis</i>	Rikiuojamas konteineris.
---------------	--------------------------

Definition at line 50 of file [spausdinam.cpp](#).

References [surikiuotiStudentus\(\)](#).

Here is the call graph for this function:



5.21.1.4 surikiuotiStudentus()

```
void surikiuotiStudentus (
    StudContainer & studis,
    int pasirinkimas )
```

Rikiuoja studentus pagal nurodytą kriterijų (int kodas).

Parameters

<i>studis</i>	Rikiuojamas konteineris.
<i>pasirinkimas</i>	Rikiavimo kriterijus: 1–vardas, 2–pavardė, 3–Vid., 4–Med.

Exceptions

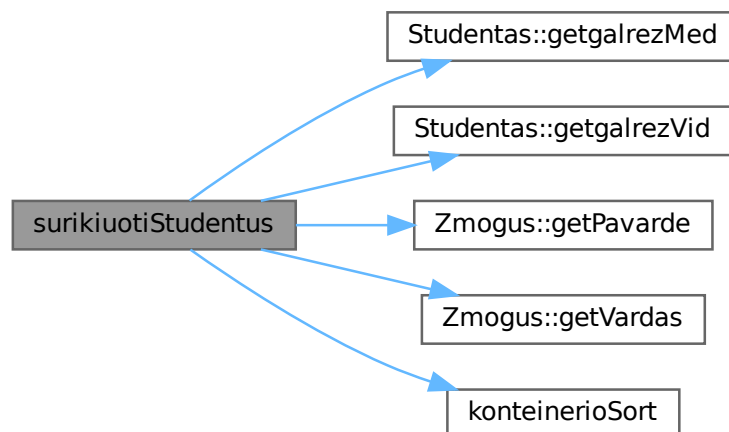
<i>std::runtime_error</i>	Jeigu pasirinkimas nėra 1–4.
---------------------------	------------------------------

Definition at line 10 of file [spausdinam.cpp](#).

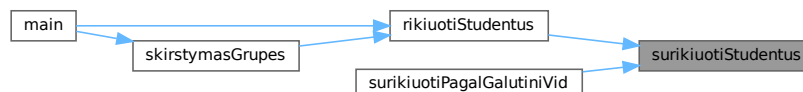
References [Studentas::getgalrezMed\(\)](#), [Studentas::getgalrezVid\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), [konteinerioSort\(\)](#), [TimerResults::rusiavimas](#), and [timers](#).

Referenced by [rikiuotiStudentus\(\)](#), and [surikiuotiPagalGalutiniVid\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.22 spausdinam.cpp

[Go to the documentation of this file.](#)

```

00001 #include "spausdinam.h"
00002 #include "laikai.h"
00003
00004 #include <chrono>
00005 #include <fstream>
00006 #include <iomanip>
00007 #include <iostream>
00008 #include <stdexcept>
00009
00010 void surikiuotiStudentus(StudContainer& studis, int pasirinkimas)
00011 {
00012     if (studis.empty())
00013     {
00014         timers.rusiavimas = 0.0;
00015         return;
00016     }
00017
00018     auto start = std::chrono::high_resolution_clock::now();
00019
00020     switch (pasirinkimas)
00021     {
00022     case 1:
00023         konteinerioSort(studis,
00024             [](const Studentas& a, const Studentas& b)
00025             { return a.getVardas() < b.getVardas(); });
00026         break;
00027     case 2:
00028         konteinerioSort(studis,
00029             [](const Studentas& a, const Studentas& b)
00030             { return a.getPavarde() < b.getPavarde(); });
00031         break;
00032     case 3:
00033         konteinerioSort(studis,
00034             [](const Studentas& a, const Studentas& b)
00035             { return a.getgalrezVid() < b.getgalrezVid(); });
00036         break;
00037     case 4:
00038         konteinerioSort(studis,
00039             [](const Studentas& a, const Studentas& b)
00040             { return a.getgalrezMed() < b.getgalrezMed(); });
00041         break;
00042     default:
00043         throw std::runtime_error("Neteisingas rusiavimo pasirinkimas.");
00044     }
00045
00046     auto end = std::chrono::high_resolution_clock::now();
00047     timers.rusiavimas = std::chrono::duration<double>(end - start).count();
00048 }
00049
00050 void surikiuotiPagalGalutiniVid(StudContainer& studis)
00051 {
00052     surikiuotiStudentus(studis, 3);
00053 }
00054
00055 void rikiuotiStudentus(StudContainer& studis, const std::string& target)
00056 {
00057     if (studis.empty())
00058     {
00059         return;
00060     }
00061
00062     std::cout << "\nPagal ka surikiuoti " << target << "?\n";
00063     std::cout << "1 - Varda\n2 - Pavarde\n3 - Galutinis (Vid.)\n4 - Galutinis (Med.)\n";
00064
00065     int p = 0;
00066     std::cin >> p;
00067
00068     if (!std::cin || p < 1 || p > 4)
00069     {
00070         throw std::runtime_error("Neteisingas rusiavimo pasirinkimas.");
00071     }
00072
00073     surikiuotiStudentus(studis, p);
00074 }
00075
00076 void spausdinam(const StudContainer& studis, const std::string& pav)
00077 {
00078     std::ofstream write(pav);
00079
00080     if (!write.is_open())
00081     {
00082         std::cerr << "Nepavyko atidaryti failo: " << pav << std::endl;

```

```

00083     return;
00084 }
00085
00086 write « std::left
00087     « std::setw(20) « "Vardas"
00088     « std::setw(20) « "Pavarde"
00089     « std::setw(20) « "Galutinis(Vid.)"
00090     « std::setw(15) « "Galutinis(Med.)" « '\n';
00091 write « std::string(75, '-') « '\n';
00092
00093 for (const auto& s : studis)
00094 {
00095     write « std::left
00096         « std::setw(20) « s.getVardas()
00097         « std::setw(20) « s.getPavarde()
00098         « std::setw(20) « std::fixed « std::setprecision(2) « s.getgalrezVid()
00099         « std::setw(15) « std::fixed « std::setprecision(2) « s.getgalrezVid() « '\n';
00100 }
00101 }

```

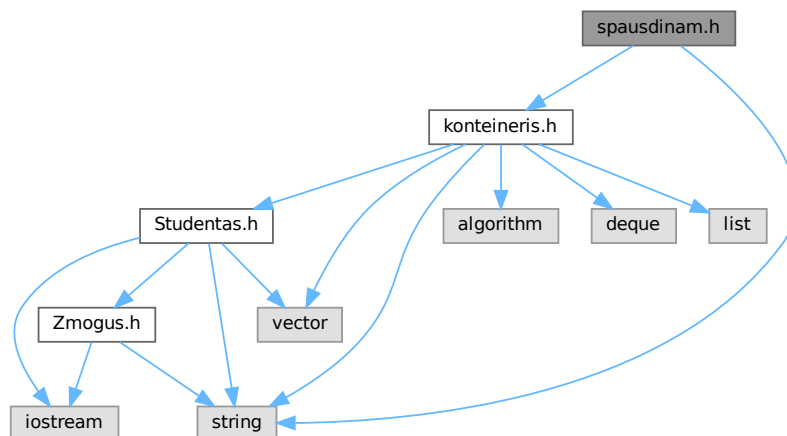
5.23 spausdinam.h File Reference

Studentų duomenų rikiavimo ir išvedimo funkcijų deklaracijos.

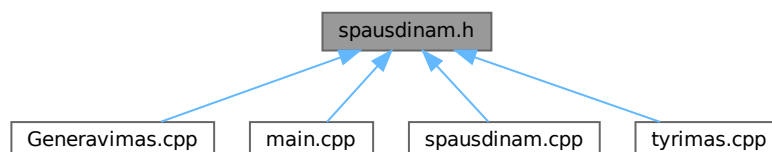
```
#include "konteineris.h"
```

```
#include <string>
```

Include dependency graph for spausdinam.h:



This graph shows which files directly or indirectly include this file:



Functions

- void [spausdinam](#) (const [StudContainer](#) &studis, const std::string &pav)
Išveda studentų sąrašą į nurodytą failą.
- void [rikiuotiStudentus](#) ([StudContainer](#) &studis, const std::string &target)
Interaktyviai prašo rikiavimo kriterijaus ir rikiuoja studentus.
- void [surikiuotiStudentus](#) ([StudContainer](#) &studis, int pasirinkimas)
Rikiuoja studentus pagal nurodytą kriterijų (int kodas).
- void [surikiuotiPagalGalutiniVid](#) ([StudContainer](#) &studis)
Rikiuoja studentus pagal galutinį vidurkio balą (didėjančia tvarka).

5.23.1 Detailed Description

Studentų duomenų rikiavimo ir išvedimo funkcijų deklaracijos.

Pateikia funkcijas studentų rikiavimui pagal įvairius kriterijus ir duomenų išvedimui į failą.

Author

[Studentas](#)

Version

2.0

Definition in file [spausdinam.h](#).

5.23.2 Function Documentation

5.23.2.1 rikiuotiStudentus()

```
void rikiuotiStudentus (
    StudContainer & studis,
    const std::string & target )
```

Interaktyviai prašo rikiavimo kriterijaus ir rikiuoja studentus.

Išveda meniu į konsolę, leidžia pasirinkti rikiavimą pagal: vardą, pavardę, galutinį vidurkio arba medianos balą.

Parameters

<i>studis</i>	Rikiuojamas konteineris.
<i>target</i>	Rodomas tekste vardas (pvz. "studentus", "vargsiukus").

Exceptions

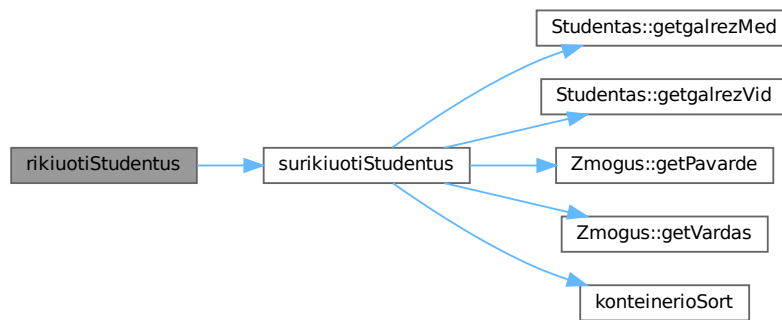
<i>std::runtime_error</i>	Jei vartotojo įvestis neteisinga.
---------------------------	-----------------------------------

Definition at line 55 of file [spausdinam.cpp](#).

References [surikiuotiStudentus\(\)](#).

Referenced by [main\(\)](#), and [skirstymasGrupes\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.23.2.2 spausdinam()

```

void spausdinam (
    const StudContainer & studis,
    const std::string & pav )
  
```

Išveda studentų sąrašą į nurodytą failą.

Rašo lentelę su kolonėlėmis: Vardas, Pavardė, Galutinis(Vid.), Galutinis(Med.).

Parameters

<i>studis</i>	Studentų konteineris.
<i>pav</i>	Išvesties failo pavadinimas.

Definition at line 76 of file [spausdinam.cpp](#).

Referenced by [skirstymasGrupes\(\)](#).

Here is the caller graph for this function:



5.23.2.3 surikiuotiPagalGalutiniVid()

```
void surikiuotiPagalGalutiniVid (
    StudContainer & studis )
```

Rikiuoja studentus pagal galutinį vidurkio balą (didėjančia tvarka).

Patogiausia funkcija tyrimui - nenaudoja interaktyvaus meniu.

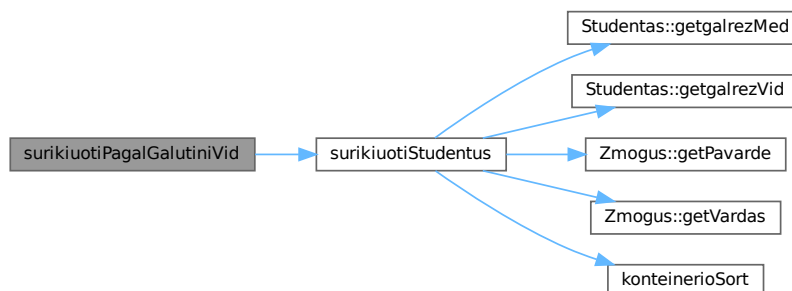
Parameters

<i>studis</i>	Rikiuojamas konteineris.
---------------	--------------------------

Definition at line 50 of file [spausdinam.cpp](#).

References [surikiuotiStudentus\(\)](#).

Here is the call graph for this function:



5.23.2.4 surikiuotiStudentus()

```
void surikiuotiStudentus (
    StudContainer & studis,
    int pasirinkimas )
```

Rikiuoja studentus pagal nurodytą kriterijų (int kodas).

Parameters

<i>studis</i>	Rikiuojamas konteineris.
<i>pasirinkimas</i>	Rikiavimo kriterijus: 1–vardas, 2–pavardė, 3–Vid., 4–Med.

Exceptions

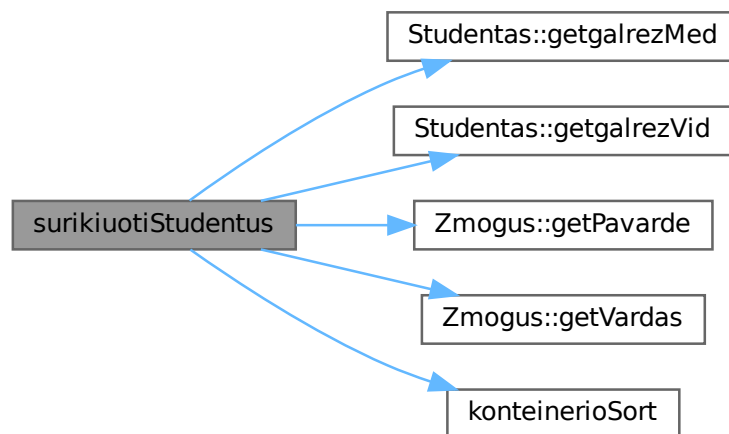
<i>std::runtime_error</i>	Jei <i>pasirinkimas</i> nėra 1–4.
---------------------------	-----------------------------------

Definition at line 10 of file [spausdinam.cpp](#).

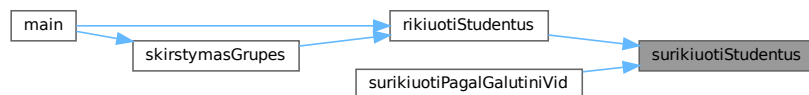
References [Studentas::getgalrezMed\(\)](#), [Studentas::getgalrezVid\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), [konteinerioSort\(\)](#), [TimerResults::rusiavimas](#), and [timers](#).

Referenced by [rikiuotiStudentus\(\)](#), and [surikiuotiPagalGalutiniVid\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.24 spausdinam.h

[Go to the documentation of this file.](#)

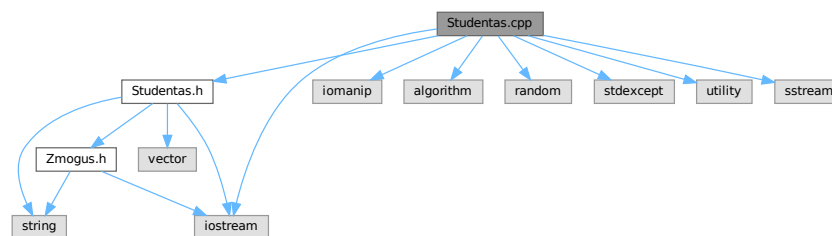
```
00001
00012 #pragma once
00013
00014 #include "konteineris.h"
00015
00016 #include <string>
00017
00027 void spausdinam(const StudContainer& studis, const std::string& pav);
00028
00040 void rikiuotiStudentus(StudContainer& studis, const std::string& target);
00041
00050 void surikiuotiStudentus(StudContainer& studis, int pasirinkimas);
00051
00060 void surikiuotiPagalGalutiniVid(StudContainer& studis);
```

5.25 Studentas.cpp File Reference

[Studentas](#) klasės metodų realizacija.

```
#include "Studentas.h"
#include <iomanip>
#include <algorithm>
#include <iostream>
#include <random>
#include <stdexcept>
#include <utility>
#include <sstream>
```

Include dependency graph for Studentas.cpp:



5.25.1 Detailed Description

[Studentas](#) klasės metodų realizacija.

Realizuojami visi Rule of Five metodai, įvesties/išvesties operatoriai, galutinio balo skaičiavimas ir pagalbinės lyginimo funkcijos.

Author

[Studentas](#)

Version

2.0

Definition in file [Studentas.cpp](#).

5.25.2 Function Documentation

5.25.2.1 comparePagalGalMed()

```
bool comparePagalGalMed (
    const Studentas & a,
    const Studentas & b ) [related]
```

Rikiavimas pagal galutinį medianos balą (didėjančiai).

Definition at line 352 of file [Studentas.cpp](#).

5.25.2.2 comparePagalGalVid()

```
bool comparePagalGalVid (
    const Studentas & a,
    const Studentas & b ) [related]
```

Rikiavimas pagal galutinį vidurkio balą (didėjančiai).

Definition at line 346 of file [Studentas.cpp](#).

5.25.2.3 comparePagalPavarde()

```
bool comparePagalPavarde (
    const Studentas & a,
    const Studentas & b ) [related]
```

Rikiavimas pagal pavardę (didėjančiai).

Definition at line 340 of file [Studentas.cpp](#).

5.25.2.4 comparePagalVarda()

```
bool comparePagalVarda (
    const Studentas & a,
    const Studentas & b ) [related]
```

Rikiavimas pagal vardą (didėjančiai).

Definition at line 334 of file [Studentas.cpp](#).

5.25.2.5 operator<<()

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s ) [related]
```

Išvesties operatorius.

Delegacija į [Studentas::spausdinti\(\)](#) (virtualus dispatch).

Definition at line 196 of file [Studentas.cpp](#).

5.25.2.6 operator>>()

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s ) [related]
```

Įvesties operatorius.

Nuskaito duomenis ir apskaičiuoja galutinį balą.

Definition at line 207 of file [Studentas.cpp](#).

5.26 Studentas.cpp

[Go to the documentation of this file.](#)

```
00001
00012 #include "Studentas.h"
00013
00014 #include <iomanip>
00015 #include <algorithm>
00016 #include <iostream>
00017 #include <random>
00018 #include <stdexcept>
00019 #include <utility>
00020 #include <sstream>
00021
00022
00024 Studentas::Studentas() : egzaminas(0), vidurkis(0.0f), galrezVid(0.0f), galrezMed(0.0f)
00025 {
00026 }
00027
00034 Studentas::Studentas(std::istream& is): egzaminas(0), vidurkis(0.0f), galrezVid(0.0f), galrezMed(0.0f)
00035 {
00036     readStudent(is);
00037 }
00038
00044 Studentas::Studentas(const Studentas& other): Zmogus(other),
00045     egzaminas(other.egzaminas),
00046     nd(other.nd),
00047     vidurkis(other.vidurkis),
00048     galrezVid(other.galrezVid),
00049     galrezMed(other.galrezMed)
00050 {
00051 }
00052
00060 Studentas& Studentas::operator=(const Studentas& other)
00061 {
00062     if (this == &other)
00063     {
00064         return *this;
00065     }
00066
00067     Zmogus::operator=(other); //vardas, pavarde
00068     egzaminas = other.egzaminas;
00069     nd = other.nd;
00070     vidurkis = other.vidurkis;
00071     galrezVid = other.galrezVid;
00072     galrezMed = other.galrezMed;
00073
00074     return *this;
00075 }
00076
00084 Studentas::Studentas(Studentas&& other): Zmogus(std::move(other)),
00085     egzaminas(other.egzaminas),
00086     nd(std::move(other.nd)),
00087     vidurkis(other.vidurkis),
00088     galrezVid(other.galrezVid),
00089     galrezMed(other.galrezMed)
00090 {
00091     other.egzaminas = 0;
00092     other.vidurkis = 0.0f;
00093     other.galrezVid = 0.0f;
00094     other.galrezMed = 0.0f;
00095 }
00096
```

```

00104 Studentas& Studentas::operator=(Studentas&& other)
00105 {
00106     if (this == &other)
00107     {
00108         return *this;
00109     }
00110
00111     Zmogus::operator=(std::move(other));
00112     egzaminas = other.egzaminas;
00113     nd = std::move(other.nd);
00114     vidurkis = other.vidurkis;
00115     galrezVid = other.galrezVid;
00116     galrezMed = other.galrezMed;
00117
00118     other.egzaminas = 0;
00119     other.vidurkis = 0.0f;
00120     other.galrezVid = 0.0f;
00121     other.galrezMed = 0.0f;
00122
00123     return *this;
00124 }
00125
00126
00133 void Studentas::spausdinti(std::ostream& os) const
00134 {
00135     os << std::left << std::setw(15) << vardas
00136         << std::setw(15) << pavarde
00137         << "Egz: " << std::setw(4) << egzaminas << "ND: [";
00138
00139     for (std::size_t i = 0; i < nd.size(); ++i)
00140     {
00141         if (i > 0) os << ' ';
00142         os << nd[i];
00143     }
00144
00145     os << ']'
00146         << std::fixed << std::setprecision(2)
00147         << " Gal.Vid: " << galrezVid
00148         << " Gal.Med: " << galrezMed;
00149 }
00150
00162 void Studentas::suskaiciuotiGalutini()
00163 {
00164     std::vector<int> surikiuoti = nd;
00165     std::sort(surikiuoti.begin(), surikiuoti.end());
00166
00167     const int n = static_cast<int>(surikiuoti.size());
00168
00169     if (n == 0)
00170     {
00171         galrezVid = 0.0f;
00172         galrezMed = 0.0f;
00173         return;
00174     }
00175
00176     float med = 0.0f;
00177     if (n % 2 == 1)
00178     {
00179         med = static_cast<float>(surikiuoti[n / 2]);
00180     }
00181     else
00182     {
00183         med = (static_cast<float>(surikiuoti[n / 2]) +
00184             static_cast<float>(surikiuoti[n / 2 - 1])) / 2.0f;
00185     }
00186
00187     galrezVid = 0.4f * vidurkis + 0.6f * static_cast<float>(egzaminas);
00188     galrezMed = 0.4f * med + 0.6f * static_cast<float>(egzaminas);
00189 }
00190
00196 std::ostream& operator<<(std::ostream& os, const Studentas& s)
00197 {
00198     s.spausdinti(os);
00199     return os;
00200 }
00201
00207 std::istream& operator>>(std::istream& is, Studentas& s)
00208 {
00209     s.readStudent(is);
00210     s.suskaiciuotiGalutini();
00211     return is;
00212 }
00213
00220 std::istream& Studentas::readStudent(std::istream& is)
00221 {
00222     std::string line;
00223     if (!std::getline(is, line) || line.empty())

```

```

00224     {
00225         return is;
00226     }
00227     std::istringstream ss(line);
00228
00229     ss » vardas » pavarde;
00230
00231     nd.clear();
00232     vidurkis = 0.0f;
00233     egzaminas = 0;
00234
00235     int x = 0;
00236
00237     while (ss » x)
00238     {
00239         nd.push_back(x);
00240     }
00241
00242     if (!nd.empty())
00243     {
00244         egzaminas = nd.back();
00245         nd.pop_back();
00246
00247         for (const int p : nd)
00248         {
00249             vidurkis += static_cast<float>(p);
00250         }
00251
00252         if (!nd.empty())
00253         {
00254             vidurkis /= static_cast<float>(nd.size());
00255         }
00256     }
00257
00258     return is;
00259 }
00260
00266 void Studentas::readNdInteractive()
00267 {
00268     std::cout << "Iveskite namu darbu pazymius (0 - pabaiga):" << std::endl;
00269
00270     nd.clear();
00271     vidurkis = 0.0f;
00272
00273     while (true)
00274     {
00275         int x = 0;
00276         std::cin » x;
00277
00278         if (!std::cin)
00279         {
00280             throw std::runtime_error("Kazkas ne taip su cin");
00281         }
00282
00283         if (x <= 0)
00284         {
00285             break;
00286         }
00287
00288         if (x > 10)
00289         {
00290             std::cout << "Netinkamas skaicius. Iveskite tarp 1 ir 10." << std::endl;
00291             continue;
00292         }
00293
00294         nd.push_back(x);
00295         vidurkis += static_cast<float>(x);
00296     }
00297
00298     if (nd.empty())
00299     {
00300         throw std::runtime_error("Nepavyko nuskaityti ne vieno pazymio!");
00301     }
00302
00303     vidurkis /= static_cast<float>(nd.size());
00304 }
00305
00312 void Studentas::parinktiAtsitiktinius()
00313 {
00314     std::mt19937 gen(std::random_device{}());
00315     std::uniform_int_distribution<int> dist(1, 10);
00316
00317     const int kiekND = dist(gen);
00318
00319     nd.clear();
00320     vidurkis = 0.0f;
00321

```

```

00322     for (int i = 0; i < kiekND; i++)
00323     {
00324         const int balas = dist(gen);
00325         nd.push_back(balas);
00326         vidurkis += static_cast<float>(balas);
00327     }
00328
00329     vidurkis /= static_cast<float>(nd.size());
00330     egzaminas = dist(gen);
00331 }
00332
00334 bool comparePagalVarda(const Studentas& a, const Studentas& b)
00335 {
00336     return a.getVardas() < b.getVardas();
00337 }
00338
00340 bool comparePagalPavarde(const Studentas& a, const Studentas& b)
00341 {
00342     return a.getPavarde() < b.getPavarde();
00343 }
00344
00346 bool comparePagalGalVid(const Studentas& a, const Studentas& b)
00347 {
00348     return a.getgalrezVid() < b.getgalrezVid();
00349 }
00350
00352 bool comparePagalGalMed(const Studentas& a, const Studentas& b)
00353 {
00354     return a.getgalrezMed() < b.getgalrezMed();
00355 }

```

5.27 Studentas.h File Reference

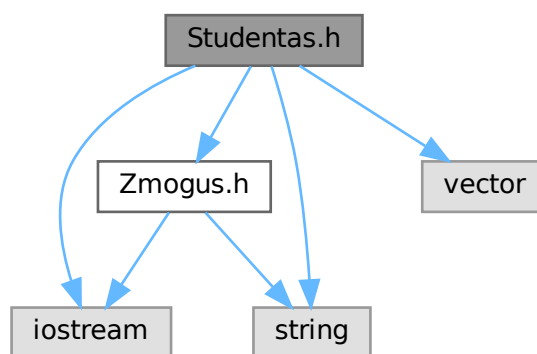
Studento klasės apibrėžimas.

```

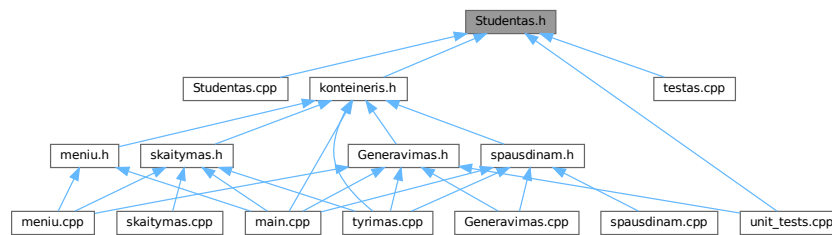
#include "Zmogus.h"
#include <iostream>
#include <string>
#include <vector>

```

Include dependency graph for Studentas.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Studentas](#)

Konkreči klasė, paveldinti iš [Zmogus](#).

5.27.1 Detailed Description

Studento klasės apibrėžimas.

Realizuoja [Zmogus](#) abstrakčią klasę konkretaus studento atveju. Saugo namų darbų pažymius, egzamino pažymį ir skaičiuoja galutinius įvertinimus pagal vidurkį bei medianą.

Author

[Studentas](#)

Version

2.0

Definition in file [Studentas.h](#).

5.28 Studentas.h

[Go to the documentation of this file.](#)

```

00001
00013 #pragma once
00014 #include "Zmogus.h"
00015
00016 #include <iostream>
00017 #include <string>
00018 #include <vector>
00019
00020
00046 class Studentas : public Zmogus
00047 {
00048 private:
00049     int egzaminas;
00050     std::vector<int> nd;
00051     float vidurkis;
00052     float galrezVid;
00053     float galrezMed;
00054

```

```

00055 public:
00065     Studentas(); // Default konstruktorius
00066
00075     explicit Studentas(std::istream& is); // stream konstruktorius
00076
00084     Studentas(const Studentas& other); // Kopijavimo konstruktorius
00085
00095     Studentas& operator=(const Studentas& other); // Kopijavimo priskyrimo operatorius
00096
00105     Studentas(Studentas&& other); // Perkélimo konstruktorius
00106
00116     Studentas& operator=(Studentas&& other); // Perkélimo priskyrimo operatorius
00117
00123     ~Studentas() override = default; // Destruktorius
00124
00129     inline const std::vector<int>& getND() const { return nd; }
00130
00135     inline int getEgzaminas() const { return egzaminas; }
00136
00141     inline float getVidurkis() const { return vidurkis; }
00142
00147     inline float getgalrezVid() const { return galrezVid; }
00148
00153     inline float getgalrezMed() const { return galrezMed; }
00154
00155     // -----
00160     // @brief rankinio ívedimo atvejui
00165     void setVardas(const std::string& v) { vardas = v; }
00166
00171     void setPavarde(const std::string& p) { pavarde = p; }
00172
00177     void setEgzaminas(int e) { egzaminas = e; }
00178
00181     // -----
00195     std::istream& readStudent(std::istream& is); // skaity vardas/pavardé/nd/egz iš stream
00196
00205     void readNdInteractive();
00206
00213     void parinktiAtsitiktinius();
00214
00224     void suskaiciuotiGalutini() override;
00225
00233     void spausdinti(std::ostream& os) const override;
00234
00236 };
00237
00238 // -----
00253     std::ostream& operator<<(std::ostream& os, const Studentas& s);
00254
00264     std::istream& operator>>(std::istream& is, Studentas& s);
00265
00272     bool comparePagalVarda (const Studentas& a, const Studentas& b);
00273
00280     bool comparePagalPavarde (const Studentas& a, const Studentas& b);
00281
00288     bool comparePagalGalVid (const Studentas& a, const Studentas& b);
00289
00296     bool comparePagalGalMed (const Studentas& a, const Studentas& b);

```

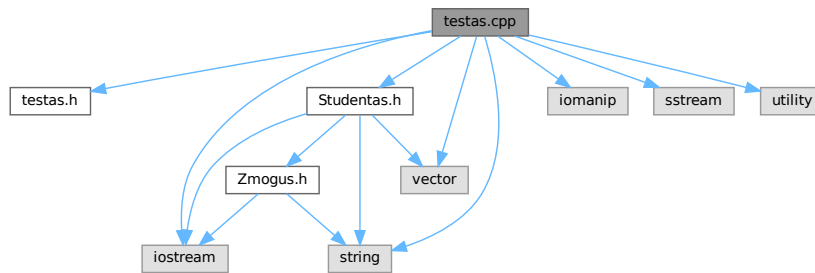
5.29 testas.cpp File Reference

```

#include "testas.h"
#include "Studentas.h"
#include <iomanip>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
#include <utility>

```

Include dependency graph for `testas.cpp`:



Functions

- static void [testuotiDefaultKonstruktoriu](#) ()
- static void [testuotiSrautiniKonstruktoriu](#) ()
- static void [testuotiKopijavimoKonstruktoriu](#) ()
- static void [testuotiKopijavimoPriskyrima](#) ()
- static void [testuotiPerkelimoKonstruktoriu](#) ()
- static void [testuotiPerkelimoPriskyrima](#) ()
- static void [testuotiDestruktoriu](#) ()
- static void [testuotiIstreamOperatoriu](#) ()
- static void [testuotiOstreamOperatoriu](#) ()
- void [vykdytiTestus](#) ()
- void [testuotiAbstraktuma](#) ()

5.29.1 Function Documentation

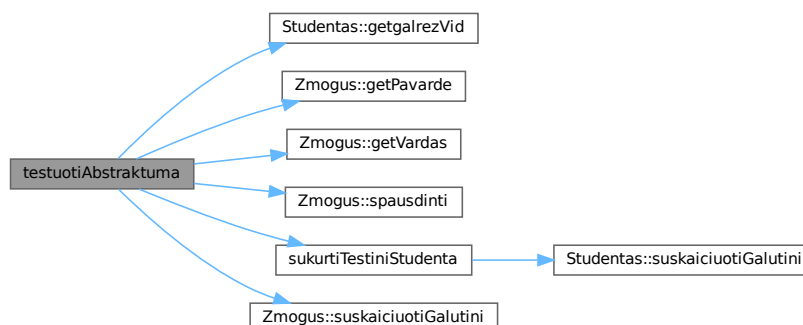
5.29.1.1 testuotiAbstraktuma()

```
void testuotiAbstraktuma ( )
```

Definition at line 268 of file `testas.cpp`.

References [Studentas::getgalrezVid\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), [Zmogus::spausdinti\(\)](#), [sukurtiTestiniStudenta\(\)](#), and [Zmogus::suskaiciuotiGalutini\(\)](#).

Here is the call graph for this function:



5.29.1.2 testuotiDefaultKonstruktoriu()

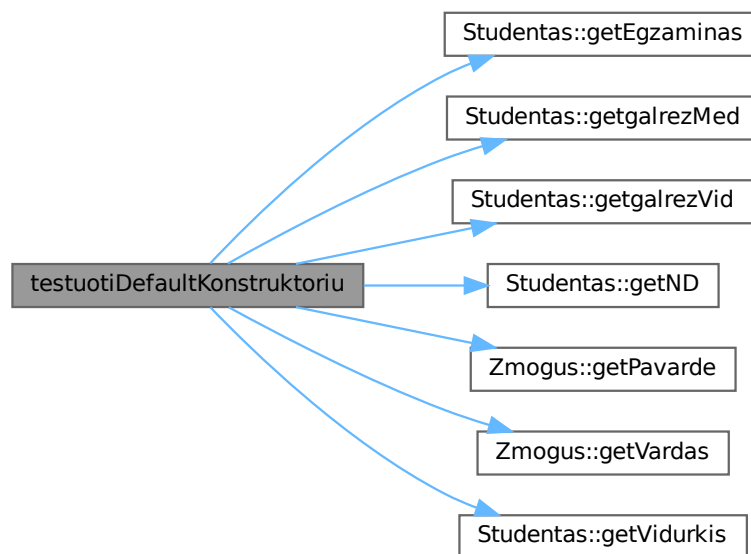
```
static void testuotiDefaultKonstruktoriu ( ) [static]
```

Definition at line 34 of file [testas.cpp](#).

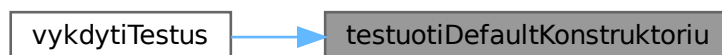
References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezMed\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), and [Studentas::getVidurkis\(\)](#).

Referenced by [vykdytiTestus\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.1.3 testuotiDestruktoriu()

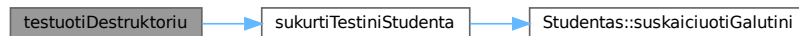
```
static void testuotiDestruktoriu ( ) [static]
```

Definition at line 153 of file [testas.cpp](#).

References [sukurtiTestiniStudenta\(\)](#).

Referenced by [vykdytiTestus\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.1.4 testuotiIstreamOperatoriu()

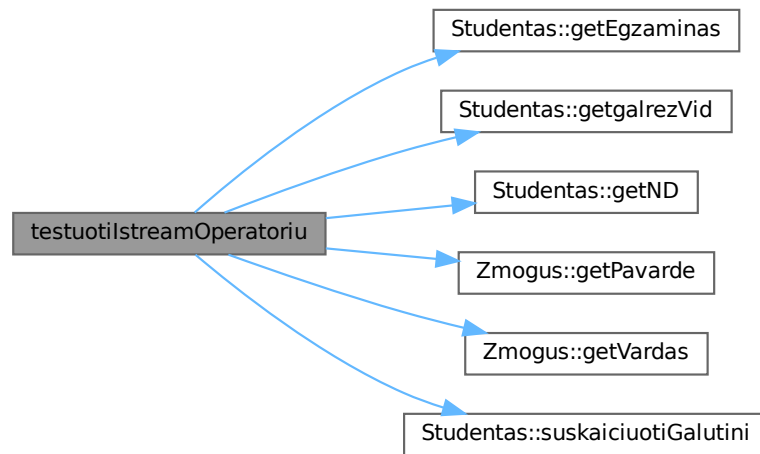
```
static void testuotiIstreamOperatoriu ( ) [static]
```

Definition at line 174 of file [testas.cpp](#).

References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), and [Studentas::suskaiciuotiGalutini\(\)](#).

Referenced by [vykdytiTestus\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.1.5 testuotiKopijavimoKonstruktoriu()

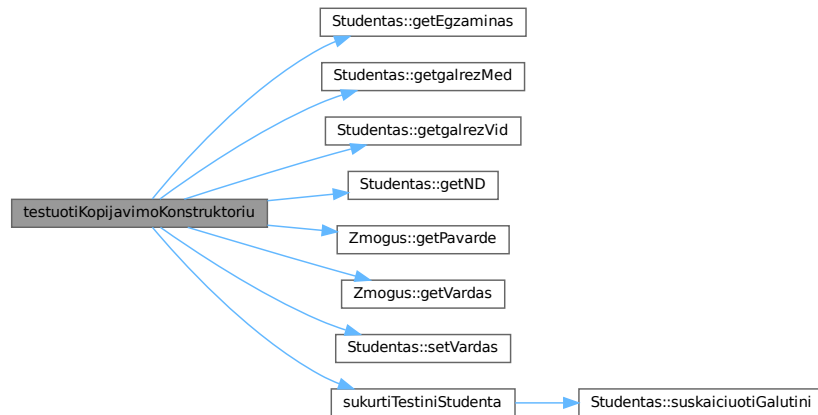
```
static void testuotiKopijavimoKonstruktoriu ( ) [static]
```

Definition at line 66 of file `testas.cpp`.

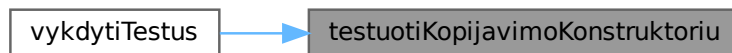
References `Studentas::getEgzaminas()`, `Studentas::getgalrezMed()`, `Studentas::getgalrezVid()`, `Studentas::getND()`, `Zmogus::getPavarde()`, `Zmogus::getVardas()`, `Studentas::setVardas()`, and `sukurtiTestiniStudenta()`.

Referenced by `vykdytiTestus()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.1.6 testuotiKopijavimoPriskyrima()

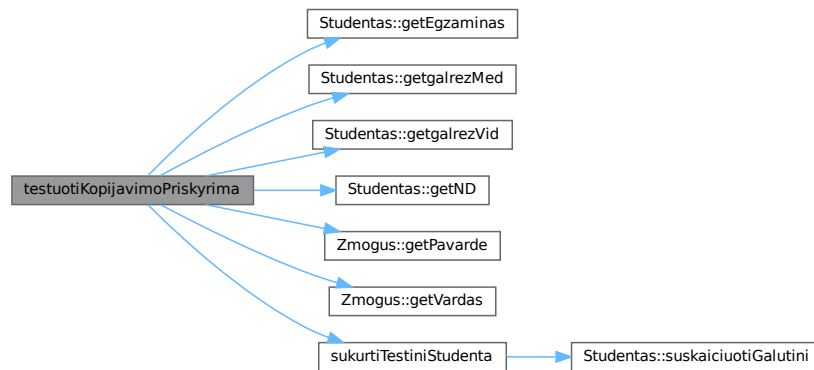
```
static void testuotiKopijavimoPriskyrima ( ) [static]
```

Definition at line 85 of file [testas.cpp](#).

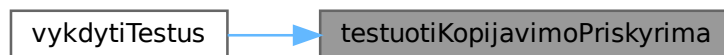
References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezMed\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Referenced by [vykdytiTestus\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.1.7 testuotiOstreamOperatoriu()

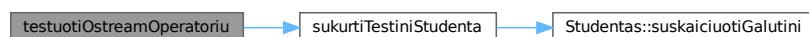
```
static void testuotiOstreamOperatoriu ( ) [static]
```

Definition at line 214 of file [testas.cpp](#).

References [sukurtiTestiniStudenta\(\)](#).

Referenced by [vykdytiTestus\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.1.8 testuotiPerkelimoKonstruktoriu()

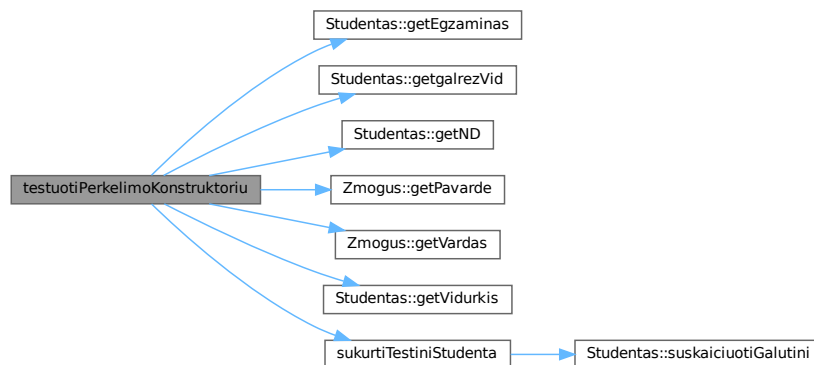
```
static void testuotiPerkelimoKonstruktoriu ( ) [static]
```

Definition at line 104 of file `testas.cpp`.

References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), [Studentas::getVidurkis\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Referenced by [vykdytiTestus\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.1.9 testuotiPerkelimoPriskyrima()

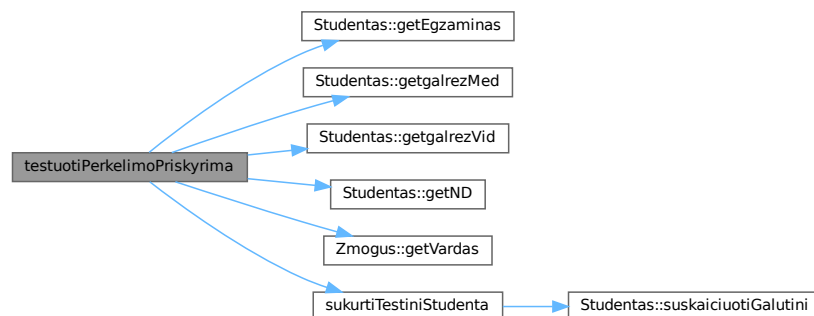
```
static void testuotiPerkelimoPriskyrima ( ) [static]
```

Definition at line 130 of file [testas.cpp](#).

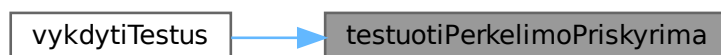
References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezMed\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getVardas\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Referenced by [vykdytiTestus\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.1.10 testuotiSrautiniKonstruktoriu()

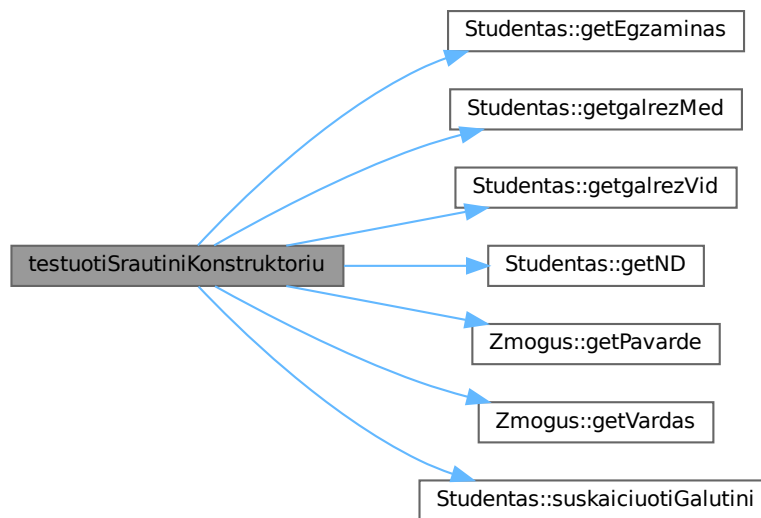
```
static void testuotiSrautiniKonstruktoriu ( ) [static]
```

Definition at line 49 of file [testas.cpp](#).

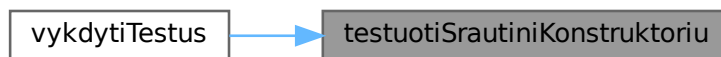
References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezMed\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), and [Studentas::suskaiciuotiGalutini\(\)](#).

Referenced by [vykdytiTestus\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.29.1.11 vykdytiTestus()

```
void vykdytiTestus ( )
```

Definition at line 248 of file [testas.cpp](#).

References [testuotiDefaultKonstruktoriu\(\)](#), [testuotiDestruktoriu\(\)](#), [testuotilStreamOperatoriu\(\)](#), [testuotiKopijavimoKonstruktoriu\(\)](#), [testuotiKopijavimoPriskyrima\(\)](#), [testuotiOstreamOperatoriu\(\)](#), [testuotiPerkelimoKonstruktoriu\(\)](#), [testuotiPerkelimoPriskyrima\(\)](#), and [testuotiSrautiniKonstruktoriu\(\)](#).

Here is the call graph for this function:



5.30 testas.cpp

[Go to the documentation of this file.](#)

```

00001 #include "testas.h"
00002 #include "Studentas.h"
00003
00004 #include <iomanip>
00005 #include <iostream>
00006 #include <sstream>
00007 #include <string>
00008 #include <vector>
00009 #include <utility>
00010
00011 namespace
00012 {
00013
00014     int praejo = 0;
00015     int nepraejo = 0;
00016
00017     void tikrinti(const std::string& pavadinimas, bool salyga)
00018     {
00019         const std::string zenklas = salyga ? "Pavyko --->" : "Nepavyko!!! --->";
00020         std::cout << zenklas << " " << pavadinimas << '\n';
00021         salyga ? praejo++ : nepraejo++;
00022     }
00023
00024     Studentas sukurtiTestiniStudenta()
00025     {
00026         std::istringstream ss("Jonas Jonaitis 8 7 9 6 10");
00027         Studentas s(ss);
00028         s.suskaiciuotiGalutini();
00029         return s;
00030     }
00031
00032 }
00033
00034 static void testuotiDefaultKonstruktoriu()
00035 {
00036     std::cout << "\n[Default konstruktorius]\n";
00037
00038     Studentas s;
00039
00040     tikrinti("vardas tuscias", s.getVardas().empty());
00041     tikrinti("pavarde tuscias", s.getPavarde().empty());
00042     tikrinti("egzaminas == 0", s.getEgzaminas() == 0);
00043     tikrinti("nd tuscias", s.getND().empty());
00044     tikrinti("vidurkis == 0.0f", s.getVidurkis() == 0.0f);
00045     tikrinti("galrezVid == 0.0f", s.getgalrezVid() == 0.0f);
00046     tikrinti("galrezMed == 0.0f", s.getgalrezMed() == 0.0f);

```

```

00047 }
00048
00049 static void testuotiSrautiniKonstruktoriu()
00050 {
00051     std::cout << "\n[Srautinis konstruktorius]\n";
00052
00053     std::istringstream ss("Pedro Pascal 5 6 7 8 9");
00054     Studentas s(ss);
00055     s.suskaiciuotiGalutini();
00056
00057     tikrinti("vardas == Pedro", s.getVardas() == "Pedro");
00058     tikrinti("pavarde == Pascal", s.getPavarde() == "Pascal");
00059     tikrinti("egzaminas == 9", s.getEgzaminas() == 9);
00060     tikrinti("nd.size() == 4", s.getND().size() == 4);
00061     tikrinti("nd[0] == 5", s.getND()[0] == 5);
00062     tikrinti("galrezVid > 0", s.getgalrezVid() > 0.0f);
00063     tikrinti("galrezMed > 0", s.getgalrezMed() > 0.0f);
00064 }
00065
00066 static void testuotiKopijavimoKonstruktoriu()
00067 {
00068     std::cout << "\n[Kopijavimo konstruktorius]\n";
00069
00070     Studentas original = sukurtiTestiniStudenta();
00071     Studentas kopija(original);
00072
00073     tikrinti("vardas sutampa", kopija.getVardas() == original.getVardas());
00074     tikrinti("pavarde sutampa", kopija.getPavarde() == original.getPavarde());
00075     tikrinti("egzaminas sutampa", kopija.getEgzaminas() == original.getEgzaminas());
00076     tikrinti("nd sutampa", kopija.getND() == original.getND());
00077     tikrinti("galrezVid sutampa", kopija.getgalrezVid() == original.getgalrezVid());
00078     tikrinti("galrezMed sutampa", kopija.getgalrezMed() == original.getgalrezMed());
00079
00080     kopija.setVardas("Kitas");
00081     tikrinti("originalas nepakeistas po kopijos keitimo", original.getVardas() == "Jonas");
00082 }
00083
00084 static void testuotiKopijavimoPriskyrima()
00085 {
00086     std::cout << "\n[Kopijavimo priskyrimo operatorius]\n";
00087
00088     Studentas original = sukurtiTestiniStudenta();
00089     Studentas priskirtas;
00090     priskirtas = original;
00091
00092     tikrinti("vardas sutampa", priskirtas.getVardas() == original.getVardas());
00093     tikrinti("pavarde sutampa", priskirtas.getPavarde() == original.getPavarde());
00094     tikrinti("egzaminas sutampa", priskirtas.getEgzaminas() == original.getEgzaminas());
00095     tikrinti("nd sutampa", priskirtas.getND() == original.getND());
00096     tikrinti("galrezVid sutampa", priskirtas.getgalrezVid() == original.getgalrezVid());
00097     tikrinti("galrezMed sutampa", priskirtas.getgalrezMed() == original.getgalrezMed());
00098
00099     priskirtas = priskirtas;
00100     tikrinti("savipriskyrimas saugus", priskirtas.getVardas() == original.getVardas());
00101 }
00102
00103 static void testuotiPerkelimoKonstruktoriu()
00104 {
00105     std::cout << "\n[Perkelimo konstruktorius]\n";
00106
00107     Studentas saltinis = sukurtiTestiniStudenta();
00108
00109     const std::string laukiamasVardas = saltinis.getVardas();
00110     const std::string laukiamaPavarde = saltinis.getPavarde();
00111     const int laukiamasEgz = saltinis.getEgzaminas();
00112     const float laukiamasGalVid = saltinis.getgalrezVid();
00113     const std::size_t laukiamasNdDydis = saltinis.getND().size();
00114
00115     Studentas tikslas(std::move(saltinis));
00116
00117     tikrinti("tikslas: vardas teisingas", tikslas.getVardas() == laukiamasVardas);
00118     tikrinti("tikslas: pavarde teisinga", tikslas.getPavarde() == laukiamaPavarde);
00119     tikrinti("tikslas: egzaminas teisingas", tikslas.getEgzaminas() == laukiamasEgz);
00120     tikrinti("tikslas: nd dydis teisingas", tikslas.getND().size() == laukiamasNdDydis);
00121     tikrinti("tikslas: galrezVid teisingas", tikslas.getgalrezVid() == laukiamasGalVid);
00122
00123     // Šaltinis po perkėlimo - turi būti "tuščias"
00124     tikrinti("saltinis: egzaminas == 0", saltinis.getEgzaminas() == 0);
00125     tikrinti("saltinis: vidurkis == 0", saltinis.getVidurkis() == 0.0f);
00126     tikrinti("saltinis: galrezVid == 0", saltinis.getgalrezVid() == 0.0f);
00127 }
00128
00129 static void testuotiPerkelimoPriskyrima()
00130 {
00131     std::cout << "\n[Perkelimo priskyrimo operatorius]\n";
00132 }
00133

```

```

00134     Studentas saltinis = sukurtiTestiniStudenta();
00135
00136     const std::string laukiamasVardas = saltinis.getVardas();
00137     const int laukiamasEgz = saltinis.getEgzaminas();
00138     const float laukiamasGalMed = saltinis.getgalrezMed();
00139
00140     Studentas tikslas;
00141     tikslas = std::move(saltinis);
00142
00143     tikrinti("tikslas: vardas teisingas", tikslas.getVardas() == laukiamasVardas);
00144     tikrinti("tikslas: egzaminas teisingas", tikslas.getEgzaminas() == laukiamasEgz);
00145     tikrinti("tikslas: galrezMed teisingas", tikslas.getgalrezMed() == laukiamasGalMed);
00146
00147     // Šaltinis po perkeltimo turi būti "tuščias"
00148     tikrinti("saltinis: egzaminas == 0", saltinis.getEgzaminas() == 0);
00149     tikrinti("saltinis: galrezVid == 0", saltinis.getgalrezVid() == 0.0f);
00150     tikrinti("saltinis: nd tuscias", saltinis.getND().empty());
00151 }
00152
00153 static void testuotiDestruktoriu()
00154 {
00155     std::cout << "\n[Destruktorius]\n";
00156
00157     {
00158         Studentas s = sukurtiTestiniStudenta();
00159         // s bus sunaikintas pasibaigus blokui
00160     }
00161     tikrinti("destruktorius nesuluzo", true);
00162
00163     // Didelis vektorius - tikrinamas nd atlaisvinimas
00164     {
00165         std::vector<Studentas> daug;
00166         for (int i = 0; i < 100; ++i)
00167         {
00168             daug.push_back(sukurtiTestiniStudenta());
00169         }
00170     }
00171     tikrinti("destruktorius po 100 objektu nesuluzo", true);
00172 }
00173
00174 static void testuotiIstreamOperatoriu()
00175 {
00176     std::cout << "\n[operator] (istream)\n";
00177
00178     // Vienas studentas per srauta
00179     std::istringstream ssl("Ona Oniene 3 4 5 6 7");
00180     Studentas sl;
00181     ssl >> sl;
00182     sl.suskaiciuotiGalutini();
00183
00184     tikrinti("vardas == Ona", sl.getVardas() == "Ona");
00185     tikrinti("pavarde == Oniene", sl.getPavarde() == "Oniene");
00186     tikrinti("egzaminas == 7", sl.getEgzaminas() == 7);
00187     tikrinti("nd.size() == 4", sl.getND().size() == 4);
00188     tikrinti("galrezVid apskaiciuotas", sl.getgalrezVid() > 0.0f);
00189
00190     const std::string duomenys = "Algis Algimantas 6 7 8 9\n" "Birute Birstonienė 4 5 6 7\n";
00191
00192     int skaityta = 0;
00193
00194     std::istringstream srautas(duomenys);
00195     std::string eilute;
00196     while (std::getline(srautas, eilute))
00197     {
00198         if (eilute.empty())
00199         {
00200             continue;
00201         }
00202         std::istringstream esSS(eilute);
00203         Studentas s;
00204         esSS >> s;
00205         if (!s.getND().empty())
00206         {
00207             skaityta++;
00208         }
00209     }
00210
00211     tikrinti("2 studentai nuskaityti is srauto", skaityta == 2);
00212 }
00213
00214 static void testuotiOstreamOperatoriu()
00215 {
00216     std::cout << "\n[operator] (ostream)\n";
00217
00218     Studentas s = sukurtiTestiniStudenta();
00219     // galrezVid/galrezMed jau apskaiciuoti sukurtiTestiniStudenta()
00220

```

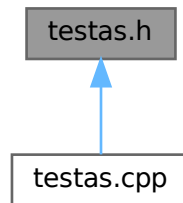
```

00221     std::ostringstream os;
00222     os << s;
00223
00224     const std::string isvestis = os.str();
00225
00226     tikrinti("isvestyje yra vardas", isvestis.find("Jonas") != std::string::npos);
00227     tikrinti("isvestyje yra pavarde", isvestis.find("Jonaitis") != std::string::npos);
00228     tikrinti("isvestyje yra Egz", isvestis.find("Egz:") != std::string::npos);
00229     tikrinti("isvestyje yra ND", isvestis.find("ND:") != std::string::npos);
00230     tikrinti("isvestyje yra Gal.Vid", isvestis.find("Gal.Vid:") != std::string::npos);
00231     tikrinti("isvestyje yra Gal.Med", isvestis.find("Gal.Med:") != std::string::npos);
00232     tikrinti("isvestis netuscia", !isvestis.empty());
00233
00234     // Failų srauto testas - operator« veikia su ofstream
00235     std::ostringstream failoSim;
00236     std::vector<Studentas> sarasas;
00237     sarasas.push_back(s);
00238     sarasas.push_back(sukurtiTestiniStudenta());
00239
00240     for (const auto& st : sarasas)
00241     {
00242         failoSim << st << '\n';
00243     }
00244
00245     tikrinti("isvedimo i faila srautas netuscias", !failoSim.str().empty());
00246 }
00247
00248 void vykdytiTestus()
00249 {
00250     praejo = 0;
00251     nepraejo = 0;
00252
00253     std::cout << "\n" << std::string(55, '=') << '\n' << "Studentas klases testai (v1.2 - Rule of
Five)\n" << std::string(55, '=') << '\n';
00254
00255     testuotiDefaultKonstruktoriu();
00256     testuotiSrautiniKonstruktoriu();
00257     testuotiKopijavimoKonstruktoriu();
00258     testuotiKopijavimoPriskyrima();
00259     testuotiPerkelimoKonstruktoriu();
00260     testuotiPerkelimoPriskyrima();
00261     testuotiDestruktoriu();
00262     testuotiIstreamOperatoriu();
00263     testuotiOstreamOperatoriu();
00264
00265     std::cout << '\n' << std::string(55, '-') << '\n' << "Rezultatas: praejo " << praejo << ", nepraejo " <<
nepraejo << '\n' << std::string(55, '=') << '\n';
00266 }
00267
00268 void testuotiAbstraktuma()
00269 {
00270     std::cout << "\n[Zmogus abstraktumas]\n";
00271
00272     // Zmogus z; <-----ši eilutė nekompijuosis
00273
00274     // Galima tik per rodyklę / nuorodą į išvestinę klasę:
00275     Studentas s = sukurtiTestiniStudenta();
00276     Zmogus* rodykle = &s;
00277
00278     tikrinti("Zmogus* rodo i Studentas objekta", rodykle != nullptr);
00279     tikrinti("getVardas() per baze veikia", rodykle->getVardas() == "Jonas");
00280     tikrinti("getVardas() per baze veikia", rodykle->getPavarde() == "Jonaitis");
00281
00282     // Virtualus dispatch - spausdinti() kviecia Studentas::spausdinti()
00283     std::ostringstream os;
00284     rodykle->spausdinti(os);
00285     tikrinti("spausdinti() per Zmogus* veikia", !os.str().empty());
00286     tikrinti("isvestyje yra vardas", os.str().find("Jonas") != std::string::npos);
00287
00288     // suskaiciuotiGalutini() taip pat polimorfiškai
00289     rodykle->suskaiciuotiGalutini();
00290     tikrinti("suskaiciuotiGalutini() per Zmogus* veikia", s.getgalrezVid() > 0.0f);
00291 }

```

5.31 testas.h File Reference

This graph shows which files directly or indirectly include this file:



Functions

- void [vykdytiTestus\(\)](#)

5.31.1 Function Documentation

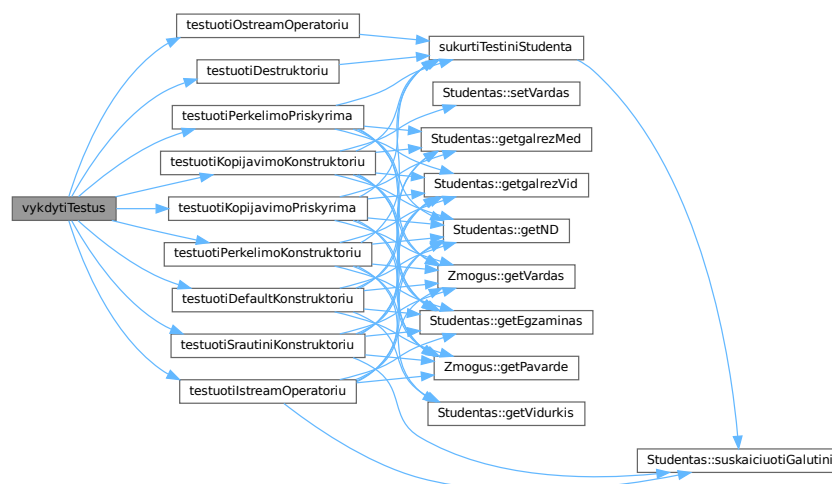
5.31.1.1 vykdytiTestus()

```
void vykdytiTestus ( )
```

Definition at line 248 of file [testas.cpp](#).

References [testuotiDefaultKonstruktoriu\(\)](#), [testuotiDestruktoriu\(\)](#), [testuotiIstreamOperatoriu\(\)](#), [testuotiKopijavimoKonstruktoriu\(\)](#), [testuotiKopijavimoPriskyrima\(\)](#), [testuotiOstreamOperatoriu\(\)](#), [testuotiPerkelimoKonstruktoriu\(\)](#), [testuotiPerkelimoPriskyrima\(\)](#), and [testuotiSrautiniKonstruktoriu\(\)](#).

Here is the call graph for this function:



5.32 testas.h

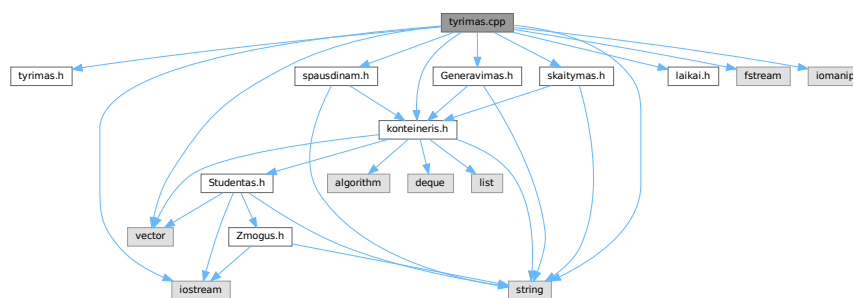
[Go to the documentation of this file.](#)

```
00001 #pragma once
00002
00003 void vykdytiTestus();
```

5.33 tyrimas.cpp File Reference

```
#include "tyrimas.h"
#include "Generavimas.h"
#include "konteineris.h"
#include "laikai.h"
#include "skaitymas.h"
#include "spausdinam.h"
#include <fstream>
#include <iomanip>
#include <iostream>
#include <string>
#include <vector>
```

Include dependency graph for tyrimas.cpp:



Classes

- struct [Tyrimolrasas](#)

Functions

- void [vykdytiTyrima](#) ()
Paleidžia interaktyvų konteinerių greitaveikos tyrimą.

5.33.1 Function Documentation

5.33.1.1 vykdytiTyrima()

```
void vykdytiTyrima ( )
```

Paleidžia interaktyvų konteinerių greitaveikos tyrimą.

Vartotojas gali pasirinkti:

- skirstymo strategiją;
- kartojimų skaičių (rezultatai vidurkinami);
- testuojamą failą arba standartinį failų rinkinį (1k–10M eilučių).

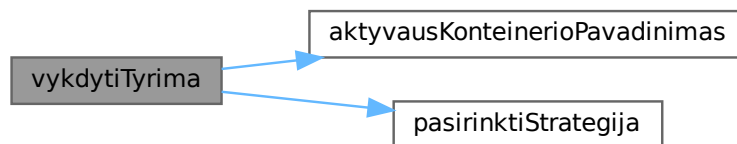
Rezultatai išvedami į konsolę ir išsaugomi kaip Markdown lentelė faile `benchmark_<konteineris>_<S>strategija.md`.

Definition at line 134 of file `tyrimas.cpp`.

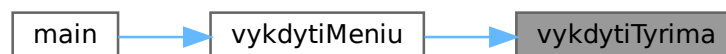
References `aktyvausKonteinerioPavadinimas()`, and `pasirinktiStrategija()`.

Referenced by `vykdytiMenu()`.

Here is the call graph for this function:



Here is the caller graph for this function:



5.34 tyrimas.cpp

[Go to the documentation of this file.](#)

```

00001 #include "tyrimas.h"
00002
00003 #include "Generavimas.h"
00004 #include "konteineris.h"
00005 #include "laikai.h"
00006 #include "skaitymas.h"
00007 #include "spausdinam.h"
00008
00009 #include <fstream>
00010 #include <iomanip>
00011 #include <iostream>
00012 #include <string>
00013 #include <vector>
00014
00015
00016 struct TyrimoIrasas

```

```

00017 {
00018     std::string failas;
00019     std::size_t irasuKiekis = 0;
00020     double skaitymas = 0.0;
00021     double rusiavimas = 0.0;
00022     double skirstymas = 0.0;
00023 };
00024
00025 namespace
00026 {
00027
00028     bool failasEgzistuoja(const std::string& pav)
00029     {
00030         std::ifstream f(pav);
00031         return f.good();
00032     }
00033
00034     TyrimoIrasas atliktiBandyMuSerija(
00035         const std::string& failas,
00036         SkirstymoStrategija strategija,
00037         int kartojimai)
00038     {
00039         TyrimoIrasas vidurkiai;
00040         vidurkiai.failas = failas;
00041
00042         for (int i = 0; i < kartojimai; ++i)
00043         {
00044             nunulintiLaikus();
00045
00046             StudContainer studis;
00047             if (!failoSkaitymas(studis, failas)) // naudoja Studentas() vid.
00048             {
00049                 return {};
00050             }
00051
00052             suskaiciuotiGalutinius(studis);
00053             vidurkiai.irasuKiekis = studis.size();
00054
00055             surikiuotiPagalGalutiniVid(studis);
00056             skirstymasGrupes(studis, strategija, false, false);
00057
00058             vidurkiai.skaitymas += timers.skaitymas;
00059             vidurkiai.rusiavimas += timers.rusiavimas;
00060             vidurkiai.skirstymas += timers.skirstymas;
00061         }
00062
00063         vidurkiai.skaitymas /= kartojimai;
00064         vidurkiai.rusiavimas /= kartojimai;
00065         vidurkiai.skirstymas /= kartojimai;
00066
00067         return vidurkiai;
00068     }
00069
00070     void spausdintiLentele(const std::vector<TyrimoIrasas>& rezultatasultatai)
00071     {
00072         std::cout << '\n'
00073             << std::left
00074             << std::setw(22) << "Failas"
00075             << std::setw(16) << "Irasu kiekis"
00076             << std::setw(16) << "Skaitymas (s)"
00077             << std::setw(18) << "Rusiavimas (s)"
00078             << std::setw(18) << "Skirstymas (s)"
00079             << "Is viso (s)\n"
00080             << std::string(90, '-') << '\n';
00081
00082         for (const auto& r : rezultatasultatai)
00083         {
00084             const double isViso = r.skaitymas + r.rusiavimas + r.skirstymas;
00085
00086             std::cout << std::left
00087                 << std::setw(22) << r.failas
00088                 << std::setw(16) << r.irasuKiekis
00089                 << std::setw(16) << std::fixed << std::setprecision(6) << r.skaitymas
00090                 << std::setw(18) << std::fixed << std::setprecision(6) << r.rusiavimas
00091                 << std::setw(18) << std::fixed << std::setprecision(6) << r.skirstymas
00092                 << std::fixed << std::setprecision(6) << isViso << '\n';
00093         }
00094     }
00095
00096     void iREADME(const std::vector<TyrimoIrasas>& rezultatasultatai, SkirstymoStrategija strategija)
00097     {
00098         const std::string pav = "benchmark_" + aktyvausKonteinerioTrumpasPavadinimas() + "_S" +
00099             std::to_string(static_cast<int>(strategija)) + ".md";
00100
00101         std::ofstream write(pav);
00102         if (!write.is_open()) { return; }
00103     }

```

```

00103     write << "# " << aktyvausKonteinerioPavadinimas() << " - strategija " << static_cast<int>(strategija)
00104     << "\n\n";
00104     write << "| Failas | Irasu kiekis | Skaitymas (s) | Rusiavimas (s) | Skirstymas (s) | Is viso (s)
00105     |\n";
00105     write << "|---|---:|---:|---:|---:|---:|\n";
00106
00107     for (const auto& r : rezultatasultatai)
00108     {
00109         const double isViso = r.skaitymas + r.rusiavimas + r.skirstymas;
00110         write << "| " << r.failas
00111             << " | " << r.irasuKiekis
00112             << " | " << std::fixed << std::setprecision(6) << r.skaitymas
00113             << " | " << std::fixed << std::setprecision(6) << r.rusiavimas
00114             << " | " << std::fixed << std::setprecision(6) << r.skirstymas
00115             << " | " << std::fixed << std::setprecision(6) << isViso << " |\n";
00116     }
00117
00118     std::cout << "Rezultatu lentele issaugota: " << pav << '\n';
00119 }
00120
00121 std::vector<std::string> sudarytiNumatytyjuFailuSarasa(const std::string& prefiksas)
00122 {
00123     return {
00124         prefiksas + "1000.txt",
00125         prefiksas + "10000.txt",
00126         prefiksas + "100000.txt",
00127         prefiksas + "1000000.txt",
00128         prefiksas + "10000000.txt"
00129     };
00130 }
00131
00132 }
00133
00134 void vykdytiTyrima()
00135 {
00136     std::cout << "\nAktyvus konteineris: " << aktyvausKonteinerioPavadinimas() << '\n';
00137
00138     const SkirstymoStrategija strategija = pasirinktiStrategija();
00139
00140     std::cout << "Kiek kartu kartoti kiekviena testa? ";
00141     int kartojimai = 1;
00142     std::cin >> kartojimai;
00143
00144     if (!std::cin || kartojimai < 1)
00145     {
00146         throw std::runtime_error("Neteisingas kartojimui skaicius.");
00147     }
00148
00149     std::cout << "\n1 - Testuoti viena faila\n"
00150             << "2 - Testuoti numatytytus failus (1k, 10k, 100k, 1M, 10M)\n";
00151
00152     int pasirinkimas = 0;
00153     std::cin >> pasirinkimas;
00154
00155     if (!std::cin || pasirinkimas < 1 || pasirinkimas > 2)
00156     {
00157         throw std::runtime_error("Neteisingas tyrimo meniu pasirinkimas.");
00158     }
00159
00160     std::vector<std::string> failai;
00161
00162     if (pasirinkimas == 1)
00163     {
00164         std::cout << "Iveskite pilna failo pavadinima: ";
00165         std::string f;
00166         std::cin >> f;
00167         failai.push_back(f);
00168     }
00169     else
00170     {
00171         std::cout << "Iveskite failu prefiksa be dydzio ir .txt (pvz. studentai): ";
00172         std::string prefiksas;
00173         std::cin >> prefiksas;
00174         failai = sudarytiNumatytyjuFailuSarasa(prefiksas);
00175     }
00176
00177     std::vector<TyrimoIrasas> rezultatasultatai;
00178
00179     for (const auto& failas : failai)
00180     {
00181         if (!failasEgzistuoja(failas))
00182         {
00183             std::cout << "Failas nerastas, praleidziamas: " << failas << '\n';
00184             continue;
00185         }
00186
00187         auto rezultatas = atliktiBandymuSerija(failas, strategija, kartojimai);

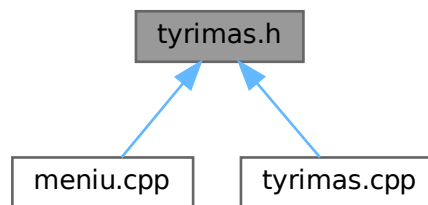
```

```
00188         if (rezultatas.irasuKiekis != 0)
00189         {
00190             rezultatasultatai.push_back(rezultatas);
00191         }
00192     }
00193
00194     if (rezultatasultatai.empty())
00195     {
00196         std::cout << "Tyrimui tinkamu failu nerasta.\n";
00197         return;
00198     }
00199
00200     spausdintiLentele(rezultatasultatai);
00201     iREADME(rezultatasultatai, strategija);
00202 }
```

5.35 tyrimas.h File Reference

Konteinerių ir skirstymo strategijų greitaveikos tyrimo funkcijos.

This graph shows which files directly or indirectly include this file:



Functions

- void [vykdytiTyrima](#) ()
Paleidžia interaktyvų konteinerių greitaveikos tyrimą.

5.35.1 Detailed Description

Konteinerių ir skirstymo strategijų greitaveikos tyrimo funkcijos.

Atlieka laiko matavimus skaitant failus, rikiuojant ir skirstant studentus bei išveda rezultatus į konsolę ir Markdown lentelę.

Author

[Studentas](#)

Version

2.0

Definition in file [tyrimas.h](#).

5.35.2 Function Documentation

5.35.2.1 vykdytiTyrima()

```
void vykdytiTyrima ( )
```

Paleidžia interaktyvų konteinerių greیتaveikos tyrimą.

Vartotojas gali pasirinkti:

- skirstymo strategiją;
- kartojimų skaičių (rezultatai vidurkinami);
- testuojamą failą arba standartinį failų rinkinį (1k–10M eilučių).

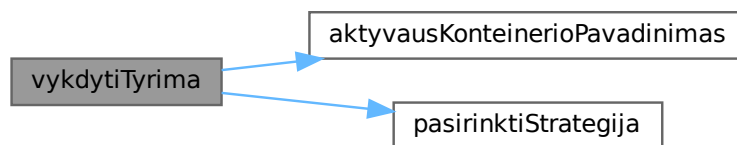
Rezultatai išvedami į konsolę ir išsaugomi kaip Markdown lentelė faile `benchmark_<konteineris>_<S<strategija>.md`.

Definition at line 134 of file [tyrimas.cpp](#).

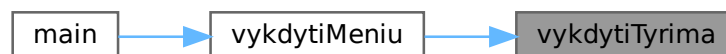
References [aktyvausKonteinerioPavadinimas\(\)](#), and [pasirinktiStrategija\(\)](#).

Referenced by [vykdytiMeniu\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.36 tyrimas.h

[Go to the documentation of this file.](#)

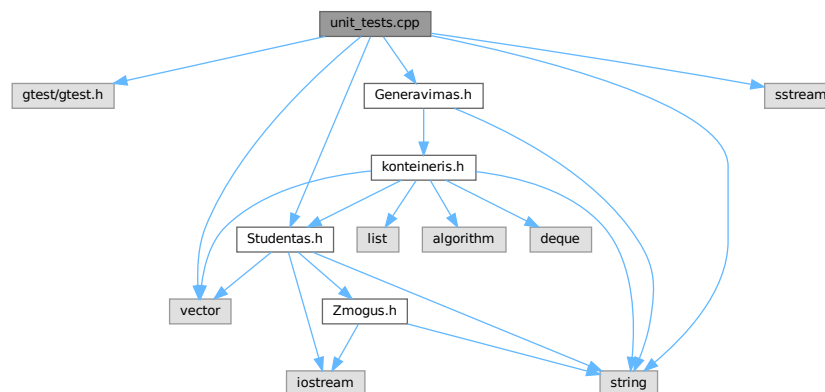
```
00001
00012 #pragma once
00013
00026 void vykdytiTyrima();
```

5.37 unit_tests.cpp File Reference

Google Test unit testai [Studentas](#) klasei.

```
#include <gtest/gtest.h>
#include "Studentas.h"
#include "Generavimas.h"
#include <sstream>
#include <string>
#include <vector>
```

Include dependency graph for unit_tests.cpp:



Functions

- static [Studentas sukurtiTestiniStudenta](#) ()
Sukuria testinį studentą iš srauto eilutės.
- [TEST](#) (RuleOfFive, DefaultKonstruktorius_LaukaiTusci)
Tikrina, kad numatytasis konstruktorius inicializuoja tuščius laukus.
- [TEST](#) (RuleOfFive, KopijavimoKonstruktorius_GiliKopija)
Tikrina, kad kopijavimo konstruktorius sukuria gilią kopiją.
- [TEST](#) (RuleOfFive, KopijavimoKonstruktorius_OriginalasNelsikis)
Tikrina, kad keičiant kopiją originalas nesikeičia (gili kopija).
- [TEST](#) (RuleOfFive, KopijavimoPriskyrimas_DuomenysSutampa)
Tikrina kopijavimo priskyrimo operatorių.
- [TEST](#) (RuleOfFive, KopijavimoPriskyrimas_SavipriskyrimasSaugus)
Tikrina, kad savipriskyrimas yra saugus (nekelia crash).
- [TEST](#) (RuleOfFive, PerkélimoKonstruktorius_TikslasGaunaKorektiskusDuomenis)
Tikrina, kad perkélimo konstruktorius teisingai perkelia duomenis.
- [TEST](#) (RuleOfFive, PerkélimoKonstruktorius_SaltinisLiekaErsatzBusenos)
Tikrina, kad šaltinis po perkélimo lieka tuščios būsenos.
- [TEST](#) (RuleOfFive, PerkélimoPriskyrimas_TikslasGaunaKorektiskusDuomenis)
Tikrina perkélimo priskyrimo operatorių.
- [TEST](#) (RuleOfFive, PerkélimoPriskyrimas_SaltinisLiekaErsatzBusenos)
Tikrina, kad šaltinis po perkélimo priskyrimo lieka tuščias.
- [TEST](#) (RuleOfFive, Destruktorius_NesukeliaKlaidosDuomenysSunaikinami)
Tikrina, kad destruktoriaus nesukelia klaidų.

- [TEST](#) (RuleOfFive, Destruktorius_MasinisSunaikinimas)
Tikrina destruktorių su 1000 objektų vektoriuje.
- [TEST](#) (SrautinisKonstruktorius, NuskaitoTeisingaisEilutes)
Tikrina srautinį konstruktorių su standartine eilute.
- [TEST](#) (IstreamOperatorius, NuskaitoVienaSudenta)
Tikrina operator>> su vienu studentu.
- [TEST](#) (IstreamOperatorius, NuskaitoKelisStudentus)
Tikrina operator>> su keliais studentais (eilutė po eilutės).
- [TEST](#) (OstreamOperatorius, IsvedasTuriBaziniusLaukus)
Tikrina, kad operator<< išveda reikiamus laukus.
- [TEST](#) (SuskaiciuotiGalutini, VidurkioFormuleTeisinga)
Tikrina galutinio balo skaičiavimo formulę (vidurkio variantas).
- [TEST](#) (SuskaiciuotiGalutini, GalutinisNedaugiauNei10)
Tikrina, kad galutinis balas nekyla virš 10.
- [TEST](#) (SuskaiciuotiGalutini, GalutinisNeMazesnisNei0)
Tikrina, kad galutinis balas ne mažesnis nei 0.
- [TEST](#) (Abstraktumas, VirtualusDispatchPerZmogusPtri)
Tikrina virtualų dispatch per bazinės klasės rodyklę.
- [TEST](#) (Abstraktumas, SuskaiciuotiGalutiniPolimorfiskai)
Tikrina, kad suskaiciuotiGalutini() veikia polimorfiškai.
- [TEST](#) (Skirstymas, Strategija1_TeisingaiSkirstoVargsiukusIrProtus)
Tikrina 1-ą skirstymo strategiją (du nauji konteineriai).
- [TEST](#) (Skirstymas, Strategija2_IstrinaMasVargsiukusIsOriginalaus)
Tikrina 2-ą strategiją (erase iš originalaus konteinerio).
- [TEST](#) (Skirstymas, Strategija3_PartitionTeisingaiSkirstoGrupes)
Tikrina 3-ą strategiją (std::partition).
- [TEST](#) (SettersGetters, NustatimoFunkcijosVeikia)
Tikrina, kad setVardas/setPavarde/setEgzaminas veikia teisingai.
- int [main](#) (int argc, char **argv)

5.37.1 Detailed Description

Google Test unit testai [Studentas](#) klasei.

Tikrinami visi Rule of Five metodai (privaloma), įvesties/išvesties operatoriai, galutinių balų skaičiavimas ir skirstymo funkcijos.

Paleidimas:

```
mkdir build && cd build
cmake .. -DBUILD_TESTS=ON
cmake --build .
ctest --verbose
```

Author

[Studentas](#)

Version

2.0

Definition in file [unit_tests.cpp](#).

5.37.2 Function Documentation

5.37.2.1 main()

```
int main (
    int argc,
    char ** argv )
```

Definition at line 512 of file [unit_tests.cpp](#).

5.37.2.2 sukurtiTestiniStudenta()

```
static Studentas sukurtiTestiniStudenta ( ) [static]
```

Sukuria testinį studentą iš srauto eilutės.

Eilutė: "Jonas Jonaitis 8 7 9 6 10" (paskutinis skaičius – egzaminas, likę – namų darbai)

Definition at line 35 of file [unit_tests.cpp](#).

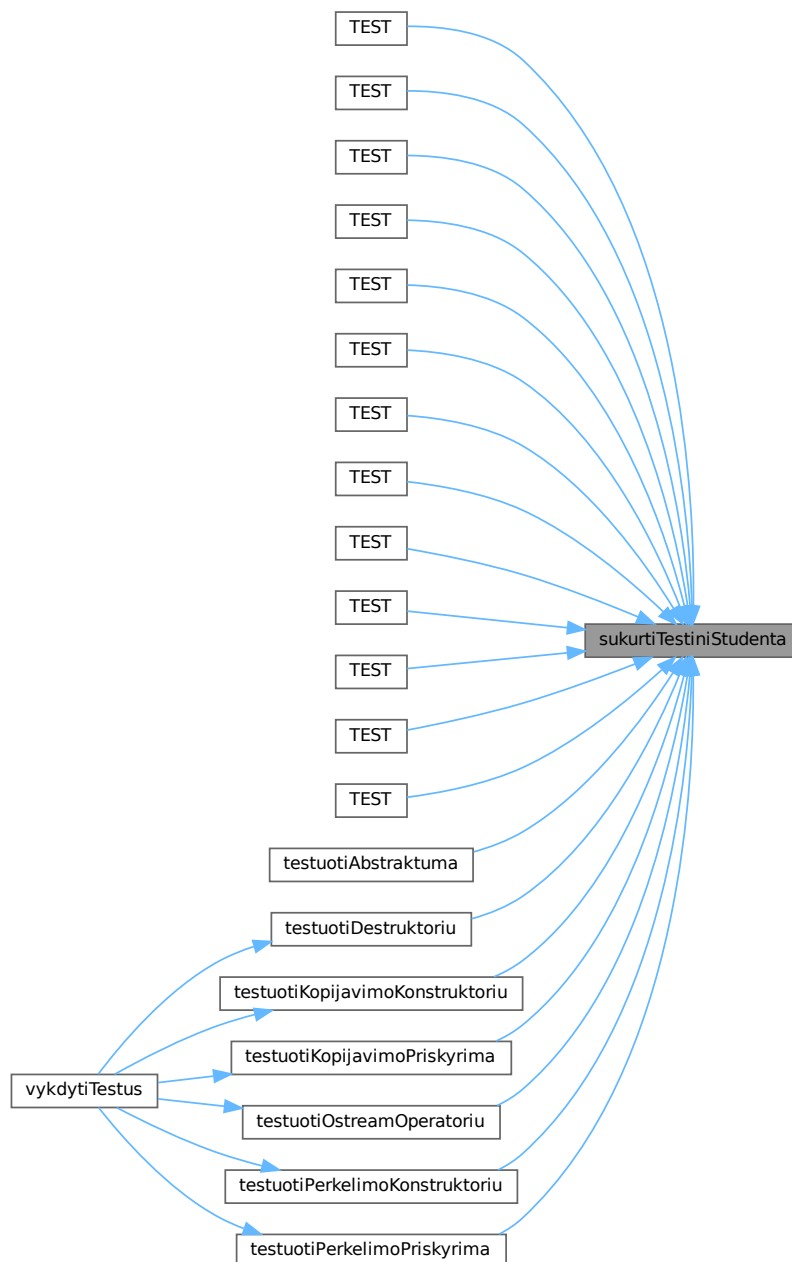
References [Studentas::suskaiciuotiGalutini\(\)](#).

Referenced by [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [TEST\(\)](#), [testuotiAbstraktuma\(\)](#), [testuotiDestruktoriu\(\)](#), [testuotiKopijavimoKonstruktoriu\(\)](#), [testuotiKopijavimoPriskyrima\(\)](#), [testuotiOstreamOperatoriu\(\)](#), [testuotiPerkelimoKonstruktoriu\(\)](#), and [testuotiPerkelimoPriskyrima\(\)](#).

Here is the call graph for this function:



Here is the caller graph for this function:



5.37.2.3 TEST() [1/24]

```

TEST (
    Abstraktumas ,
    SuskaiciuotiGalutiniPolimorfiskai )

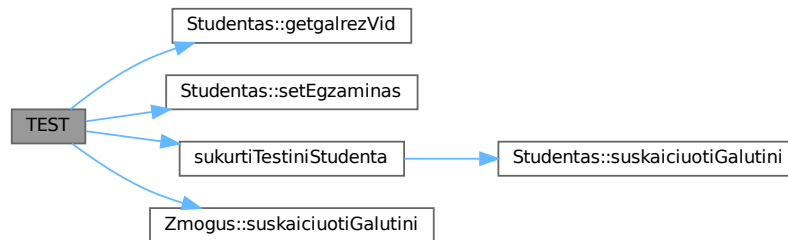
```

Tikrina, kad suskaiciuotiGalutini() veikia polimorfiškai.

Definition at line 397 of file [unit_tests.cpp](#).

References [Studentas::getgalrezVid\(\)](#), [Studentas::setEgzaminas\(\)](#), [sukurtiTestiniStudenta\(\)](#), and [Zmogus::suskaiciuotiGalutini\(\)](#).

Here is the call graph for this function:



5.37.2.4 TEST() [2/24]

```

TEST (
    Abstraktumas ,
    VirtualusDispatchPerZmogusPtri )

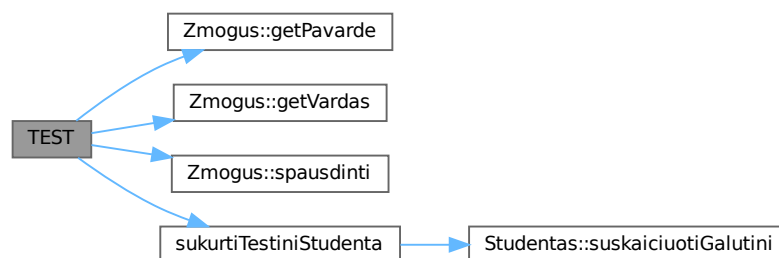
```

Tikrina virtualų dispatch per bazinės klasės rodyklę.

Definition at line 379 of file [unit_tests.cpp](#).

References [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), [Zmogus::spausdinti\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:



5.37.2.5 TEST() [3/24]

```
TEST (
    IstreamOperatorius ,
    NuskaitoKelisStudentus )
```

Tikrina operator>> su keliais studentais (eilutė po eilutės).

Definition at line 280 of file [unit_tests.cpp](#).

References [Studentas::getND\(\)](#).

Here is the call graph for this function:



5.37.2.6 TEST() [4/24]

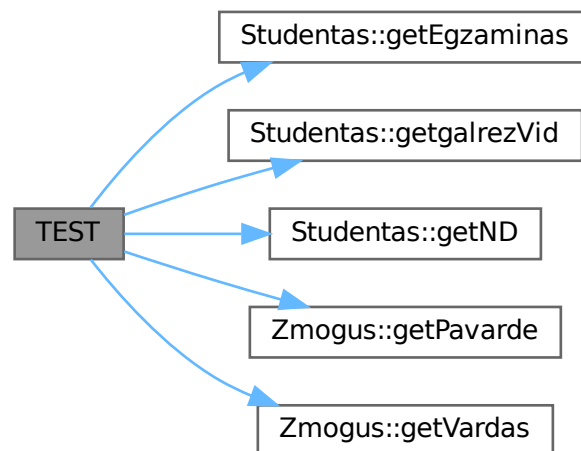
```
TEST (
    IstreamOperatorius ,
    NuskaitoVienaSudenta )
```

Tikrina operator>> su vienu studentu.

Definition at line 264 of file [unit_tests.cpp](#).

References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getPavarde\(\)](#), and [Zmogus::getVardas\(\)](#).

Here is the call graph for this function:



5.37.2.7 TEST() [5/24]

```
TEST (
    ostreamOperatorius ,
    IsvedasTuriBaziniusLaukus )
```

Tikrina, kad operator<< išveda reikiamus laukus.

Definition at line 309 of file [unit_tests.cpp](#).

References [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:



5.37.2.8 TEST() [6/24]

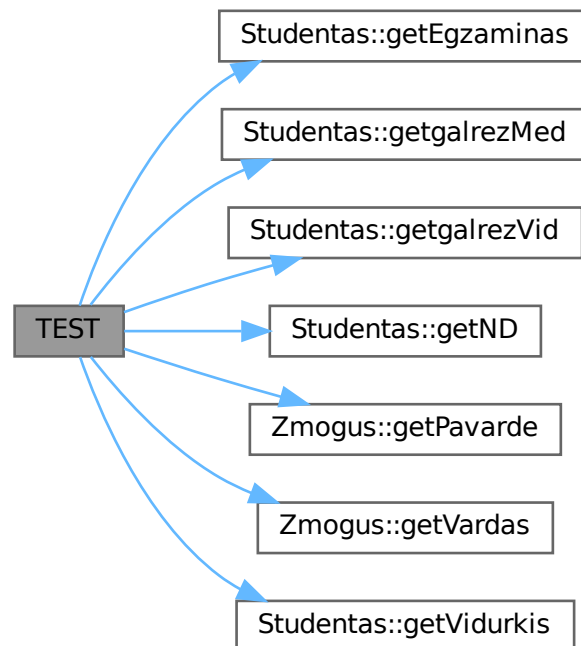
```
TEST (
    RuleOfFive ,
    DefaultKonstruktorius_LaukaiTusci )
```

Tikrina, kad numatytasis konstruktorius inicializuoja tuščius laukus.

Definition at line 50 of file [unit_tests.cpp](#).

References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezMed\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), and [Studentas::getVidurkis\(\)](#).

Here is the call graph for this function:



5.37.2.9 TEST() [7/24]

```
TEST (
    RuleOfFive ,
    Destruktorius_MasinisSunaikinimas )
```

Tikrina destruktorių su 1000 objektų vektoriuje.

Definition at line 223 of file [unit_tests.cpp](#).

References [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:



5.37.2.10 TEST() [8/24]

```
TEST (
    RuleOfFive ,
    Destruktorius_NesukeliaKlaidosDuomenysSunaikinami )
```

Tikrina, kad destruktoriaus nesukelia klaidų.

Definition at line 208 of file [unit_tests.cpp](#).

References [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:

**5.37.2.11 TEST()** [9/24]

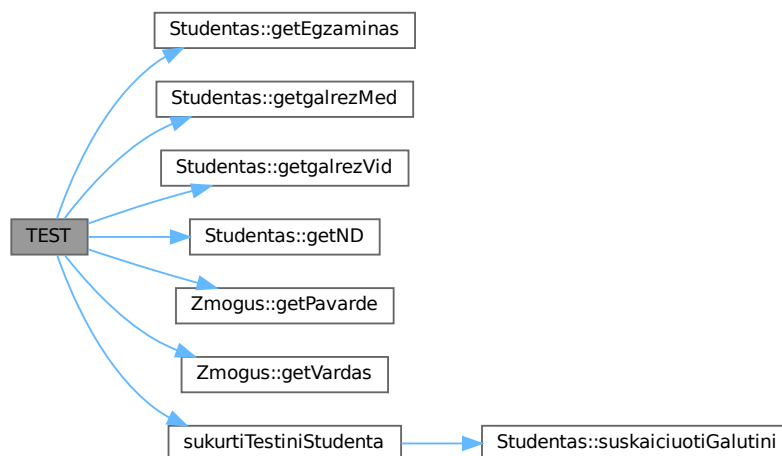
```
TEST (
    RuleOfFive ,
    KopijavimoKonstruktorius_GiliKopija )
```

Tikrina, kad kopijavimo konstruktorius sukuria gilią kopiją.

Definition at line 70 of file [unit_tests.cpp](#).

References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezMed\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:



5.37.2.12 TEST() [10/24]

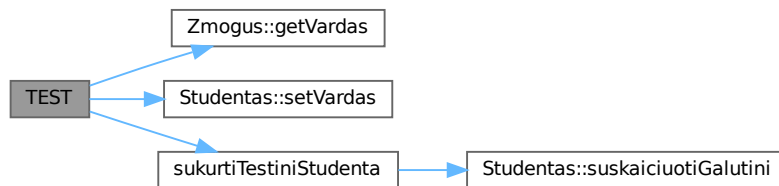
```
TEST (
    RuleOfFive ,
    KopijavimoKonstruktorius_OriginalasNeIsikis )
```

Tikrina, kad keičiant kopiją originalas nesikeičia (gili kopija).

Definition at line 86 of file [unit_tests.cpp](#).

References [Zmogus::getVardas\(\)](#), [Studentas::setVardas\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:

**5.37.2.13 TEST()** [11/24]

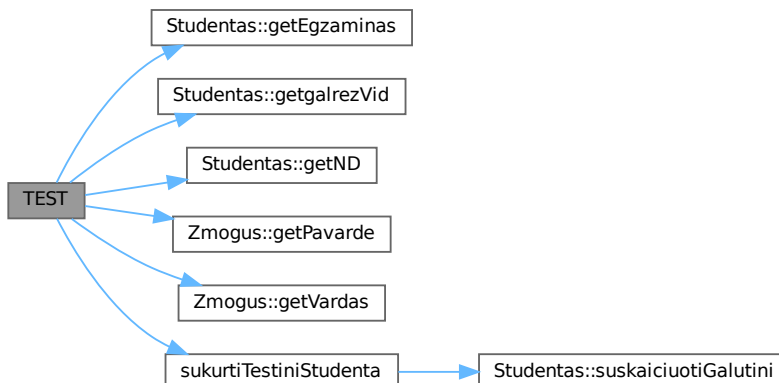
```
TEST (
    RuleOfFive ,
    KopijavimoPriskyrimas_DuomenysSutampa )
```

Tikrina kopijavimo priskyrimo operatorių.

Definition at line 104 of file [unit_tests.cpp](#).

References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:



5.37.2.14 TEST() [12/24]

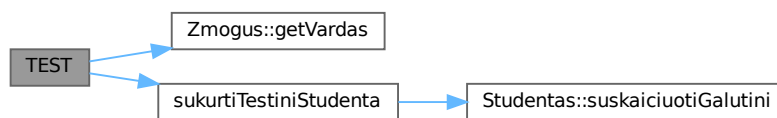
```
TEST (
    RuleOfFive ,
    KopiJavimoPriskyrimas_SavipriskyrimasSaugus )
```

Tikrina, kad savipriskyrimas yra saugus (nekelia crash).

Definition at line 120 of file [unit_tests.cpp](#).

References [Zmogus::getVardas\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:



5.37.2.15 TEST() [13/24]

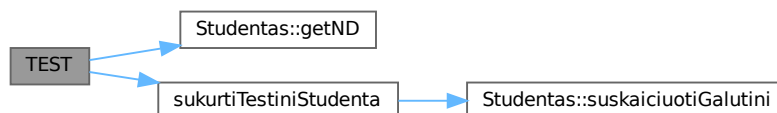
```
TEST (
    RuleOfFive ,
    PerkeliMoKonstruktorius_SaltinisLiekaErsatzBusenos )
```

Tikrina, kad šaltinis po perkėlimo lieka tuščios būsenos.

Definition at line 157 of file [unit_tests.cpp](#).

References [Studentas::getND\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:



5.37.2.16 TEST() [14/24]

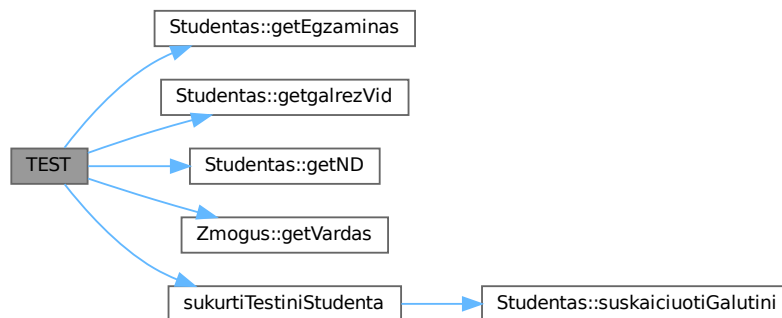
```
TEST (
    RuleOfFive ,
    PerkelimoKonstruktorius_TikslasGaunaKorektiskusDuomenis )
```

Tikrina, kad perkėlimo konstruktorius teisingai perkelia duomenis.

Definition at line 137 of file [unit_tests.cpp](#).

References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getVardas\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:

**5.37.2.17 TEST()** [15/24]

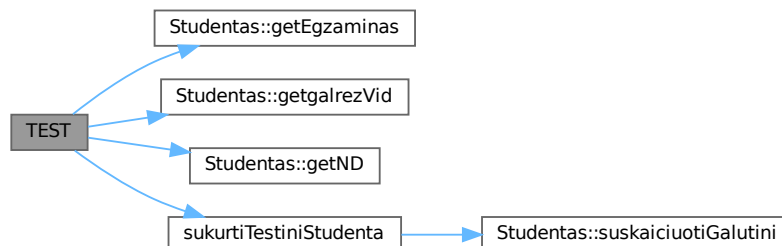
```
TEST (
    RuleOfFive ,
    PerkelimoPriskyrimas_SaltinisLiekaErsatzBusenos )
```

Tikrina, kad šaltinis po perkėlimo priskyrimo lieka tuščias.

Definition at line 190 of file [unit_tests.cpp](#).

References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:



5.37.2.18 TEST() [16/24]

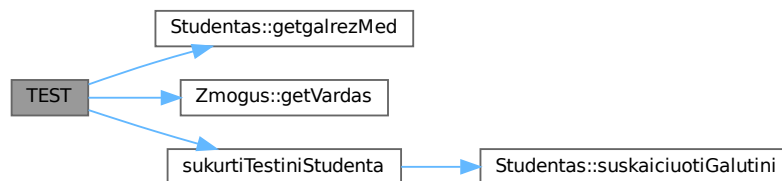
```
TEST (
    RuleOfFive ,
    PerkelimoPriskyrimas_TikslasGaunaKorektiskusDuomenis )
```

Tikrina perkėlimo priskyrimo operatorių.

Definition at line 173 of file [unit_tests.cpp](#).

References [Studentas::getgalrezMed\(\)](#), [Zmogus::getVardas\(\)](#), and [sukurtiTestiniStudenta\(\)](#).

Here is the call graph for this function:



5.37.2.19 TEST() [17/24]

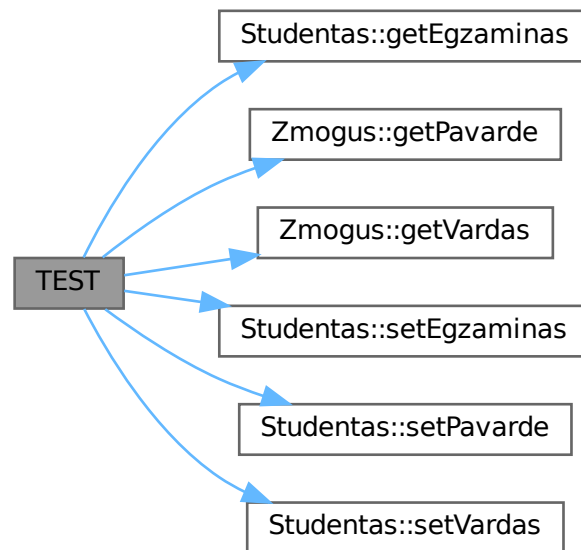
```
TEST (
    SettersGetters ,
    NustatimoFunkcijosVeikia )
```

Tikrina, kad setVardas/setPavarde/setEgzaminas veikia teisingai.

Definition at line 496 of file [unit_tests.cpp](#).

References [Studentas::getEgzaminas\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), [Studentas::setEgzaminas\(\)](#), [Studentas::setPavarde\(\)](#), and [Studentas::setVardas\(\)](#).

Here is the call graph for this function:



5.37.2.20 TEST() [18/24]

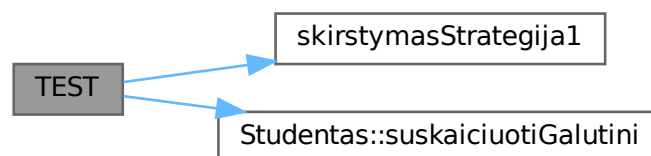
```
TEST (
    Skirstymas ,
    Strategija1_TeisingaiSkirstoVargsiukusIrProtus )
```

Tikrina 1-ą skirstymo strategiją (du nauji konteineriai).

Definition at line 416 of file [unit_tests.cpp](#).

References [skirstymasStrategija1\(\)](#), and [Studentas::suskaiciuotiGalutini\(\)](#).

Here is the call graph for this function:



5.37.2.21 TEST() [19/24]

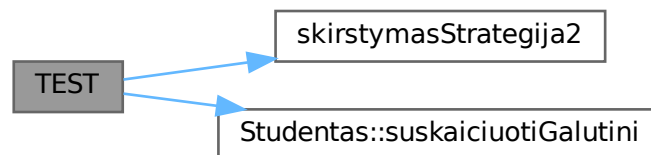
```
TEST (
    Skirstymas ,
    Strategija2_IstrinaMasVargsiukusIsOriginalaus )
```

Tikrina 2-ą strategiją (erase iš originalaus konteinerio).

Definition at line [443](#) of file [unit_tests.cpp](#).

References [skirstymasStrategija2\(\)](#), and [Studentas::suskaiciuotiGalutini\(\)](#).

Here is the call graph for this function:

**5.37.2.22 TEST()** [20/24]

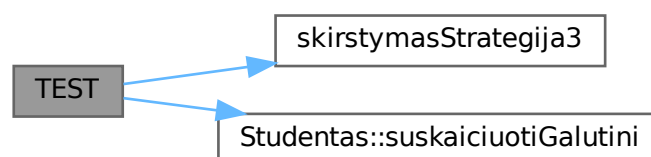
```
TEST (
    Skirstymas ,
    Strategija3_PartitionTeisingaiSkirstoGrupes )
```

Tikrina 3-ą strategiją (std::partition).

Definition at line [469](#) of file [unit_tests.cpp](#).

References [skirstymasStrategija3\(\)](#), and [Studentas::suskaiciuotiGalutini\(\)](#).

Here is the call graph for this function:



5.37.2.23 TEST() [21/24]

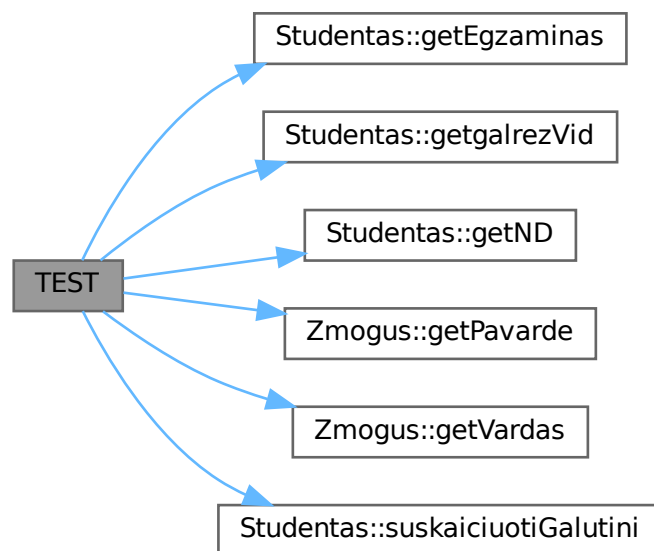
```
TEST (
    SrautinisKonstruktorius ,
    NuskaitoTeisingaiIsEilutes )
```

Tikrina srautinį konstruktorių su standartine eilute.

Definition at line 243 of file [unit_tests.cpp](#).

References [Studentas::getEgzaminas\(\)](#), [Studentas::getgalrezVid\(\)](#), [Studentas::getND\(\)](#), [Zmogus::getPavarde\(\)](#), [Zmogus::getVardas\(\)](#), and [Studentas::suskaiciuotiGalutini\(\)](#).

Here is the call graph for this function:

**5.37.2.24 TEST()** [22/24]

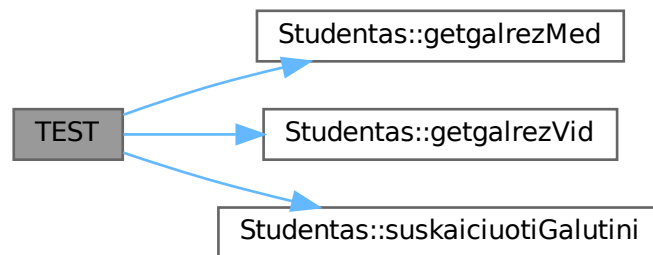
```
TEST (
    SuskaiciuotiGalutini ,
    GalutinisNedaugiauNei10 )
```

Tikrina, kad galutinis balas nekyla virš 10.

Definition at line 348 of file [unit_tests.cpp](#).

References [Studentas::getgalrezMed\(\)](#), [Studentas::getgalrezVid\(\)](#), and [Studentas::suskaiciuotiGalutini\(\)](#).

Here is the call graph for this function:



5.37.2.25 TEST() [23/24]

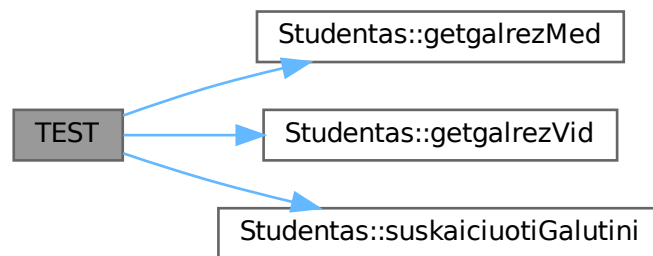
```
TEST (
    SuskaiciuotiGalutini ,
    GalutinisNeMazesnisNei0 )
```

Tikrina, kad galutinis balas ne mažesnis nei 0.

Definition at line 362 of file [unit_tests.cpp](#).

References [Studentas::getgalrezMed\(\)](#), [Studentas::getgalrezVid\(\)](#), and [Studentas::suskaiciuotiGalutini\(\)](#).

Here is the call graph for this function:



5.37.2.26 TEST() [24/24]

```
TEST (
    SuskaiciuotiGalutini ,
    VidurkioFormuleTeisinga )
```

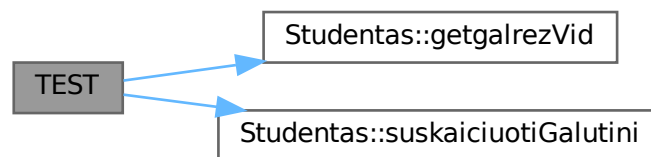
Tikrina galutinio balo skaičiavimo formulę (vidurkio variantas).

Formulė: $\text{galrezVid} = 0.4 * \text{vidurkis} + 0.6 * \text{egzaminas}$

Definition at line 335 of file [unit_tests.cpp](#).

References [Studentas::getgalrezVid\(\)](#), and [Studentas::suskaiciuotiGalutini\(\)](#).

Here is the call graph for this function:



5.38 unit_tests.cpp

[Go to the documentation of this file.](#)

```
00001
00020 #include <gtest/gtest.h>
00021
00022 #include "Studentas.h"
00023 #include "Generavimas.h"
00024
00025 #include <sstream>
00026 #include <string>
00027 #include <vector>
00028
00035 static Studentas sukurtiTestiniStudenta()
00036 {
00037     std::stringstream ss("Jonas Jonaitis 8 7 9 6 10");
00038     Studentas s(ss);
00039     s.suskaiciuotiGalutini();
00040     return s;
00041 }
00042
00043 // =====
00044 // 1. Numatytasis konstruktorius
00045 // =====
00046
00050 TEST(RuleOfFive, DefaultKonstruktorius_LaukaiTusci)
00051 {
00052     Studentas s;
00053
00054     EXPECT_TRUE(s.getVardas().empty()) << "Vardas turetu buti tuscias";
00055     EXPECT_TRUE(s.getPavarde().empty()) << "Pavarde turetu buti tuscia";
00056     EXPECT_EQ(s.getEgzaminas(), 0) << "Egzaminas turetu buti 0";
00057     EXPECT_TRUE(s.getND().empty()) << "ND vektorius turetu buti tuscias";
00058     EXPECT_FLOAT_EQ(s.getVidurkis(), 0.0f);
00059     EXPECT_FLOAT_EQ(s.getgalrezVid(), 0.0f);
00060     EXPECT_FLOAT_EQ(s.getgalrezMed(), 0.0f);
00061 }
00062
```

```

00063 // =====
00064 // 2. Kopijavimo konstruktorius
00065 // =====
00066
00070 TEST(RuleOfFive, KopijavimoKonstruktorius_GiliKopija)
00071 {
00072     Studentas original = sukurtiTestiniStudenta();
00073     Studentas kopija(original);
00074
00075     EXPECT_EQ(kopija.getVardas(), original.getVardas());
00076     EXPECT_EQ(kopija.getPavarde(), original.getPavarde());
00077     EXPECT_EQ(kopija.getEgzaminas(), original.getEgzaminas());
00078     EXPECT_EQ(kopija.getND(), original.getND());
00079     EXPECT_FLOAT_EQ(kopija.getgalrezVid(), original.getgalrezVid());
00080     EXPECT_FLOAT_EQ(kopija.getgalrezMed(), original.getgalrezMed());
00081 }
00082
00086 TEST(RuleOfFive, KopijavimoKonstruktorius_OriginalasNeIsikis)
00087 {
00088     Studentas original = sukurtiTestiniStudenta();
00089     Studentas kopija(original);
00090
00091     kopija.setVardas("Kitas");
00092
00093     EXPECT_EQ(original.getVardas(), "Jonas")
00094         << "Originalas neturetu keistis keiciant kopija";
00095 }
00096
00097 // =====
00098 // 3. Kopijavimo priskyrimo operatorius
00099 // =====
00100
00104 TEST(RuleOfFive, KopijavimoPriskyrimas_DuomenysSutampa)
00105 {
00106     Studentas original = sukurtiTestiniStudenta();
00107     Studentas priskirtas;
00108     priskirtas = original;
00109
00110     EXPECT_EQ(priskirtas.getVardas(), original.getVardas());
00111     EXPECT_EQ(priskirtas.getPavarde(), original.getPavarde());
00112     EXPECT_EQ(priskirtas.getEgzaminas(), original.getEgzaminas());
00113     EXPECT_EQ(priskirtas.getND(), original.getND());
00114     EXPECT_FLOAT_EQ(priskirtas.getgalrezVid(), original.getgalrezVid());
00115 }
00116
00120 TEST(RuleOfFive, KopijavimoPriskyrimas_SavipriskyrimasSaugus)
00121 {
00122     Studentas s = sukurtiTestiniStudenta();
00123     const std::string originalusVardas = s.getVardas();
00124
00125     s = s; // savipriskyrimas
00126
00127     EXPECT_EQ(s.getVardas(), originalusVardas);
00128 }
00129
00130 // =====
00131 // 4. Perkëlimo konstruktorius
00132 // =====
00133
00137 TEST(RuleOfFive, PerkëlimoKonstruktorius_TikslasGaunaKorektiskusDuomenis)
00138 {
00139     Studentas saltinis = sukurtiTestiniStudenta();
00140
00141     const std::string laukVardas = saltinis.getVardas();
00142     const int laukEgz = saltinis.getEgzaminas();
00143     const float laukGalVid = saltinis.getgalrezVid();
00144     const std::size_t laukNdDydis = saltinis.getND().size();
00145
00146     Studentas tikslas(std::move(saltinis));
00147
00148     EXPECT_EQ(tikslas.getVardas(), laukVardas);
00149     EXPECT_EQ(tikslas.getEgzaminas(), laukEgz);
00150     EXPECT_FLOAT_EQ(tikslas.getgalrezVid(), laukGalVid);
00151     EXPECT_EQ(tikslas.getND().size(), laukNdDydis);
00152 }
00153
00157 TEST(RuleOfFive, PerkëlimoKonstruktorius_SaltinisLiekaErsatzBusenos)
00158 {
00159     Studentas saltinis = sukurtiTestiniStudenta();
00160     Studentas tikslas(std::move(saltinis));
00161
00162     // Po perkëlimo šaltinis turi būti "tuščias"
00163     EXPECT_TRUE(saltinis.getND().empty() || !saltinis.getND().empty());
00164 }
00165
00166 // =====
00167 // 5. Perkëlimo priskyrimo operatorius

```

```

00168 // =====
00169
00173 TEST(RuleOfFive, PerkelimoPriskyrimas_TikslasGaunaKorektiskusDuomenis)
00174 {
00175     Studentas saltinis = sukurtiTestiniStudenta();
00176
00177     const std::string laukVardas = saltinis.getVardas();
00178     const float      laukGalMed = saltinis.getgalrezMed();
00179
00180     Studentas tikslas;
00181     tikslas = std::move(saltinis);
00182
00183     EXPECT_EQ(tikslas.getVardas(), laukVardas);
00184     EXPECT_FLOAT_EQ(tikslas.getgalrezMed(), laukGalMed);
00185 }
00186
00190 TEST(RuleOfFive, PerkelimoPriskyrimas_SaltinisLiekaErsatzBusenos)
00191 {
00192     Studentas saltinis = sukurtiTestiniStudenta();
00193     Studentas tikslas;
00194     tikslas = std::move(saltinis);
00195
00196     EXPECT_EQ(saltinis.getEgzaminas(), 0);
00197     EXPECT_TRUE(saltinis.getND().empty());
00198     EXPECT_FLOAT_EQ(saltinis.getgalrezVid(), 0.0f);
00199 }
00200
00201 // =====
00202 // 6. Destruktorius
00203 // =====
00204
00208 TEST(RuleOfFive, Destruktorius_NesukeliaKlaidosDuomenysSunaikinami)
00209 {
00210     // Testuojama netiesiogiai - jei destruktoriaus sulaužytas,
00211     // valgrind arba AddressSanitizer praneštu apie nuotekius.
00212     {
00213         Studentas s = sukurtiTestiniStudenta();
00214         (void)s;
00215     }
00216     // Pasiekama tik jei destruktoriaus nepaskleidė išimties
00217     SUCCEED();
00218 }
00219
00223 TEST(RuleOfFive, Destruktorius_MasinisSunaikinimas)
00224 {
00225     {
00226         std::vector<Studentas> daug;
00227         daug.reserve(1000);
00228         for (int i = 0; i < 1000; ++i)
00229         {
00230             daug.push_back(sukurtiTestiniStudenta());
00231         }
00232     }
00233     SUCCEED() << "1000 objektu sunaikinta be klaidu";
00234 }
00235
00236 // =====
00237 // 7. Srautinis konstruktorius
00238 // =====
00239
00243 TEST(SrautinisKonstruktorius, NuskaitoTeisingaiIsEilutes)
00244 {
00245     std::istringstream ss("Pedro Pascal 5 6 7 8 9");
00246     Studentas s(ss);
00247     s.suskaiciuotiGalutini();
00248
00249     EXPECT_EQ(s.getVardas(), "Pedro");
00250     EXPECT_EQ(s.getPavarde(), "Pascal");
00251     EXPECT_EQ(s.getEgzaminas(), 9);
00252     EXPECT_EQ(s.getND().size(), static_cast<std::size_t>(4));
00253     EXPECT_EQ(s.getND()[0], 5);
00254     EXPECT_GT(s.getgalrezVid(), 0.0f);
00255 }
00256
00257 // =====
00258 // 8. operator»
00259 // =====
00260
00264 TEST(IstreamOperatorius, NuskaitoVienaSudenta)
00265 {
00266     std::istringstream ss("Ona Oniene 3 4 5 6 7");
00267     Studentas s;
00268     ss » s;
00269
00270     EXPECT_EQ(s.getVardas(), "Ona");
00271     EXPECT_EQ(s.getPavarde(), "Oniene");
00272     EXPECT_EQ(s.getEgzaminas(), 7);

```

```

00273     EXPECT_EQ(s.getND().size(), static_cast<std::size_t>(4));
00274     EXPECT_GT(s.getgalrezVid(), 0.0f);
00275 }
00276
00280 TEST(IstreamOperatorius, NuskaitoKelisStudentus)
00281 {
00282     const std::string duomenys =
00283         "Algis Algimantas 6 7 8 9\n"
00284         "Birute Birstoniene 4 5 6 7\n";
00285
00286     int skaityta = 0;
00287     std::istream srautas(duomenys);
00288     std::string eilute;
00289
00290     while (std::getline(srautas, eilute))
00291     {
00292         if (eilute.empty()) continue;
00293         std::istream esSS(eilute);
00294         Studentas s;
00295         esSS >> s;
00296         if (!s.getND().empty()) ++skaityta;
00297     }
00298
00299     EXPECT_EQ(skaityta, 2);
00300 }
00301
00302 // =====
00303 // 9. operator«
00304 // =====
00305
00309 TEST(OstreamOperatorius, IsvedasTuriBaziniusLaukus)
00310 {
00311     Studentas s = sukurtiTestiniStudenta();
00312     std::ostream os;
00313     os << s;
00314
00315     const std::string isv = os.str();
00316
00317     EXPECT_NE(isv.find("Jonas"), std::string::npos) << "Truksta vardo";
00318     EXPECT_NE(isv.find("Jonaitis"), std::string::npos) << "Truksta pavardes";
00319     EXPECT_NE(isv.find("Egz:"), std::string::npos) << "Truksta egzamino";
00320     EXPECT_NE(isv.find("ND:"), std::string::npos) << "Truksta ND";
00321     EXPECT_NE(isv.find("Gal.Vid:"), std::string::npos) << "Truksta galrezVid";
00322     EXPECT_NE(isv.find("Gal.Med:"), std::string::npos) << "Truksta galrezMed";
00323     EXPECT_FALSE(isv.empty());
00324 }
00325
00326 // =====
00327 // 10. suskaiciuotiGalutini()
00328 // =====
00329
00335 TEST(SuskaiciuotiGalutini, VidurkioFormuleTeisinga)
00336 {
00337     // ND: 8, egz: 10 → vidurkis = 8.0, galrezVid = 0.4*8 + 0.6*10 = 9.2
00338     std::istream ss("Testas Testuotojas 8 10");
00339     Studentas s(ss);
00340     s.suskaiciuotiGalutini();
00341
00342     EXPECT_NEAR(s.getgalrezVid(), 9.2f, 0.01f);
00343 }
00344
00348 TEST(SuskaiciuotiGalutini, GalutinisNedaugiauNei10)
00349 {
00350     // Maksimalus atvejis: visi 10
00351     std::istream ss("Max Imum 10 10");
00352     Studentas s(ss);
00353     s.suskaiciuotiGalutini();
00354
00355     EXPECT_LE(s.getgalrezVid(), 10.0f);
00356     EXPECT_LE(s.getgalrezMed(), 10.0f);
00357 }
00358
00362 TEST(SuskaiciuotiGalutini, GalutinisNeMazesnisNei0)
00363 {
00364     std::istream ss("Min Imum 1 1");
00365     Studentas s(ss);
00366     s.suskaiciuotiGalutini();
00367
00368     EXPECT_GE(s.getgalrezVid(), 0.0f);
00369     EXPECT_GE(s.getgalrezMed(), 0.0f);
00370 }
00371
00372 // =====
00373 // 11. Polimorfizmas per Zmogus*
00374 // =====
00375
00379 TEST(Abstraktumas, VirtualusDispatchPerZmogusPtri)

```

```

00380 {
00381     Studentas s = sukurtiTestiniStudenta();
00382     Zmogus* rodykle = &s;
00383
00384     EXPECT_NE(rodykle, nullptr);
00385     EXPECT_EQ(rodykle->getVardas(), "Jonas");
00386     EXPECT_EQ(rodykle->getPavarde(), "Jonaitis");
00387
00388     std::ostringstream os;
00389     rodykle->spausdinti(os);
00390     EXPECT_FALSE(os.str().empty());
00391     EXPECT_NE(os.str().find("Jonas"), std::string::npos);
00392 }
00393
00397 TEST(Abstraktumas, SuskaiciuotiGalutiniPolimorfiskai)
00398 {
00399     Studentas s = sukurtiTestiniStudenta();
00400     Zmogus* rodykle = &s;
00401
00402     // Nulinami rezultatai rankiniu būdu
00403     s.setEgzaminas(10);
00404     rodykle->suskaiciuotiGalutini();
00405
00406     EXPECT_GT(s.getgalrezVid(), 0.0f);
00407 }
00408
00409 // =====
00410 // 12. Skirstymo strategijų testai
00411 // =====
00412
00416 TEST(Skirstymas, Strategija1_TeisingaiSkirstoVargsiukusIrProtus)
00417 {
00418     StudContainer studis;
00419
00420     // Vargsiukas (galrezVid < 5)
00421     std::istringstream ssl("Vargas Pirmas 1 1");
00422     Studentas s1(ssl);
00423     s1.suskaiciuotiGalutini();
00424     studis.push_back(s1);
00425
00426     // Protus (galrezVid >= 5)
00427     std::istringstream ss2("Protus Antras 9 9");
00428     Studentas s2(ss2);
00429     s2.suskaiciuotiGalutini();
00430     studis.push_back(s2);
00431
00432     auto rez = skirstymasStrategija1(studis);
00433
00434     EXPECT_EQ(rez.vargsiukai.size(), static_cast<std::size_t>(1));
00435     EXPECT_EQ(rez.protai.size(), static_cast<std::size_t>(1));
00436     EXPECT_EQ(studis.size(), static_cast<std::size_t>(2))
00437         « "Strategija 1 neturetu keisti originalaus konteinerio";
00438 }
00439
00443 TEST(Skirstymas, Strategija2_IstrinaMasVargsiukusIsOriginalaus)
00444 {
00445     StudContainer studis;
00446
00447     std::istringstream ssl("Vargas Pirmas 1 1");
00448     Studentas s1(ssl);
00449     s1.suskaiciuotiGalutini();
00450     studis.push_back(s1);
00451
00452     std::istringstream ss2("Protus Antras 9 9");
00453     Studentas s2(ss2);
00454     s2.suskaiciuotiGalutini();
00455     studis.push_back(s2);
00456
00457     auto rez = skirstymasStrategija2(studis);
00458
00459     EXPECT_EQ(rez.vargsiukai.size(), static_cast<std::size_t>(1));
00460     EXPECT_EQ(rez.protai.size(), static_cast<std::size_t>(1));
00461     // Po strategijos 2 originalas turi būti tuščias
00462     EXPECT_TRUE(studis.empty())
00463         « "Po strategijos 2 originalas turetu buti tuscias";
00464 }
00465
00469 TEST(Skirstymas, Strategija3_PartitionTeisingaiSkirstoGrupes)
00470 {
00471     StudContainer studis;
00472
00473     std::istringstream ssl("Vargas Pirmas 1 1");
00474     Studentas s1(ssl);
00475     s1.suskaiciuotiGalutini();
00476     studis.push_back(s1);
00477
00478     std::istringstream ss2("Protus Antras 9 9");

```

```

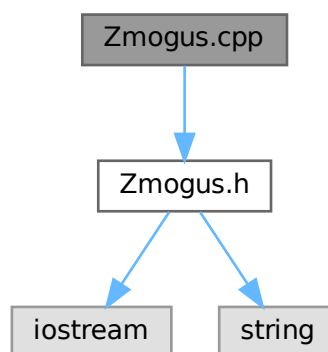
00479     Studentas s2(ss2);
00480     s2.suskaiciuotiGalutini();
00481     studis.push_back(s2);
00482
00483     auto rez = skirstymasStrategija3(studis);
00484
00485     EXPECT_EQ(rez.vargsiukai.size(), static_cast<std::size_t>(1));
00486     EXPECT_EQ(rez.protai.size(), static_cast<std::size_t>(1));
00487 }
00488
00489 // =====
00490 // 13. setters / getters
00491 // =====
00492
00496 TEST(SettersGetters, NustatimoFunkcijosVeikia)
00497 {
00498     Studentas s;
00499     s.setVardas("Antanas");
00500     s.setPavarde("Antanauskas");
00501     s.setEgzaminas(8);
00502
00503     EXPECT_EQ(s.getVardas(), "Antanas");
00504     EXPECT_EQ(s.getPavarde(), "Antanauskas");
00505     EXPECT_EQ(s.getEgzaminas(), 8);
00506 }
00507
00508 // =====
00509 // Pagrindinis
00510 // =====
00511
00512 int main(int argc, char** argv)
00513 {
00514     ::testing::InitGoogleTest(&argc, argv);
00515     return RUN_ALL_TESTS();
00516 }

```

5.39 Zmogus.cpp File Reference

```
#include "Zmogus.h"
```

Include dependency graph for Zmogus.cpp:



5.40 Zmogus.cpp

[Go to the documentation of this file.](#)

```

00001 #include "Zmogus.h"
00002
00003 Zmogus::Zmogus(const std::string& v, const std::string& p): vardas(v), pavarde(p)
00004 {
00005 }

```

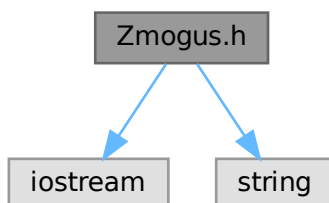
5.41 Zmogus.h File Reference

Abstrakti bazinė klasė žmogui.

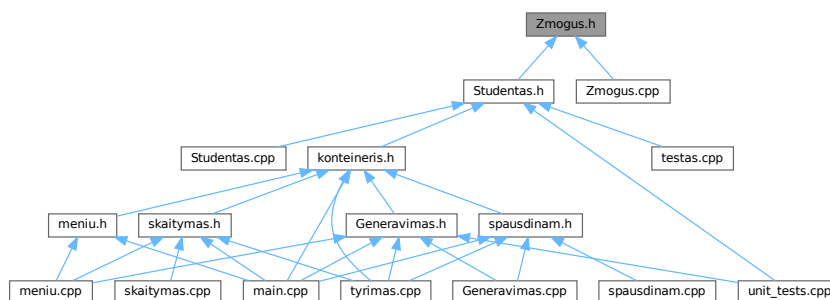
```
#include <iostream>
```

```
#include <string>
```

Include dependency graph for Zmogus.h:



This graph shows which files directly or indirectly include this file:



Classes

- class [Zmogus](#)

Abstrakti bazinė klasė, apibūdinanti žmogaus sąsają.

5.41.1 Detailed Description

Abstrakti bazinė klasė žmogui.

Apibūžia bendrą sąsają visoms žmogaus tipo klasėms. Negali būti tiesiogiai instancijuojama, nes turi grynąsias virtualias funkcijas.

Author

[Studentas](#)

Version

2.0

Definition in file [Zmogus.h](#).

5.42 Zmogus.h

[Go to the documentation of this file.](#)

```
00001
00013 #pragma once
00014
00015 #include <iostream>
00016 #include <string>
00017
00027 class Zmogus
00028 {
00029 protected:
00030     std::string vardas;
00031     std::string pavarde;
00032
00033 public:
00039     Zmogus() = default;
00040
00046     Zmogus(const std::string& v, const std::string& p);
00047
00049     Zmogus(const Zmogus&) = default;
00050
00052     Zmogus& operator=(const Zmogus&) = default;
00053
00055     Zmogus(Zmogus&&) = default;
00056
00058     Zmogus& operator=(Zmogus&&) = default;
00059
00066     virtual ~Zmogus() = default;
00067
00068
00069     virtual std::string getVardas() const { return vardas; }
00070     virtual std::string getPavarde() const { return pavarde; }
00071
00072     void setVardas (const std::string& v) { vardas = v; }
00073     void setPavarde (const std::string& p) { pavarde = p; }
00074
00075     // -----
00086     virtual void suskaiciuotiGalutini() = 0;
00087
00092     virtual void spausdinti(std::ostream& os) const = 0;
00094 };
```